# ksmech

Previsione di meccanismi di reazione come sequenze di passi elementari con token orbitali localizzati e fisica di Klopman–Salem

Documentazione tecnica: struttura del programma, impostazioni, scelte progettuali, risultati dei training e scoperte

Progetto KSMECH (ChemRho)

9 novembre 2026

Stato del codice: ramo `main` (`a023e58`), specifica v10, `SAMPLE_VERSION` 19;
risultati fino al passo 9 della fase P9 del piano open-shell

**Sommario**

KSMECH è un sistema di deep learning che predice i meccanismi delle reazioni organiche come sequenze di passi elementari. Ogni passo non è predetto come un insieme di frecce curve, ma come il cambiamento che produce sulla struttura di Lewis: la *matrice di reazione* $\boldsymbol{R} = \Delta\boldsymbol{B}$ di Dugundji e Ugi, cioè quali legami si formano e si rompono e dove finiscono le coppie di elettroni. Le molecole sono calcolate con il metodo semi-empirico GFN2-xTB, i loro orbitali sono localizzati (IBO) e rappresentati come *token* accanto agli atomi; un transformer a due tracce (token e coppie) riceve nella traccia delle coppie le grandezze d'interazione orbitale dell'equazione di Klopman–Salem che non richiedono una geometria di avvicinamento (il fattore spettrale di gap $G$, il suo analogo a due livelli $g_2$, l'accoppiamento tra centri). Un decodificatore ancorato su un legame che si forma predice $\boldsymbol{R}$; il prodotto è $\boldsymbol{B} + \boldsymbol{R}$, che viene ri-featurizzato per il passo successivo.

Questo documento descrive in dettaglio la struttura del programma (234 moduli Python e circa 55 000 righe, di cui 28 600 nel motore e 26 100 nell'applicazione web), tutte le impostazioni con i loro valori di default e il motivo di ciascuna scelta, la storia delle decisioni (dal 19 ottobre al 9 novembre 2026), i risultati di tutti i training eseguiti (68 run, di cui 62 complete, su una GPU locale AMD e su pod NVIDIA in cloud) e le scoperte fatte lungo il percorso: dalla bocciatura della geometria di contatto, all'equivalenza tra configurazione *general* e *legacy*, al costo di circa 2 punti sul dominio a guscio chiuso dell'addestramento congiunto con i passi radicalici, fino al confondimento hardware scoperto nelle ablazioni finali.

# Indice


**1 Introduzione** **4**
- 1.1 Il problema . . . 4
- 1.2 L'idea centrale . . . 4
- 1.3 Principi guida . . . 4
- 1.4 Cronologia sintetica . . . 5
- 1.5 Organizzazione del documento . . . 6

**2 Fondamenti fisici** **7**
- 2.1 Struttura elettronica: GFN2-xTB . . . 7
- 2.2 Analisi di popolazione . . . 7
- 2.3 Orbitali localizzati . . . 7
- 2.4 L'equazione di Klopman–Salem . . . 8
  - 2.4.1 Perché nessuna geometria di contatto . . . 8
- 2.5 Descrittori della DFT concettuale . . . 9
- 2.6 Matrici legame–elettroni . . . 9
- 2.7 Descrittori sterici . . . 10

**3 Architettura del software** **11**
- 3.1 Organizzazione del repository . . . 11
- 3.2 Mappa dei pacchetti . . . 11
- 3.3 Comandi . . . 12
- 3.4 Dipendenze e ambienti . . . 12
- 3.5 Regole d'ingegneria . . . 13
- 3.6 Test . . . 13

**4 Dati: sorgenti, ingest e leggi di dominio (`ksmech.data`)** **14**
- 4.1 Le sorgenti . . . 14
- 4.2 Canonicalizzazione e schema . . . 14
- 4.3 Il dominio: esclusioni e riparazioni contate . . . 15
- 4.4 Identità di risonanza . . . 16
- 4.5 Split, snapshot congelati e coorti . . . 16
- 4.6 Provenienza degli artefatti . . . 17
- 4.7 Dataset privato . . . 18

**5 Il piano open-shell (fasi P0–P9)** **19**
- 5.1 La regola di tolleranza U20 . . . 19
- 5.2 Spin: tabelle e leggi . . . 20
- 5.3 RMechDB via FlowER (P8) . . . 21
- 5.4 Release B e merge . . . 22
- 5.5 P9: il modello open-shell . . . 22

**6 Calcolo quantistico (`ksmech.qm`)** **23**
- 6.1 Geometria . . . 23
- 6.2 Struttura elettronica e descrittori . . . 23
- 6.3 Spin e molteplicità . . . 24
- 6.4 Batch, cache e indice . . . 24
- 6.5 Accelerazioni del 9 novembre . . . 25

**7 Token orbitali e atomici (`ksmech.orbitals`) 26**
7.1 Localizzazione . . . 26
7.1.1 Configurazione attuale . . . 26
7.1.2 Perché IBO e non Pipek–Mezey . . . 26
7.2 Occupazioni di riferimento e gusci aperti . . . 26
7.3 Descrittori del token orbitale . . . 27
7.4 Il token atomico . . . 27
7.5 Il classificatore diagnostico . . . 28
7.6 Ruoli di donatore e accettore . . . 28
7.7 SOMO, lacune e popolazione SOMO . . . 28

**8 Feature di coppia geometry-free (`ksmech.physics`) 30**
8.1 Il fattore spettrale di gap $G$ . . . 30
8.2 Il gap a due livelli $g_2$ (P7) . . . 30
8.3 Accoppiamento tra centri e allineamento . . . 30
8.4 Il vettore di feature . . . 31
8.5 Dalla coppia di orbitali alla coppia di atomi . . . 32
8.6 Storia: lo stage B e la geometria di contatto (archiviato) . . . 32

**9 Etichette e campioni (`ksmech.arrows`) 33**
9.1 Dalle frecce alla matrice di reazione . . . 33
9.2 La classe parsimoniosa . . . 33
9.3 Chiusura per simmetria (schema 17) . . . 33
9.4 Augmentation di risonanza . . . 34
9.5 ReactMech: etichette senza frecce . . . 34
9.6 Il campione e lo store . . . 35

**10 Il modello (`ksmech.model`) 36**
10.1 Configurazione . . . 36
10.2 Embedding dei token . . . 37
10.3 La traccia delle coppie (encoder denso) . . . 37
10.4 Lo strato dell'encoder denso . . . 37
10.5 Encoder alternativi . . . 38
10.6 Il decodificatore delle modifiche . . . 38
10.7 Loss . . . 39

**11 Training (`ksmech.train`) 40**
11.1 Configurazione . . . 40
11.2 Batching a budget di celle . . . 40
11.3 Caricamento e filtri . . . 40
11.4 Gestione dei valori non finiti . . . 41
11.5 Metriche, checkpoint e directory della run . . . 41
11.6 Hardware e tempi delle run . . . 41

**12 Decodifica e valutazione (`ksmech.eval`) 43**
12.1 Il beam decoder . . . 43
12.2 Riferimenti . . . 43
12.3 Confronti appaiati, coorti, fette . . . 44
12.4 Importanza e traccia degli embedding . . . 44

**13 Risultati sperimentali** **45**
13.1 Riferimenti non addestrati e ranker GBDT . . . 45
13.1.1 Il floor fisico sulle ancore . . . 45
13.1.2 Ranker della prima freccia . . . 46
13.2 I primi training (26–28 ottobre) . . . 46
13.3 Il pilota ReactMech e la valutazione incrociata . . . 47
13.4 P0b: GENERAL contro LEGACY . . . 48
13.5 I ritorni del pilota ReactMech (P5) . . . 48
13.6 Stabilità del riferimento GBDT . . . 49
13.7 Release A: P4 passo 7 e ablazione dei metalli . . . 49
13.8 P9 passo 8: la porta U20 e l'addestramento congiunto . . . 49
13.9 P9 passo 9: le ablazioni . . . 50
13.10 Curve di apprendimento . . . 51
13.11 Importanza per permutazione degli input . . . 52

**14 Scoperte e lezioni** **54**
14.1 Sulla fisica . . . 54
14.2 Sui dati e sulle etichette . . . 54
14.3 Sul training e sulla valutazione . . . 55
14.4 Sull'ingegneria . . . 55

**15 Applicazione web ChemRho, client e server MCP** **56**
15.1 Struttura a strati . . . 56
15.2 Calcolo, chimica e predittori . . . 56
15.3 Funzioni dell'interfaccia . . . 57
15.4 Client HTTP e server MCP . . . 57

**16 Lavoro futuro e questioni aperte** **59**
16.1 Questioni aperte immediate . . . 59
16.2 I token orbitali servono al modello? . . . 59
16.3 Reti di reazione . . . 59

**A Codici di motivo** **61**

**B Versioni** **63**

**C Comandi tipici** **64**

**D Glossario** **65**

# 1 Introduzione

## 1.1 Il problema

Un meccanismo di reazione organica è una sequenza (più precisamente una rete) di passi elementari, ognuno dei quali attraversa un solo stato di transizione. La notazione con cui i chimici lo scrivono, le frecce curve, descrive lo spostamento di coppie (o di singoli) elettroni. KSMECH apprende la distribuzione condizionata

$$P_\theta(\boldsymbol{R} \mid \mathcal{M})$$

di un passo elementare $\boldsymbol{R}$ dato il *pool* corrente $\mathcal{M}$, cioè il multiinsieme di tutte le specie presenti (reagenti, intermedi, sottoprodotti, spettatori) scritto come un'unica matrice legame–elettroni $\boldsymbol{B}$. Il modello è quindi una *policy*: definita su qualunque pool, definisce l'intera rete di reazione e non una singola traiettoria. Il prodotto di un passo è $\boldsymbol{B}' = \boldsymbol{B} + \boldsymbol{R}$; nessuna molecola viene mai generata da zero, la massa si conserva per costruzione e la carica con un solo vincolo lineare.

Le frecce curve *non* sono il bersaglio della previsione: sono una lettura di $\boldsymbol{R}$, ricostruita dopo la previsione insieme alle coppie di orbitali che guidano il passo. Tre ragioni motivano questa scelta (spec. v8, 23 ottobre 2026):

1. la decomposizione di $\boldsymbol{R}$ in frecce non è unica e l'ambiguità è fisica: quando un legame è ionico nella funzione d'onda (O–Li di un alcossido, N–P di una betaina di Mitsunobu, C–Cl di un intermedio tetraedrico) la coppia del legame e il doppietto dell'atomo sono lo stesso orbitale localizzato, quindi nessuna etichetta a livello orbitale può dire quale dei due «si è mosso»; $\boldsymbol{R}$ è lo stesso in entrambi i casi;
2. $\boldsymbol{R}$ è ciò che il passo *fa*: il prodotto è $\boldsymbol{B} + \boldsymbol{R}$ e la conservazione della carica è il vincolo $\sum_i R_{ii} + 2\sum_{i<j} R_{ij} = 0$;
3. $\boldsymbol{R}$ è sparso e piccolo: su PMechDB un passo cambia da uno a otto ordini di legame, sempre di $\pm 1$, e gli elettroni non leganti di al più due atomi, di $\pm 2$.

## 1.2 L'idea centrale

L'equazione di Klopman–Salem scrive l'energia d'interazione al secondo ordine tra due frammenti come somma su coppie di un orbitale pieno e uno vuoto: esattamente gli oggetti che una freccia curva nomina. KSMECH mette quindi la rappresentazione nel linguaggio della teoria delle perturbazioni:

- le molecole sono rappresentate come due famiglie di token, **atomi** e **orbitali localizzati** (IBO, calcolati da GFN2-xTB);
- un token orbitale porta solo descrittori fisici continui (occupazione, pesi sui centri, quote $s/p/d$, carattere $\sigma/\pi$ come numero, popolazione di sovrapposizione, decomposizione spettrale sugli orbitali canonici), mai un'etichetta da chimico;
- la **traccia delle coppie** del transformer è inizializzata con le grandezze d'interazione orbitale che non richiedono una geometria d'avvicinamento: il fattore spettrale di gap $G$ del termine pieno–vuoto, il suo analogo a due livelli $g_2$ e, dentro una molecola, l'accoppiamento di Wolfsberg–Helmholz tra i centri e l'allineamento stereoelettronico; ogni testa d'attenzione le legge come bias appreso;
- la previsione è fatta sugli **atomi**, perché il cambiamento di una struttura di Lewis è definito sugli atomi ed è lo stesso a qualunque orbitale localizzato lo si attribuisca.

## 1.3 Principi guida

Lo sviluppo è governato da un piccolo numero di principi che ricorrono in tutte le decisioni documentate in `docs/decisions.md`:

**Nessuna tassonomia del chimico come input.**
Tipi di orbitale ($\sigma$, $\pi$, doppietto, $\sigma^*$, ...), ibridazione, aromaticità e gruppi funzionali sono calcolati e salvati con `model_input=False`: servono a test (`pytest -m diagnostic`), diagnostica di allineamento, interfaccia e ablazioni «aggiunte come input», mai al modello. Il carattere orbitale deve generalizzare tra elementi: $\sigma^*$(C–Cl), $\sigma^*$(C–Br) e $\sigma^*$(C–I) devono apparire come lo stesso tipo di accettore a energie diverse.

**Nessuna regola arbitraria nel percorso del modello.**
Ogni regola che decide cosa il modello vede, cosa può predire o come si costruiscono i bersagli è una di tre cose: una legge fisica o una contabilità esatta (conservazione di elettroni e carica, conteggi non negativi, ordini di legame tra 0 e 3); una grandezza calcolata dal metodo quantistico, che il modello pesa come input; oppure un'opzione dietro un flag di configurazione il cui default è giustificato da un'ablazione datata. Liste di elementi, percentili del dataset e soglie «go/no-go» sono sostituite da una delle tre. Questo principio è stato chiesto esplicitamente dall'utente il 28–29 ottobre 2026 (fase P0a).

**La cura dei dati è contata.**
Le regole che tengono un passo fuori dai dati (riparazioni di errori di disegno, esclusioni di specie fuori dominio) sono *curation*, non regole del modello: una legge per fenomeno, mai condizionata da elemento o parità, e ogni passo toccato è contato con un codice di motivo e riportato.

**Nessun fallimento silenzioso.**
Ogni molecola saltata è registrata con un codice e contata; i conteggi sono riportati a fine run.

**Ablazione come metodo.**
Ogni scelta che si afferma importante è accoppiata a un flag che la rimuove; il modello è riaddestrato con e senza, con tre seed, e la differenza confrontata con il suo rumore (test di McNemar appaiato).

**Provenienza e riproducibilità.**
Ogni artefatto porta un'intestazione di provenienza (hash del codice, hash della specifica, hash degli input, configurazione); i loader rifiutano gli artefatti obsoleti (`STALE_ARTIFACT`) e training e valutazione rifiutano build da un albero sporco.

## 1.4 Cronologia sintetica

Il progetto è stato sviluppato in circa tre settimane. La tabella 1.1 riassume le tappe principali; i capitoli successivi le descrivono in dettaglio.

**Tabella 1.1:** Cronologia del progetto (date del registro delle decisioni).

| Data | Tappa |
|---|---|
| 19 ott | Scaffold, livello dati (PMechDB), split per molecola, livello QM (GFN2-xTB), primi token orbitali (Pipek–Mezey). |
| 20 ott | Descrittori continui al posto dei tipi orbitali («Prompt 3b»); spettri completi per $G$; frecce come modifiche della matrice BE; modulo di fisica e «gate» di Klopman–Salem non addestrato; kernel di contatto orientato (fase 5b). |
| 21 ott | Integrali di risonanza a livello AO, testa congelata in eV, esclusioni di dominio; revisione del codice (`docs/review_2026-10-21.md`) e piano di correzione; intestazioni di provenienza. |
| 22 ott | Split per molecola come default (`random_leaky` per lo split per passo); chiave di risonanza; IBO come localizzazione di default (spec v7.2); store dei campioni a shard. |
| 23 ott | Ranker LightGBM come riferimento veloce; **spec v8**: il bersaglio diventa la matrice di reazione, non le frecce. |

| Data | Tappa |
|---|---|
| 24 ott | Etichette di edit (classe parsimoniosa), copertura 99,62% dei passi di training PMechDB. |
| 25 ott | Chiave sigma come identità di risonanza (spec v8.1); **spec v9**: lo «stage B» con geometria di contatto esce da `main` (archiviato in `archive/stage-b`); modello, trainer, floor fisico; tre encoder; workbench molecolare nell'app web. |
| 26 ott | Primo training (flex, RunPod RTX 3090 Ti); augmentation di risonanza; classi di etichette chiuse per simmetria; pilota ReactMech da 10 000 passi; dati QMugs per il pre-addestramento. |
| 27 ott | Decoder dei passi nel motore (`ksmech-eval decode`); app web in sotto-pacchetti; client HTTP, CLI e server MCP; account; OCR e wiki; marchio ChemRho. |
| 28 ott | PMechDB e ReactMech in un'unica checkout, run multi-sorgente; **piano open-shell** (fasi P0–P9) concordato. |
| 29 ott | P0a: generalità e leggi di dominio unificate (`SAMPLE_VERSION` 18); P0b: coorte `closed_base`, confronto appaiato, preset; dataset privato su Hugging Face. |
| 30 ott | Decisione P0b: GENERAL contro LEGACY; P1 (riparazioni di disegno). |
| 31 ott | P2 (scansione degli stati di spin, tabelle `spin_theory_v1`/`spin_table_v1`), P2b (occupazioni di riferimento a plateau), P3 (open shell nei passi polari). |
| 1–4 nov | `spin_table_v2`; P4 (record QM etichettati per spin, metalli Cu(II)/Fe(III)); P5 (ritorni del pilota ReactMech, audit dativo). |
| 5–6 nov | P6 (contabilità degli elettroni, chiave d'identità v2), P7 (SOMO donatori e lacune, $g_2$, canali own-pair e di ricombinazione), **spec v10 approvata**, P8 (RMechDB via FlowER, Release B, `SAMPLE_VERSION` 19), codice P9 (testa open-shell, schermo di decodifica); merge in `main`. |
| 8 nov | Training P4 passo 7 (Release A, ablazione metalli) e P9 passo 8 (`pm_control`, `joint`, `pm_v19`) su 2×RTX 3090. |
| 9 nov | P9 passo 9 (ablazioni, RTX 5090); importanza per permutazione; tblite su OpenBLAS e ottimizzatore ANC/RFO per il QM. |

## 1.5 Organizzazione del documento

Il capitolo 2 richiama i fondamenti fisici. Il capitolo 3 descrive l'architettura del software. I capitoli 4–9 seguono la pipeline dei dati: sorgenti e leggi di dominio, calcolo quantistico, token, feature di coppia, etichette. I capitoli 10–12 descrivono il modello, il training e la valutazione. Il capitolo 13 raccoglie tutti i risultati sperimentali e il capitolo 14 le scoperte e le lezioni apprese. Seguono l'applicazione web (capitolo 15), il lavoro futuro (capitolo 16) e le appendici (codici di motivo, versioni, comandi).

# 2 Fondamenti fisici

Questo capitolo riassume la teoria su cui il modello è costruito (parte I della specifica `docs/ksmech_architecture.tex`, v10). Serve a leggere le scelte dei capitoli successivi.

## 2.1 Struttura elettronica: GFN2-xTB

Lo stato fondamentale di una molecola a guscio chiuso è approssimato da un determinante di Slater di orbitali molecolari (MO) $\psi_k = \sum_\mu C_{\mu k}\chi_\mu$, soluzione del problema di Roothaan $\boldsymbol{FC} = \boldsymbol{SC\epsilon}$. Due proprietà sono usate ovunque:

- **Koopmans**: $-\epsilon_{\text{HOMO}}$ e $-\epsilon_{\text{LUMO}}$ approssimano energia di ionizzazione e affinità elettronica, quindi le energie orbitali sono la scala naturale della forza di donatori e accettori;
- **Brillouin**: a convergenza $F_{kv} = 0$ tra occupati e virtuali; dentro una molecola convergente non resta nulla da «perturbare» al primo ordine. Per questo la teoria delle perturbazioni intermolecolare si applica a frammenti *separati*, e il termine orbitale valutato dentro una molecola è identicamente nullo.

Il metodo è GFN2-xTB (Bannwarth, Ehlert, Grimme 2019), tight-binding semi-empirico con base minima di valenza (STO-$n$G; $1s$ su H, $ns, np$ sugli elementi del gruppo principale, uno shell $d$ di polarizzazione su Si, P, S, Cl, Br, I), elettrostatica isotropa e anisotropa, dispersione D4 e solvatazione implicita ALPB (acqua). Poiché la base è minima, il numero di MO è pari al numero di AO di valenza, non ci sono orbitali di core e ogni virtuale ha una lettura semplice come controparte antilegante o vuota di un occupato. Esporta $\boldsymbol{S}$, $\boldsymbol{F}$, $\boldsymbol{C}$, $\boldsymbol{\epsilon}$, le occupazioni, le cariche di Mulliken e la mappa AO→atomo con il momento angolare di ogni AO. La struttura dell'Hamiltoniana di ordine zero, di tipo Hückel esteso ($H^{(0)}_{\mu\nu} \propto \frac{1}{2}K(h_\mu + h_\nu)S_{\mu\nu}$), giustifica l'approssimazione di Wolfsberg–Helmholz usata per gli integrali di risonanza.

I gusci aperti sono calcolati *ristretti*, con occupazioni frazionarie e senza polarizzazione di spin. GFN2-xTB applica una temperatura elettronica (smearing di Fermi); per molecole con gap HOMO–LUMO evanescente restano occupazioni $0 < n_k < 2$: tali molecole sono marcate (`fermi_smeared`) e non forzate.

## 2.2 Analisi di popolazione

La popolazione di Mulliken di un singolo orbitale sull'atomo $A$ e la popolazione di sovrapposizione tra due atomi sono

$$Q_A^{(k)} = \sum_{\mu\in A}\sum_{\nu} C_{\mu k}S_{\mu\nu}C_{\nu k}, \qquad \Pi_{AB}^{(k)} = 2\sum_{\mu\in A}\sum_{\nu\in B} C_{\mu k}S_{\mu\nu}C_{\nu k}. \tag{2.1}$$

Il segno di $\Pi$ dice se l'orbitale è legante ($\Pi > 0$) o antilegante ($\Pi < 0$) tra $A$ e $B$. Queste sono le uniche grandezze usate per dire che cosa un orbitale *è*.

## 2.3 Orbitali localizzati

Il determinante è invariante per rotazioni unitarie degli occupati tra loro (e dei virtuali tra loro). La localizzazione sceglie, dentro lo stesso sottospazio, la rotazione che rende ogni orbitale il più possibile locale: è un cambio di base, non un'approssimazione e non un'assunzione chimica.

Pipek–Mezey massimizza $\mathcal{P}_p = \sum_r \sum_A (Q_A^{(r)})^p$ con $p = 2$ e popolazioni di Mulliken; ha un difetto noto che si è manifestato nel progetto: nello spazio $\pi$ di un anello aromatico il funzionale è quasi piatto, le soluzioni tipo Kekulé e delocalizzate hanno quasi lo stesso valore, le iterazioni raggiungono il limite e gli orbitali restituiti sono un punto arbitrario su un plateau. Gli *intrinsic bond orbitals*

(IBO, Knizia 2013) lo risolvono con popolazioni IAO ed esponente $p = 4$; in una base minima di valenza come quella di GFN2-xTB le IAO si riducono agli AO ortogonalizzati simmetricamente, $\boldsymbol{S}^{1/2}\boldsymbol{C}$, cioè popolazioni di Löwdin. KSMECH usa IBO ($p = 4$, Löwdin) separatamente sui blocchi occupato, virtuale e singolarmente occupato, a partire da una fattorizzazione di Cholesky con pivot della matrice densità di ciascun blocco (deterministica e già localizzata). Il benzene converge in poche decine di sweep a tre legami $\pi$ a due centri (una delle due strutture di Kekulé, scelta dalla partenza deterministica).

**Decomposizione spettrale.** Un orbitale localizzato $\boldsymbol{l}_r = \sum_k U_{kr}\boldsymbol{c}_k$ non è autofunzione di $\boldsymbol{F}$; il suo contenuto esatto sugli autostati è dato dai pesi

$$p_{rk} = U_{kr}^2, \qquad \sum_k p_{rk} = 1, \qquad E_r = \sum_k p_{rk}\epsilon_k. \tag{2.2}$$

Qualunque espressione di secondo ordine con un denominatore energetico si scrive come somma su coppie canoniche pesate da $p_{rk}p_{sm}$. L'insieme delle energie canoniche non dipende dalla localizzazione: tre LMO $\pi$ di un anello, qualunque struttura di Kekulé sia stata scelta, portano peso totale esattamente uno su ciascuno dei tre orbitali $\pi$ canonici. È ciò che rende i sistemi aromatici e coniugati non problematici.

## 2.4 L'equazione di Klopman–Salem

Per due frammenti chiusi $A$ e $B$ che si avvicinano, l'energia d'interazione al secondo ordine è

$$\Delta E = \underbrace{-\sum_{a,b}(q_a + q_b)\beta_{ab}S_{ab}}_{\text{(1) repulsione pieno–pieno}} + \underbrace{\sum_{k<l}\frac{Q_kQ_l}{\varepsilon R_{kl}}}_{\text{(2) Coulomb}} + \underbrace{\sum_{r\in\mathrm{occ}(A)}\sum_{s\in\mathrm{virt}(B)}\frac{2\big(\sum_{a,b}c_{ra}c_{sb}\beta_{ab}\big)^2}{E_r - E_s} + (A \leftrightarrow B)}_{\text{(3) interazione orbitale pieno–vuoto}}. \tag{2.3}$$

Il termine (1) è il contenuto quantitativo dell'ingombro sterico, il (2) il controllo di carica (domina a gap grande, reagenti «hard»), il (3) il controllo orbitale (domina a gap piccolo, reagenti «soft»; ristretto alla sola coppia HOMO/LUMO è la teoria FMO di Fukui). Con l'approssimazione di Wolfsberg–Helmholz $\beta_{ab} = KS_{ab}\frac{1}{2}(H_{aa} + H_{bb})$, $K \approx 1{,}75$, e con orbitali localizzati riespansi sugli autostati, il termine (3) di una coppia $(r, s)$ si fattorizza:

$$\Delta E^{(3)}_{rs} \approx \big(K\bar{F}_{rs}S_{rs}\big)^2 \sum_{k\in\mathrm{occ}}\sum_{m\in\mathrm{virt}}\frac{2\,p_{rk}p_{sm}}{\epsilon_k - \epsilon_m} =: \big(K\bar{F}_{rs}S_{rs}\big)^2\,G(r,s), \tag{2.4}$$

il prodotto di un *accoppiamento* che dipende da come i due lobi si sovrappongono alla geometria d'avvicinamento e di un *fattore spettrale di gap* $G$ che dipende solo da energie canoniche e pesi di localizzazione.

### 2.4.1 Perché nessuna geometria di contatto

Dentro una molecola convergente il termine (3) è identicamente zero (Brillouin): la geometria dei reagenti non è la geometria reattiva. Una catena che cicla deve prima ripiegarsi, esattamente come due molecole devono diffondere insieme. In entrambi i casi la sovrapposizione che decide la reattività è quella *al contatto*, nella posa che la reazione adotta, e nessun calcolo sui reagenti separati la fornisce. Inoltre i termini (1) e (3) sono entrambi del secondo ordine nella sovrapposizione, il loro rapporto non ha minimo lungo la coordinata d'avvicinamento, e la distanza di contatto è fissata dalla repulsione di scambio a corto raggio che l'espansione trascura. Un accoppiamento valutato a una geometria di contatto è quindi un modello dell'avvicinamento, non una proprietà dei reagenti.

**Scoperta – la geometria di contatto peggiora la fisica.** Il progetto ha costruito e misurato esplicitamente la via alternativa («stage B», spec v8.1): energia libera di contatto configurazionale per ogni coppia di orbitali, i tre termini KS al contatto, una testa congelata. Non addestrati, i termini di contatto classificavano la prima freccia di un passo *peggio* del solo $G$ (PMechDB, split per molecola, test top-1: circa 3% per la testa congelata, 14,3% per $\mathrm{KS}_3 + \mathrm{KS}_2$, 20,4% per $G$). Il 25 ottobre lo stage B è uscito da `main` (spec v9) ed è archiviato con il suo codice nel ramo `archive/stage-b`.

Il modello conserva quindi solo il fattore intrinseco alle due orbitali, $G$, per ogni coppia (intra e intermolecolare), e i descrittori della posa dei reagenti per le coppie interne a una molecola.

## 2.5 Descrittori della DFT concettuale

Potenziale chimico, durezza ed elettrofilicità: $\mu \approx \frac{1}{2}(\epsilon_H + \epsilon_L)$, $\eta \approx \frac{1}{2}(\epsilon_L - \epsilon_H)$, $\omega = \mu^2/2\eta$. Le funzioni di Fukui condensate esistono in due famiglie: alle differenze finite su $N \pm 1$ ($\tilde{f}_A^{\pm}$, includono il rilassamento dell'intera densità) e a orbitali congelati ($f_A^+ = Q_A^{(\mathrm{LUMO})}$, $f_A^- = Q_A^{(\mathrm{HOMO})}$, la forma dell'orbitale di frontiera, che è la grandezza da cui dipende il termine (3)). Il descrittore duale $\Delta f_A = f_A^+ - f_A^-$ separa siti elettrofili e nucleofili. La generalizzazione ai $K$ orbitali più vicini al gap dà i pesi di frontiera per atomo $\omega_{A,k} = Q_A^{(k)}$ (HOMO−1, HOMO, LUMO, LUMO+1), che portano l'informazione regiochimica (orto/para contro meta) che nessun singolo orbitale localizzato può portare.

## 2.6 Matrici legame–elettroni

Una struttura di Lewis di $n$ atomi è la matrice simmetrica $\boldsymbol{B} \in \mathbb{N}^{n\times n}$ (Dugundji–Ugi): $B_{ij}$ è l'ordine di legame formale, $B_{ii} = N_i$ il numero di elettroni non leganti dell'atomo $i$. La carica formale segue: $Z_i^{\mathrm{val}} - B_{ii} - \sum_{j\neq i} B_{ij}$. Una freccia curva è la modifica più piccola:

- doppietto su $i \to$ atomo $j$: $R_{ii} = -2$, $R_{ij} = R_{ji} = +1$;
- legame $(i,k) \to$ atomo $j$ con il nuovo legame su $i$: $R_{ik} = -1$, $R_{ij} = +1$;
- legame $(i,k) \to$ doppietto su $k$ (gruppo uscente): $R_{ik} = -1$, $R_{kk} = +2$.

Un passo elementare è la somma delle matrici delle sue frecce applicate insieme. $N_i$ conta anche gli elettroni singoli: un centro radicalico ha $N_i$ dispari e il numero di elettroni spaiati $u$ di un disegno è il numero di atomi con $N_i$ dispari (alzato solo dove la tabella delle specie la tiene ad alto spin, mai letto dalla percezione dei radicali di RDKit, che darebbe due radicali a $[\mathrm{Br}^+]$, $[\mathrm{OH}^+]$ o a un $\mathrm{CH}_2$ di singoletto). Una semi-freccia sposta un elettrone: l'omolisi di $(i,k)$ è $R_{ik} = -1$, $R_{ii} = R_{kk} = +1$. Una sorgente di passi a un elettrone scrive $R_{ii} \in \{-2,-1,0,+1,+2\}$; le sorgenti polari mantengono $R_{ii} \in \{-2,0,+2\}$. È una regola di etichetta della sorgente, mai un input del modello.

**La legge del flusso elettronico (spec v10).** Le frecce spostano elettroni solo tra atomi legati. Si uniscano in componenti connesse gli atomi il cui ordine di legame cambia. Il flusso di $\boldsymbol{R}$ è *bilanciato* quando ogni atomo il cui $N$ cambia sta su un legame cambiato e ogni componente $K$ conserva i suoi elettroni,

$$\sum_{i\in K} R_{ii} + 2 \sum_{(i,j)\in K,\, i<j} R_{ij} = 0. \tag{2.5}$$

Ogni passo polare è bilanciato; così ogni omolisi, ricombinazione e trasferimento di atomo. Quattro esiti sono nominati e contati, e nessuno è un passo di training: il *passo nullo* (`NULL_STEP`, $\boldsymbol{R} = 0$), lo *spostamento non coniugato* (`NONCONJUGATED_SHIFT`: due lati con la stessa chiave d'identità ma nessun flusso bilanciato), il *trasferimento di singolo elettrone* (`SET_STEP`, solo voci atomiche) e il *trasferimento elettronico dissociativo* (`DISSOCIATIVE_ET_STEP`, voci di legame con flusso sbilanciato in ogni membro della classe).

## 2.7 Descrittori sterici

Il termine repulsivo (1) richiede la sovrapposizione degli orbitali pieni alla geometria reattiva, che nessun calcolo sul frammento fornisce. Il volume sepolto percentuale $V_A^{\mathrm{bur}}$ (sfera di raggio 3,5 Å centrata sull'atomo, idrogeni inclusi) è usato come proxy geometrico: un descrittore di input, non un prior sulla reattività.

# 3 Architettura del software

## 3.1 Organizzazione del repository

Il progetto è un pacchetto Python (layout `src/`, build `uv_build`, Python fissato a $\geq 3.11, < 3.12$ per tenere prevedibile l'insieme di wheel torch/rdkit/tblite) gestito con `uv`. Directory principali: `src/ksmech/` (motore e app web), `tests/`, `apps/ui/` (frontend), `packages/` (pacchetto di visualizzazione `@ksmech/viz`), `tools/ocr/` (ambiente isolato per l'OCR), `content/wiki/` (wiki), `scripts/` (analisi e sonde), `docs/` (specifica, registro delle decisioni, schemi, piani), `data/` (dati grezzi, cache, dati elaborati, modelli; non versionato).

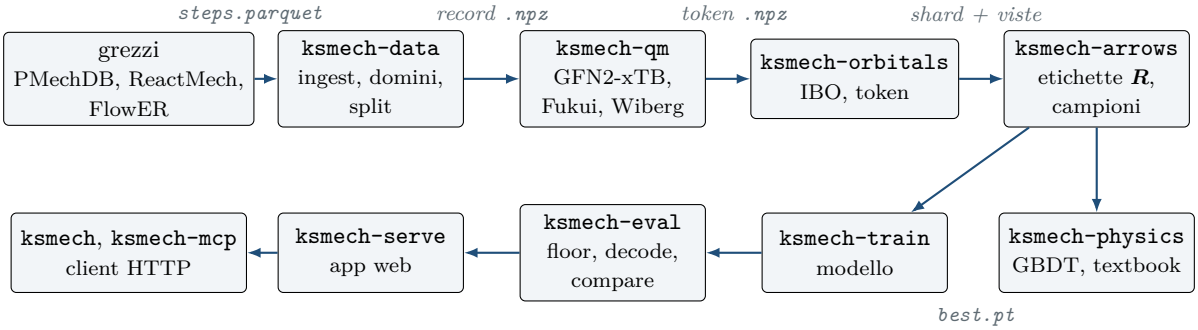



**Figura 3.1:** La pipeline: ogni comando legge gli artefatti del precedente, verificandone la provenienza.

## 3.2 Mappa dei pacchetti

**Tabella 3.1:** Sotto-pacchetti del motore (`src/ksmech`), escluso `serve`. Righe di codice Python.

| Pacchetto | File | Righe | Contenuto |
|---|---|---|---|
| `data` | 23 | 6 265 | Loader delle sorgenti (`pmechdb`, `reactmech`, `rmechdb`, `mech_uspto`, `qmugs`), validazione e ingest, leggi di dominio (`domain.py`, 955 righe: 13 codici, 16 classi, `DomainPolicy`), ridisegni di spin (`pairing.py`), spin (`spin.py`, `spin_table.py`, `spin_theory.py`), risonanza e chiavi d'identità, split, audit di leakage, statistiche, indagine open-shell, provenienza. |
| `qm` | 19 | 4 161 | `QMConfig` e calcolatore GFN2, ottimizzatori (L-BFGS-B, ANC/RFO), embedding, batch con manifest e tombstone, record e cache, tag di spin, Fukui, indici di Wiberg, $V^{\mathrm{bur}}$, layout AO, preload OpenBLAS, single point QMugs, indice SQLite, rilevatore del gruppo puntuale (704 righe). |
| `orbitals` | 10 | 3 262 | Localizzazione IBO/PM, descrittori, classificatore diagnostico, pesi di frontiera, token (`tokens.py`, 1 068 righe: `CandidateConfig`, esclusioni, lacune), build del dataset, chiralità, gruppi funzionali. |

| Pacchetto | File | Righe | Contenuto |
|---|---|---|---|
| `arrows` | 16 | 5 134 | Frecce come modifica BE, allineamento diagnostico, `ReactionMatrix` ed etichette (`delta_be.py`), disegni di risonanza, chiusura per simmetria, struttura di Lewis, indice globale dei token, campioni (`steps.py`, 1 106 righe), store a shard, collazione dei batch, round trip, H mobili di ReactMech. |
| `physics` | 10 | 2 166 | Parametri (`K`, `BETA_INTER`), $G$ e $g_2$, accoppiamento WH, feature di coppia geometry-free, flag di coppia, statistiche, ranker LightGBM, casi da manuale. |
| `model` | 10 | 1 851 | `ModelConfig` e preset, input non appresi, encoder denso/flex/sparse, decodificatore, loss, beam decoder. |
| `train` | 4 | 890 | `TrainConfig`, ciclo di training, batching a budget di celle, CLI. |
| `eval` | 10 | 2 197 | Floor, decodifica, confronto appaiato, importanza per permutazione, coorti, pinning, traccia degli embedding come pagina HTML. |
| `client` | 8 | 1 831 | Client HTTP `KsmechClient` e CLI `ksmech`. |
| `mcp` | 9 | 768 | Server MCP (FastMCP) sopra il client. |
| `processes.py` | 1 | 49 | Pool *spawned* con un thread BLAS per processo. |
| Totale motore | | 28 577 | (l'app web `serve`: 113 moduli, 26 101 righe; capitolo 15) |

## 3.3 Comandi

**Tabella 3.2:** Punti d'ingresso (`[project.scripts]` di `pyproject.toml`).

| Comando | Sottocomandi principali |
|---|---|
| `ksmech-data` | `ingest`, `stats`, `open-shell-survey`, `freeze-sample`, `leakage-audit`, `provenance` |
| `ksmech-qm` | `run`, `qmugs`, `bond-index`, `index refresh\|stats` |
| `ksmech-orbitals` | `build`, `show`, `fg-vocab` |
| `ksmech-arrows` | `roundtrip`, `align`, `samples`, `edit-labels`, `freeze-splits` |
| `ksmech-physics` | `gbdt`, `textbook` (anche `-radical`) |
| `ksmech-train` | `fit` |
| `ksmech-eval` | `floor`, `decode`, `compare`, `importance`, `pin`, `cohort`, `embedding` |
| `ksmech-serve` | server web e amministrazione (utenti, OCR, re-key, export) |
| `ksmech` / `ksmech-mcp` | client HTTP / server MCP |

## 3.4 Dipendenze e ambienti

Dipendenze del motore: `rdkit`, `numpy`, `scipy`, `pandas`, `pyarrow`, `tblite`, `morfeus-ml`, `tqdm`, `rich`, `lightgbm>=4.7.0` (aggiunto su richiesta esplicita dell'utente quando il GBDT in numpy fu ucciso per memoria; la regola «preferire numpy» è stata poi allentata dall'utente il 29 ottobre). `hydra-core` è dichiarato ma inutilizzato: ogni configurazione è una dataclass congelata esposta via argparse. Gruppi: `dev` (pytest, ruff, pre-commit, pytest-xdist), `serve` (FastAPI, uvicorn, SQLModel, pydantic…), `client`, `mcp`, `rocm` (default: torch 2.14.1+rocm7.2 con triton-rocm 3.8.0) e `cuda` (torch 2.14.1 cu130), in conflitto tra loro. Il lock copre solo linux x86_64.

**Hardware.** Sviluppo e piccoli training su una AMD Radeon RX 6600 XT (8 GB, gfx1032) con ROCm 7.2: gfx1032 non ha kernel rocBLAS propri e serve `HSA_OVERRIDE_GFX_VERSION=10.3.0` (i kernel gfx1030); senza, `torch.cuda.is_available()` è vero ma il primo kernel va in segmentation fault (exit 139), quindi `ksmech-train` imposta l'override da sé quando rileva un target RDNA2 senza kernel. Prodotti di matrici: 5,6 TFLOPS in float32, 8,0 in float16. I training di confronto girano su pod NVIDIA noleggiati (RunPod, vast.ai): RTX 3090 Ti, RTX 5080, RTX 5090, 2×RTX 3090. CPU locale Ryzen 7 2700X (16 thread), 16 GB di RAM dopo la rimozione di un modulo difettoso; regola: al più 15 processi pesanti.

## 3.5 Regole d'ingegneria

- Mai inventare una colonna di dataset: aprire il file reale e documentarne lo schema in `docs/data_schema.md`.
- Ogni funzione di featurizzazione ha un test sulle 10 molecole fisse (etano, etanolo, acetone, acetaldeide, acetato di metile, clorometano, metilammina, acetonitrile, benzene, acroleina) e, se una fase open-shell la cambia, sulla lista `OPEN_SHELL` (radicali, superossido, anione radicale del benzene, $O_2$ tripletto, $Br^{\bullet}$, $^{\bullet}OH$, NO, Cl[C]Cl, [C]=O, [N]=[N], ozono, $CuCl_2$, $FeBr_4^-$).
- Nessun fallimento silenzioso; cache QM su disco; `pytest` prima di dichiarare finito un lavoro; ogni scelta non ovvia in `docs/decisions.md` con la data; funzioni sotto le 60 righe.
- Ruff (lunghezza di riga 100, regole `E,F,I,UP,B,SIM,NPY`); hook di pre-commit: ruff, poi la suite non-realdata in 14 processi (pytest-xdist), poi i test su dati reali uno alla volta, ciascuno in 14 processi *spawned*. Da 37 minuti su un core a pochi minuti: la regola del filtro dei candidati in forma chiusa ha portato il test di allineamento da 1 441 s a 11,7 s.

## 3.6 Test

145 file `tests/test_*.py` (48 dell'app web), circa 1 561 funzioni di test e 30 158 righe; l'ultimo conteggio registrato nel registro delle decisioni (codice P9) è di 2 338 test di unità superati e 67 test su dati reali. Marcatori: `realdata` (file grezzi reali, saltati se assenti; 22 file), `slow` (ottimizzazioni GFN2), `diagnostic` (etichette del classificatore), `serve` (app web con chimica sintetica o stub, mai xTB), `e2e` (Playwright, solo con `KSMECH_E2E=1`). Test notevoli: invarianza per rotazione delle feature di coppia; nessun tensore di input del modello contiene un tipo orbitale; la configurazione `pair_cells='valid'` riproduce bit per bit un batch v17 (`test_legacy_reproduction.py`); stabilità degli split congelati; confine motore/app web (`test_engine_boundary.py`: il motore non importa mai `ksmech.serve`).

**Scoperta – l'hook di commit e le variabili di git.** Due `git commit -only` fallivano con «files were modified by this hook»: git esporta un indice temporaneo come `GIT_INDEX_FILE`, e un test che eseguiva `git init/add/commit` in un repository di prova scriveva in quell'indice (una copia dell'indice reale passò da 154 file a 1). Da allora `provenance.git_env()` toglie tutte le variabili `GIT_*` per i comandi git interni e per i test.

# 4 Dati: sorgenti, ingest e leggi di dominio (`ksmech.data`)

## 4.1 Le sorgenti

**Tabella 4.1:** Sorgenti di dati e loro ruolo.

| Sorgente | Ruolo e caratteristiche |
|---|---|
| PMechDB | Passi polari curati a mano, con frecce come indici di mappe atomiche. Set di sviluppo. Si ingeriscono i 13 104 record `mc_only` della release PMechRP (`fold0/raw/*.txt`; ogni fold e lo split per scaffold sono permutazioni dello stesso insieme); gli split «mixed» aggiungono 10 000 trasferimenti di protone generati combinatorialmente e sono esclusi. Nessuna classe, pathway, ordine dei passi o condizione: ogni passo è isolato. `step_id` = `"pmechdb-"` + i primi 16 caratteri dello SHA-1 del testo grezzo. |
| ReactMech | 67 classi di reazione, passi mappati *senza frecce*; l'etichetta è la differenza dei due lati (capitolo 9). Usato per split fuori distribuzione; pilota da 10 000 passi (licenza CC BY 4.0). |
| RMechDB (`rmechdb`) | Passi radicalici (a un elettrone): per ora le righe RS/RC di FlowER, in futuro il CSV ufficiale. 4 535 passi dopo la deduplicazione; split `rc_molecule` (RC per molecola, RS in training). Licenza CC-BY-NC-ND: artefatti solo locali, nel dataset privato e sui pod, mai pubblici. |
| FlowER | Circa 2,14 milioni di righe con suffisso numerico (train+val+test), generate da template su USPTO-Full. Destinato allo scale-up; le sue righe RS/RC sono ingerite come rmechdb, PM e PC mai. |
| mech-USPTO-31K | Benchmark tenuto da parte, mai usato in training. Una riga per reazione con una lista piatta di frecce, senza confini tra passi: `load` solleva `NotImplementedError`, perché una conversione parziale produrrebbe falsi passi elementari. |
| QMugs | Circa 665 000 molecole drug-like; per il pre-addestramento dell'encoder sono previsti single point GFN2-xTB alle geometrie di QMugs (batch di 100 000 avviato, solo locale). |

## 4.2 Canonicalizzazione e schema

- Le molecole sono lette con `removeHs=False` (gli H mappati sono estremi di frecce) e sanitizzate con `SANITIZE_ALL ^ SANITIZE_SETAROMATICITY`: niente percezione dell'aromaticità, perché PMechDB è scritto in forma di Kekulé e le frecce nominano legami singoli e doppi specifici. Verificato: 0 delle 26 182 stringhe canoniche di PMechDB porta atomi aromatici.
- La stringa canonica è `MolToSmiles(..., kekuleSmiles=True)` sull'intero lato multi-frammento, mappe incluse (due forme di Kekulé danno stringhe diverse, di proposito); l'identità di molecola per statistiche e scaffold usa una chiave separata senza mappe.
- Una freccia è salvata *come disegnata* (coda: doppietto, legame o radicale; testa: atomo o legame); `Arrow.destination()` ne deriva la destinazione. La rappresentazione è senza perdite.
- Ordine di validazione: parsing → bilancio degli atomi pesanti (`UNBALANCED`) → bilancio di H → bilancio di carica → unicità e biiezione delle mappe → atomi delle frecce mappati → legami sorgente presenti (`BOND_NOT_FOUND`) → **replay delle frecce**: le frecce applicate come un'unica

modifica della matrice BE devono riprodurre i prodotti modulo risonanza (ARROWS_MISMATCH; decisione dell'utente del 20 ottobre).
- Re-ingest di PMechDB: 13 104 record → 13 090 accettati, 14 rifiutati (12 SANITIZE_FAILED: complessi *ate* ipervalenti come Br[Br-]Br; 1 MAP_NOT_BIJECTIVE; 1 ARROWS_MISMATCH, un Na dativo che sparisce senza freccia). Round trip delle frecce del dataset: 100% dei passi di training.

## 4.3 Il dominio: esclusioni e riparazioni contate

Ogni motivo per cui un passo non è un campione di training è un codice contato in stats.json; la frazione coperta è riportata accanto a ogni accuratezza. La storia di queste regole è istruttiva.

**Prima versione (21–24 ottobre).** Un frammento con numero di elettroni dispari senza elettroni radicalici in RDKit non è calcolabile a guscio chiuso. Un'indagine su tutti i 14 382 frammenti unici di PMechDB ne trovò 100 (non solo i 20 SPIN_PARITY già incontrati dal batch), tutti tranne uno con un metallo. Regole: METAL_PLACEHOLDER (metallo non alcalino con al più un vicino: catalizzatori di Lewis disegnati senza leganti, C=[O+][Ti]), CHARGE_REPAIRED (R1: si toglie un $-1$ da un atomo del gruppo 14 tetravalente; R2: legame onio→accettore $d^0/d^{10}$ letto come dativo), OPEN_SHELL_METAL (soprattutto addotti di Cu(II) e Fe(III)), CHARGE_UNREPAIRED. Il 24 ottobre si aggiunse RADICAL_DRAWN: 51 passi con atomi radicalici in RDKit, quasi tutti errori di disegno (metili scritti [C:20] senza H, letti come quartetti).

**Leggi unificate (P0a, 29 ottobre).** Il principio «nessuna regola arbitraria» ha sostituito le regole condizionate da elemento e parità con una legge per fenomeno:

**Legge dativa (D dative onium->M)**
: un onio (non metallo positivo diverso da C) legato a un metallo o a B è un donatore dativo; l'accettore è ricaricato del massimo $j$ che lascia pari il numero di elettroni del frammento, la carica netta del lato $\geq 0$ e i totali dei due lati uguali; poi il passo è rivalidato. La forma letterale («il metallo prende $-1$ per onio») è stata scartata dopo la misura: avrebbe ricaricato a torto 101 passi PMechDB e 58 ReactMech. Un audit dello stato di ossidazione su 20 casi (scripts/probes/dative_audit.py) li ha trovati tutti corretti.

**Legge del segnaposto**
: un metallo fuori dal blocco $s$ con esattamente un vicino e carica $\leq 0$ è un catalizzatore disegnato senza leganti, a qualunque parità (METAL_PLACEHOLDER), tranne quando compensa un donatore onio ($ML_1$, [Pd-][P+]R3); ioni liberi e $MX^+$ sono specie definite.

METAL_SUSPECT
: un frammento pari con un metallo 3d lasciato $d^1$–$d^9$ dalla contabilità degli stati di ossidazione: un flag, mai un'esclusione.

Effetti misurati: su PMechDB la legge dativa cambia 0 passi, quella del segnaposto ne rimuove 24 e gli 8 passi con Sn passano da RADICAL_DRAWN a METAL_PLACEHOLDER (passi di training 12 464 → 12 442); sul pilota ReactMech 49 passi sono ricaricati (25 Ni, 24 Pd) e 205 passi esclusi tornano nel dominio (109 Pd(0) $L_1$, 96 OPEN_SHELL_METAL).

**Riparazioni di disegno (P1, 30 ottobre).** Applicate prima delle altre leggi, conservate solo se il passo si rivalida e solo su atomi disegnati allo stesso modo nei due lati: V1 (un C neutro con 3 elettroni radicalici e un vicino riceve 3 H; il C-2 dell'imidazolo di CDI riconosciuto per topologia riceve un H), A1 (un atomo alcalino neutro nudo riceve +1), BL_NAMED (l'unico azoto $\beta$-lattamico di PMechDB disegnato senza H, riparazione nominata di un singolo passo). Su PMechDB: 9 metili, 4 K/Na nudi, 1 $\beta$-lattame; RADICAL_DRAWN 44 → 30. Su ReactMech: 62 passi CDI.

**Stato attuale di PMechDB** (data/processed/pmechdb/stats.json): 13 090 passi, 482 passi di risonanza, **12 512 passi di training**. Esclusioni: RESONANCE_STEP 482, METAL_PLACEHOLDER

56, MULTIPLICITY_AMBIGUOUS 36, RADICAL_DRAWN 5, SPIN_NOT_CONSERVED 2, CHARGE_UNREPAIRED 1; 15 riparazioni di carica conservate (V1 9, A1 4, R1 1, BL 1). Classi di dominio ammesse: CARBENE_SINGLET 12, OSM_CU2 23, OSM_FE3 19, RADICAL_SPECTATOR 11. Frecce per passo: 1 (2 401), 2 (8 570), 3 (1 960), 4 (156), 5 (3). Frammenti reagenti 29 404 (10 683 unici, 2,25 per passo); atomi pesanti per passo: mediana 16, media 20,5, massimo 220.

## 4.4 Identità di risonanza

Prodotti ed etichette sono confrontati modulo risonanza. La chiave d'identità ha avuto quattro generazioni:

1. **chiave enumerata con mappe** (20 ottobre): lo SMILES canonico minimo sulle strutture di `ResonanceMolSupplier`, al più 1 000; dipendeva dall'ordine degli atomi (il fenossido dava due chiavi) e veniva troncata nel 7% dei lati;
2. **chiave enumerata senza mappe** con H congelati (22 ottobre), test per intersezione di insiemi invece che per uguaglianza dei minimi (RDKit enumera C=N–X da $C^+$–$N^-$–X ma non il contrario);
3. **chiave sigma** (25 ottobre, «opzione B» scelta dall'utente): lo SMILES canonico senza mappe dello scheletro $\sigma$ (ogni legame singolo, niente aromaticità né stereochimica), con il numero di H di ogni atomo congelato, cariche ed elettroni spaiati azzerati, seguito dalla carica totale. Due strutture di Lewis a guscio chiuso con la stessa chiave hanno gli stessi atomi, legami $\sigma$, H e carica totale, quindi lo stesso numero di elettroni: sono forme di risonanza. Non enumera nulla, quindi è esatta e mai troncata;
4. **chiave d'identità v2** (P6, 4 novembre): per un disegno senza elettroni spaiati è la chiave sigma (ogni chiave a guscio chiuso resta invariata); per un guscio aperto vi aggiunge il multiinsieme ordinato di (SMILES sigma del frammento, carica, numero di spaiati). Separa un radicale dal suo ione e $D^{\bullet}$+A da $D^+$+$A^{\bullet-}$, mentre `[C]=O` e `[C-]#[O+]` restano una specie.

**Scoperta – RDKit non è un test di appartenenza alla risonanza.** La chiave enumerata concordava tra le forme di risonanza di un prodotto solo nel 59% dei prodotti campionati di PMechDB (l'InChI con H fissi nel 58%); la chiave sigma nel 100%. Inoltre, dal reagente disegnato RDKit raggiunge il prodotto disegnato solo in 147 delle 471 coppie di risonanza di PMechDB, perché non enumera a partire da centri con ottetto incompleto; e `ResonanceMolSupplier` restituisce «non forme»: rideriva gli H degli atomi non tra parentesi (un O del nitro diventa `[OH2+]`) e restituisce strutture radicaliche (l'azide come N=N–N con due radicali). Le forme usate per le classi di etichette e per l'augmentation sono quindi quelle di RDKit con H congelati, filtrate per stesso numero di H per atomo, carica totale ed elettroni spaiati.

**Passi di risonanza e coppie.** Un passo i cui due lati hanno la stessa chiave d'identità con le mappe conservate e un flusso elettronico bilanciato non cambia legami $\sigma$, H, carica o spin di alcun atomo: è un'interconversione di risonanza, esce dal bersaglio di training ed è conservato come *coppia di risonanza* (`resonance_pairs.parquet`; 482 dei 13 090 passi PMechDB accettati, 548 dei 10 000 del pilota ReactMech). Lati identici sono `NULL_STEP`, stessa chiave senza flusso bilanciato `NONCONJUGATED_SHIFT`: entrambi contati, nessuno è una coppia di risonanza. Gli **scambi degeneri** (stesse molecole, atomi che cambiano partner: scambio tiolo–disolfuro, scambio di protone tiolato/tiolo, shift cationico degenere in una gabbia simmetrica) sono passi reali e restano (3 su PMechDB; decisione dell'utente del 25 ottobre: «un meccanismo segue gli atomi»).

## 4.5 Split, snapshot congelati e coorti

**Split per molecola (default).** Due passi che condividono una molecola non banale (SMILES canonico senza mappe con stereochimica, più di 3 atomi pesanti; acqua, $H_3O^+$, $OH^-$, alogenuri, ioni metallici, MeOH e $NH_3$ ignorati), **nei reagenti o nei prodotti**, finiscono nella stessa parte. Contare solo i reagenti lasciava attraversare le parti al 32% dei collegamenti tra passi (X→Y e Y→Z non condividono reagenti), quanto uno split casuale; con i prodotti, 0. Il costo: i reagenti

comuni ($HSO_4^-$, $H_2SO_4$, $AcO^-$, AcOH, $TsO^-$, $Et_3NH^+$, $TFA^-$, piridina) legano il 78% dei passi in un unico gruppo che va in training; validazione e test vengono dal 22% restante e sono quindi fuori distribuzione per costruzione. Lo split per passo è stato rinominato `random_leaky` (con lo stesso seed, 457 passi di test avevano il reagente più grande in training) e si costruisce solo se nominato; il vecchio nome `random` solleva un errore.

**Altri split.** `scaffold` (scaffold del frammento reagente più grande; lo scaffold aciclico è un gruppo del 37,5% che va in training), `loco` (leave-one-class-out, per sorgenti con classi: su ReactMech le classi acid-halide, anhydride, carbamic-halide, haloformate, sul-halide), `loco_all` (la classe tenuta fuori come test, i passi senza classe in train e val), `rc_molecule` e `flower_given` per rmechdb.

**Snapshot congelati (U1).** Gli split sono *append-only*: uno snapshot di ogni split, con uno schema proprio (`FROZEN_SPLIT_SCHEMA`) che nessuna versione dei campioni invalida, tiene ogni passo esistente nella sua parte quando si aggiungono passi o si ricostruiscono i dati. Un gruppo che contiene passi congelati mantiene la loro parte, un gruppo nuovo è piazzato con un hash della sua chiave, e un passo nuovo che unisce gruppi congelati in due parti diverse è lasciato fuori e contato (`SPLIT_CONFLICT`). Uno snapshot si sostituisce solo a mano (come in P4, quando i passi con Cu hanno unito gruppi e lo snapshot per molecola di PMechDB è stato esteso).

**Coorti.** Una coorte è un insieme fissato di id di passi; ogni numero è riportato per coorte. `closed_base` (fissata il 29 ottobre, codice `d1b869b`) è l'insieme dei campioni PMechDB dello split per molecola dopo P0a: train 9 979, val 1 238, test 1 182 (12 399), con digest per colonna di ogni campione (64 colonne, ciascuna un SHA-256 troncato a 64 bit, 18,7 MB); i cambiamenti successivi sono verificati confrontando questi digest per contenuto. I dati aggiunti formano coorti proprie e non sono mai mescolati in un numero di riferimento: `added` (13 passi riparati in P1), `open_shell` (9), `carbene_singlet` (10), `open_shell_metal` (38), `p5_returns`, e le coorti di leakage di rmechdb (`leak_guard_joint`, `rc_heldout_overlap`, `rc_heldout_shared`, `closed_base_rmechdb_overlap`).

## 4.6 Provenienza degli artefatti

Ogni artefatto ha un'intestazione `Provenance` (`kind`, `schema_version`, `code_git_hash`, `code_dirty` e `code_diff_sha256`, `spec_sha256` della specifica, `inputs` con lo SHA-256 di ogni input, `config`, `built_at`, `content_sha256`); nei `.npz` è il membro `__provenance__`, nei JSON la chiave `provenance`. Le catene: record QM (input: SMILES canonico; config: metodo, solvatazione, conformero) → file di token (input: il record; config: `DescriptorConfig` e parametri di localizzazione) → shard dei campioni (parquet, token) → indice → vista per split → checkpoint e report. I loader confrontano schema, config e input e rifiutano un artefatto obsoleto (`STALE_ARTIFACT`); training e valutazione rifiutano build da un albero sporco (`require_clean`; l'hash copre `git diff HEAD` più i file non tracciati sotto `src/`). Hash del codice, della specifica e data sono solo tracciamento, così un commit da solo non invalida le cache. `ksmech-data provenance <artefatto>` stampa la catena e dice se è obsoleta.

**Scoperta – modificare file tracciati durante una build.** Poiché il flag di albero sporco copre `git diff HEAD`, modificare un file tracciato (anche la specifica o un test) mentre una build gira rende i suoi output rifiutati. È successo una volta durante P4 (modifiche ai test, annullate in pochi minuti, nessun record risultato sporco); da allora la regola è non toccare file tracciati durante una build.

## 4.7 Dataset privato

I dati derivati (store dei campioni, 20 885 file di token e altrettanti sidecar di Wiberg impacchettati in 17 archivi tar da circa 500 MB) sono su un dataset Hugging Face privato (`rhoMech-data`): la versione v18 occupava 210 file e 9,63 GB (upload 73 minuti, download 20); la revisione attuale contiene lo store 19 delle tre sorgenti più la sovrapposizione `release_a/` (store 18 per P4). Privato per i termini di PMechDB (via PMechRP, non stabiliti) e la licenza NC-ND di RMechDB; un pod riceve un token di sola lettura cancellato dopo il download.

# 5 Il piano open-shell (fasi P0–P9)

Il piano per portare i gusci aperti in KSMECH (`docs/open_shell_plan.md`) è stato concordato il 28 ottobre con 23 decisioni (U1–U23). La sua filosofia: i gusci aperti non erano mai stati rifiutati da una decisione unica, ma rimandati regola per regola (`RADICAL_DRAWN`, `OPEN_SHELL_METAL`, `SPIN_PARITY`, `METAL_PLACEHOLDER`, la maschera `fermi_smeared`). Nessuno dei 290 passi esclusi di PMechDB e del pilota ReactMech era un passo radicalico: quasi tutti erano errori di disegno. Il piano lavora «dall'esterno verso l'interno»: prima generalità e riferimento, poi i gusci aperti nei passi polari (Release A), poi la contabilità elettronica e i passi radicalici (Release B), infine il modello.

**Tabella 5.1:** Fasi del piano open-shell.

| Fase | Data | Contenuto |
|---|---|---|
| P0 | 28 ott | Infrastruttura: snapshot di split congelati, coorti, versioni di floor e decoder. |
| P0a | 29 ott | Generalità: tutte le celle di coppia con flag, extra orbitali, finestra di frontiera esatta, etichette ReactMech dal Kekulé più vicino, caps per sorgente, leggi di dominio unificate (`SAMPLE_VERSION` 18, token schema 10, spec v9.1). |
| P0b | 29–30 ott | Coorte `closed_base`, confronto appaiato, preset; GENERAL contro LEGACY su pod. |
| P1 | 30 ott | Riparazioni di disegno V1, A1, `BL_NAMED`. |
| P2 | 31 ott | Ogni molteplicità derivata due volte (scansione spGFN2 e teoria) e combinata: `spin_theory_v1`, `spin_table_v1`. |
| P2b | 31 ott | Occupazioni di riferimento a plateau, frontiera radicalica, `atom_fukui` (spec v9.2). |
| P3 | 31 ott | Gusci aperti nei passi polari di PMechDB, ammessi da tre leggi contate. |
| v2 | 1 nov | `spin_table_v2`: dialocarbeni e difenilvinilidene decisi. |
| P4 | 3 nov | Spin da `data.spin.assign`, record QM etichettati, tabella dei metalli (Cu(II), Fe(III)), coorte dei metalli. |
| P5 | 4 nov | Ritorni del pilota ReactMech (riparazioni dative), audit dativo, rivalutazioni per fetta. Fine della **Release A**. |
| P6 | 4 nov | Contabilità elettronica: $u$ dal disegno, legge del flusso, chiave d'identità v2, codici `SET_STEP`/`DISSOCIATIVE_ET_STEP`, `SAMPLE_VERSION` 19. |
| P7 | 5 nov | SOMO donatori e lacune, popolazione SOMO, $G$ pesato per occupazione, $g_2$, canali own-pair e di ricombinazione, floor `own_break`. |
| P8 | 5–6 nov | RMechDB via FlowER, un'unica legge di spin e dominio, build della **Release B**; spec v10 approvata. |
| P9 | 6–9 nov | Testa atomica a cinque classi, schermo di decodifica, preset `open_shell`; training (passi 8 e 9). |

## 5.1 La regola di tolleranza U20

Ogni anello della catena di confronti (Release A contro P0b GENERAL; `pm_control` contro Release A; `joint` contro `pm_control`) usa la stessa regola, unilaterale sul top-1 full-R di `closed_base` val: il braccio A fallisce quando la sua media è più bassa di più di max(2× deviazione standard combinata delle 6 run dell'anello, 1 punto). Accanto si riporta il test esatto di McNemar unilaterale, $p = P(X \geq n_{ab})$ con $X \sim \mathrm{Bin}(n_{ab} + n_{ba}, \frac{1}{2})$, in aritmetica intera, sui *hit* a maggioranza dei tre seed e sulle 9 coppie di seed (`eval.paired`, `ksmech-eval compare`). Il confronto rifiuta bracci

decodificati su campioni, parti, coorti, beam, caps o versioni del decoder diversi. Regola di default di P9 passo 9: una regola (maschera o filtro) è attiva di default solo se il test appaiato la mostra migliore ($p < 0{,}05$) senza cali su `closed_base`; un input fisicamente motivato resta se è migliore o non inferiore entro la tolleranza.

## 5.2 Spin: tabelle e leggi

**Due derivazioni indipendenti (P2).** Su richiesta dell'utente («fai le due cose in parallelo: il calcolo spGFN2 e le regole chimiche come la teoria del campo dei leganti, la teoria delle rappresentazioni…») ogni molteplicità è derivata due volte:

**(A) calcolo**
spGFN2 ottimizzato in ALPB acqua a ogni molteplicità ammessa (Fe(III) a $S = 5/2, 3/2, 1/2$, Cu(II) come controllo, specie di FlowER con $u \geq 3$), conclusivo quando lo stato più basso sta sotto ogni altro di più della barra d'errore del metodo: organici 7 kcal/mol = 0,304 eV (spGFN2 su AC12), metalli di transizione 17,9 kcal/mol = 0,776 eV (MAD di spGFN2 sul sottoinsieme 3d di TM90S, Neugebauer et al. 2023);

**(B) teoria**
`spin_theory_v1`: conteggio $d^n$, campo dei leganti ($O_h$: $d^{1-3}$, $d^8$, $d^9$, $d^{10}$ esito unico; $d^{4-7}$ dipendente dal campo; $T_d$ alto spin), gruppo puntuale e irrep di frontiera, regola di Hund, Gleiter–Hoffmann per i carbeni; nessuna regola nomina una specie;

**(C) combinazione**
(scelta dall'utente) A e B concordano → assegna; B indeterminata → A decide entro la sua barra; ogni disaccordo → `MULTIPLICITY_AMBIGUOUS`.

Su 156 specie (64 organiche, 61 Cu/Fe, 31 `METAL_SUSPECT`): concordi 59, B indeterminata e A conclusiva 37, A inconclusiva 15, disaccordo 5, A fallita 40. Fe(III) ad alto spin confermato per 13 su 20 specie di Fe e mai contraddetto; Cu(II) $u = 1$ per 36 su 41.

**Scoperta – il punto di sella lineare dei carbeni.** ETKDG disegna Cl[C]Cl e Br[C]Br esattamente lineari (180,0°). Da quel punto di sella il tripletto sembrava più basso (0,69/0,64 eV); da una partenza spostata di 0,02 Å $CCl_2$ si piega a 112,9° (singoletto) e il singoletto risulta più basso di 1,01 eV ($CBr_2$: 0,76 eV). Da qui due leggi: ogni stato è ottimizzato da due partenze (l'embedding e un «calcio» di 0,02 Å, seed 0) e si tiene il più basso convergente; e `BOND_FORMED`: una partenza fallisce se due atomi non metallici a distanza di grafo $\geq 2$ finiscono entro 1,2× la somma dei raggi covalenti (un epossido, un biradicale di Norrish II che si chiude a ciclobutano, lo shift di Fritsch–Buttenberg–Wiechell del $Ph_2C{=}C$:).

**`spin_table_v2` (1 novembre).** L'utente ha scelto quattro cambiamenti alla tabella invece di un arbitro DFT (PySCF non installato): B riottimizza dalla partenza spostata quando questa finisce più bassa di $10^{-5}$ Ha; regola 4a, un vinilidene $R_2C{=}C$: è singoletto (Stang 1978); regola 4b, un carbene con due vicini $\pi$-donatori con doppietto è singoletto (Bourissou 2000); in A, uno stato di carbene che forma un legame da ogni partenza prende il single point verticale alla geometria di produzione come limite superiore. Risultato: $CCl_2$, $CBr_2$, $Ph_2C{=}C$: → singoletto; `COC(=O)[C]C(=O)OC` → tripletto; nessun verdetto su metalli o biradicali cambia.

**Le leggi di dominio dei gusci aperti (P3).** Una specie disegnata con elettroni spaiati in un passo polare è nel dominio quando valgono tre leggi, ognuna con il suo codice: (u) il suo numero di spaiati (RDKit se $\leq 1$, altrimenti la tabella) si conserva nel passo (`SPIN_NOT_CONSERVED`); (R) ogni voce atomica di $\boldsymbol{R}$ è pari, cioè un passo polare non crea, distrugge o sposta un elettrone spaiato (`NOT_ELEMENTARY`); (S) la sua SOMO è intera (`SOMO_NOT_INTEGRAL`). Un passo ammesso è `RADICAL_SPECTATOR` o `RADICAL_REACTIVE`. PMechDB ammette per regola, le altre sorgenti polari escludono. Ammessi 9 campioni (una riduzione di Birch, tre deprotonazioni di perossili, quattro perossili → superossido, una protonazione del superossido) più 2 coppie di risonanza; un file di curation elenca 5 passi che le leggi ammettono ma una lettura del disegno rifiuta. Il dodicesimo

caso, l'addotto $Et_3B$–$O_2$ (`5d007b38`), resta `SPIN_NOT_CONSERVED`: la tabella gli assegna $u = 0$; la decisione è lasciata all'utente.

**Metalli (P4).** `data.spin.assign` restituisce conteggio, sorgente (`rdkit/species/metal`) e motivo. Tabella dei metalli `metal_v1.json`: Cu(II) $u = 1$, Fe(III) ad alto spin $u = 5$ senza donatori C o P, Cu(II) con donatore H o I incompatibile (il Cu(II) li ossida); ogni riga vale per una specie solo dove la tabella delle specie la conferma (regola C). `OPEN_SHELL_METAL` 78 si divide in `OSM_CU2` 23, `OSM_FE3` 19, `MULTIPLICITY_AMBIGUOUS` 36 (12 Cu, 24 Fe, di cui 14 $Br_2$–$FeBr_4^-$). 49 specie calcolate con etichetta di spin (48 riuscite; l'addotto $Et_3B$–$O_2$ fallisce come `BOND_FORMED`). La legge (S) legge solo i frammenti disegnati con elettroni radicalici (decisione dell'utente: un Cu(II) con smearing resta un campione marcato).

**Contabilità elettronica (P6).** $u$ è il numero di atomi con $N_i$ dispari nel disegno (`spin_of_drawing`); la tabella lo alza, mai lo abbassa: una riga di Hund (nella v2 solo [O] e [O][O]) vale per ogni disegno con la sua chiave, così O=O, [O][O] e $[O^+][O^-]$ sono un unico tripletto; le altre righe valgono solo per il disegno scansionato. $N$ è letto da ogni disegno perché la percezione dei radicali di RDKit separerebbe `[C]=O` da `[C-]#[O+]` e conterebbe male $[Br^+]$, $[OH^+]$ e il $CH_2$ di singoletto. Le forme di un radicale si ottengono con un *proxy anionico*: ogni centro radicalico prende un elettrone in più, si enumerano le forme dell'anione, e in ogni forma ogni atomo negativo del gruppo coniugato restituisce l'elettrone e porta il radicale (benzile e fenossile 4 forme, pentadienile 3, allile 2, TEMPO 1; $NO_3^\bullet$ un disegno per O).

**Un'unica legge di spin (P8).** Un singoletto di tabella disegnato con $N$ dispari è ridisegnato a guscio chiuso dove gli spostamenti di elettroni lungo i legami lo raggiungono (`PAIRED_DRAWING`: N#N, ozono, Criegee; ordinamento: ottetti completi, meno cariche, minore $\sum q\chi$ di Pauling, così il Criegee è C=$[O^+][O^-]$, meno legami cambiati; i metalli non sono mai ridisegnati); un caso di Hund disegnato appaiato è ridisegnato spaiato (`HIGH_SPIN_DRAWING`: O=O); una specie non metallica con $u \geq 2$ che non è un caso di Hund, o un singoletto che nessun disegno a guscio chiuso raggiunge, è `MULTI_RADICAL_SPECIES`, salvo esenzioni contate. Il censimento di spin prima del QM rifiuta 37 specie rmechdb (`SPIN_CENSUS_REFUSED`). Una sorgente di passi a un elettrone conserva $u$ solo dove $\boldsymbol{R}$ non sposta un singolo elettrone.

## 5.3 RMechDB via FlowER (P8)

FlowER ha 2 174 470 righe, 2 140 367 con suffisso numerico (righe train/test: RC 1 383/136, RS 3 223/346, PM 12 322/294, PC 14 746/1 653; nessuna validazione). Il loader `data.rmechdb` toglie i gas di bagno, applica i ridisegni di spin prima di calcolare l'id (reazione canonica senza mappe) e tiene un passo per id, RC prima di RS: 5 088 righe → 4 535 passi (553 `DUPLICATE_STEP`, 165 righe con gas di bagno). QM a 14 worker: 4 277 specie, 4 227 record, 13 fallimenti, 37 rifiutate. Etichette 4 293/4 333 (99,08%; restano 11 reazioni radicaliche dell'ozono, che la legge del flusso rifiuta con l'ozono disegnato O=$[O^+][O^-]$, e l'omolisi del persolfato). Split `rc_molecule`: RC 950/118/120 passi + 3 096 RS in training; campioni 3 916/111/113. `flower_given`: 3 782/106/445. Campioni totali 4 184 (`SOMO_NOT_INTEGRAL` 109).

**Audit di leakage** (`ksmech-data leakage-audit`, specie per chiave v2): 36 passi di training rmechdb condividono una specie con val/test di `closed_base` (`cohort_leak_guard_joint`, tolti dal training congiunto come `LEAK_GUARD`); 79 passi di training PMechDB condividono una specie con val/test RC (una fetta, mai rimossi); 26 passi val/test RC con il training di PMechDB.

## 5.4 Release B e merge

Store `SAMPLE_VERSION` 19: PMechDB 12 469 campioni, ReactMech 9 256, rmechdb 4 184; rispetto alla v18 ReactMech è identico e PMechDB differisce solo nei 9 campioni `RADICAL_SPECTATOR` (lacune). Migrazione in `main` via reflink (12 122 file nuovi, 182 sostituiti; le copie v18 in `data/processed/v18_reference/`), re-key dei 1 388 stati di pool salvati dall'app web (69 cambiati), merge `87d8956` (P6 con P0–P5), `840e481` (P7), `5417210` (P8), poi il codice P9 (`73fb691`); niente è stato pubblicato sul remoto. Il controllo di provenienza su 438 802 file ne ha trovati 432 251 validi e 211 obsoleti (run e decodifiche vecchie).

## 5.5 P9: il modello open-shell

- Testa atomica $\{-2, -1, 0, +1, +2\}$ (preset `open_shell`); $-1$ richiede $N \geq 1$, $-2$ richiede $N \geq 2$; nessuna maschera a livello di pool. Un modello polare mantiene $\{-2, 0, +2\}$.
- **Schermo di decodifica a guscio chiuso**: su un pool senza elettroni spaiati ogni voce atomica $+1$ deve stare su un estremo di un legame con voce $-1$ (`ODD_OFF_BROKEN_BOND`, sempre attivo: con nessun elettrone spaiato e nessun salto elettronico, $N$ di un atomo diventa dispari solo tenendo un elettrone di un legame che perde; ammette omolisi, frammentazione azo, $Cl_2$ con RH). Il rifiuto di ogni voce $-1$ su tale pool non è una legge ma un'inferenza su quali passi esistono: è il filtro opzionale `-closed-minus1-filter`, spento di default dopo l'ablazione. `DECODE_VERSION` 4.
- `LEAK_GUARD` automatico per pmechdb+rmechdb; `-val-cohort`; metriche `open_top1`, `open_r_exact`, `open_n` sui pool con un elettrone spaiato.
- Non realizzabile al momento del batch: il braccio con la maschera dei metalli $d$ (richiede la ricostruzione dello store).

I risultati dei training di P4 (Release A), P9 passo 8 e passo 9 sono nel capitolo 13.

# 6 Calcolo quantistico (`ksmech.qm`)

Ogni frammento molecolare distinto (per SMILES canonico) di ogni passo è calcolato una volta con GFN2-xTB attraverso la libreria `tblite` (wheel PyPI 0.7.0). I risultati sono salvati su disco, chiave SMILES canonico più id del conformero; la stringa del metodo fa parte della chiave.

## 6.1 Geometria

1. **Embedding**: RDKit ETKDGv3 con idrogeni espliciti, pre-rilassamento MMFF (UFF se mancano parametri), pruning per RMSD; il batch PMechDB usa **un conformero** per molecola (decisione dell'utente; `embed` ne genera 3 di default). L'ordine atomico è `Chem.AddHs(Chem.MolFromSmiles(smiles_canonico))`: il batch deduplica su SMILES canonico senza mappe e ogni passo ritrova la corrispondenza con le mappe atomiche per substructure match.
2. **Ottimizzazione**: GFN2-xTB con solvatazione implicita ALPB (acqua). L'ottimizzatore di default è L-BFGS-B di SciPy sulle energie e i gradienti di tblite in coordinate cartesiane (`tblite-lbfgs`), convergenza a $\max|\nabla E| \le 3\cdot10^{-4}$ Hartree/Bohr (`QMConfig.fmax`), al più 500 iterazioni, altrimenti `OPT_NOT_CONVERGED`. Timeout per molecola 900 s (`SIGALRM` nel worker; il primo pilota a 300 s perdeva steroidi da 90–100 atomi pesanti).
3. **Controllo di connettività**: `CONNECTIVITY_CHANGED` scatta solo quando un legame dell'input si allunga oltre 1,3× la somma dei raggi covalenti; nuovi contatti sono ammessi e i legami a Li, Na, Mg, K non sono controllati (contatti elettrostatici).
4. **Fallback con vincoli**: se un legame si rompe, si riottimizza dalla geometria di force field con un muro armonico a fondo piatto su ogni legame controllato, $E = k\max(0, d-1{,}2(r_i+r_j))^2$, $k = 1$ Hartree/Bohr$^2$; il single point finale è senza vincoli e il record riceve l'avviso `RESTRAINED_GEOM` (geometrie «come disegnate», non minimi; meno affidabili per energie e cariche).

**Decisione – solvatazione in acqua (19 ottobre).** ALPB acqua è usata per ogni ottimizzazione e single point, inclusi i calcoli di Fukui a $N \pm 1$. Motivo: le specie di PMechDB sono soprattutto intermedi carichi; su un pilota di 300 molecole il rilassamento in fase gas rompeva un legame nel 19,7% dei casi (intermedi tetraedrici protonati che perdono acqua, C(O$^-$)Cl che perde Cl$^-$), ALPB acqua lo riduce al 12,3%. Il solvente fa parte della chiave di cache: stringa del metodo `GFN2-xTB/ALPB-water/tblite-lbfgs`.

Effetto cumulativo delle scelte sul pilota di 300 molecole: fase gas con grafo stretto 78,7% → ALPB acqua 86,0% → test di allungamento 89,0% → fallback con vincoli **98,7%** di successi.

## 6.2 Struttura elettronica e descrittori

Il single point finale esporta $\boldsymbol{S}$, la matrice di Fock convergente (tblite espone solo l'Hamiltoniana di core: la Fock è ricostruita come $\boldsymbol{SC}\,\mathrm{diag}(\boldsymbol{\epsilon})\,\boldsymbol{C}^\top\boldsymbol{S}$, valida perché $\boldsymbol{C}^\top\boldsymbol{SC} = \boldsymbol{I}$ a $10^{-15}$), $\boldsymbol{C}$, $\boldsymbol{\epsilon}$, occupazioni, cariche di Mulliken. La mappa AO→atomo non è esposta da tblite ed è ricostruita dalla lista degli shell per elemento.

**Scoperta – l'ordine degli shell nei metalli di transizione.** La prima mappa AO assumeva l'ordine $s, p, d$ su ogni atomo, ma tblite ordina gli shell per elemento: da Sc a Cu, da Y ad Ag e da La ad Au lo shell $d$ viene per primo (Ti: $3d, 4s, 4p$). Le matrici erano giuste, le etichette `ao_l` no: 22 molecole (16 Ti, 4 Cu, 1 Fe, 1 Sc) avevano quote $s/p/d$ e la regola «tipo $d$» sbagliate. Corretto leggendo la lista degli shell di tblite (`qm.ao.ao_layout`); 22 file di token ricostruiti. Anche l'ordine dei $p$ è peculiare, $(p_y, p_z, p_x)$, fissato da un test su HF lungo ciascun asse.

**Fukui.** Cariche di Mulliken $q$ (positive = povere di elettroni), stesso metodo e stessa geometria senza riottimizzazione: $f^+ = q(N) - q(N+1)$, $f^- = q(N-1) - q(N)$, duale $= f^+ - f^-$ (positivo:

sito elettrofilo). Sono salvate sia le funzioni alle differenze finite sia quelle a orbitali congelati. Su acetaldeide la $f^+$ alle differenze finite ha il massimo sull'O (0,34), non sul C carbonilico (0,17), perché la risposta rilassata sposta carica sull'atomo elettronegativo; la forma a orbitali congelati dà C 0,60 (e $f^-$ dell'N della metilammina 0,83). Per questo il modello usa di default quelle congelate (`ModelConfig.atom_fukui='frozen'`, alternative `fd` e `both`). Un SCF fallito a $N \pm 1$ dà array NaN e l'avviso `FUKUI_SCF_FAIL` senza far fallire la molecola. HOMO è il più alto MO con occupazione $\geq 0{,}5$.

**Pesi di frontiera.** Popolazione di Mulliken sull'atomo dei blocchi HOMO−1, HOMO, LUMO, LUMO+1; un blocco somma i livelli canonici entro 0,05 eV (finestra «legacy») così che i pesi non dipendano dalla mescolanza arbitraria dentro un livello degenere (gli $e_{1g}/e_{2u}$ del benzene danno pesi simmetrici sull'anello). Un secondo insieme somma solo i livelli degeneri a $10^{-4}$ eV (`frontier_blocks='exact'`): la finestra di 0,05 eV unisce livelli non degeneri nel 22% delle molecole PMechDB, perciò è un flag deciso da ablazione. Esempi: HOMO dell'anisolo ipso 0,165, orto 0,110/0,141, meta 0,048/0,020, para 0,239; LUMO del nitrobenzene orto 0,084, meta 0,001, para 0,094. Per un guscio aperto HOMO e LUMO sono il *gruppo aperto* (ogni livello con occupazione di riferimento strettamente tra 0 e 2), HOMO−1 il più alto livello a occupazione 2 e LUMO+1 il più basso a 0 (convenzione radicalica, P2b).

**Sterica.** `morfeus` con densità 0,004 invece di 0,001: 4,5 volte più veloce su una molecola di 33 atomi, $V^{\text{bur}}$ cambia meno di 0,15 punti. Dal 9 novembre il calcolo usa una griglia condivisa (vedi sotto).

## 6.3 Spin e molteplicità

La carica totale è la somma delle cariche formali RDKit. Il numero di elettroni spaiati *non* è più letto dalla percezione dei radicali di RDKit: dal piano open-shell (P2–P8) lo decide un modulo (`data.spin`) con due tabelle versionate, descritto nella sezione 5.2. Una specie il cui conteggio differisce da quello di RDKit (un carbene di singoletto, CO disegnato con due elettroni radicalici, un addotto di Cu(II) o Fe(III)) è calcolata a quel conteggio con un'etichetta di metodo propria (`+spin-v1-u<u>`), così ogni altra chiave di cache resta invariata. La sua geometria è ottimizzata dall'embedding e dall'embedding spostato di un «calcio» fisso di 0,02 Å (un embedding simmetrico, il $CCl_2$ lineare, può essere un punto di sella del singoletto); un ottimo che forma un legame che il disegno non ha (due atomi non legati più vicini di 1,2 volte i raggi covalenti) è lo stato di un'altra molecola e viene scartato (`BOND_FORMED`). Una specie senza riga in tabella con un numero dispari di elettroni fallisce come `SPIN_PARITY`. I gusci aperti sono calcolati ristretti, con occupazioni frazionarie.

## 6.4 Batch, cache e indice

- `ksmech-qm run -source S -workers N` calcola i record mancanti in processi *spawned* con `OMP_NUM_THREADS=1`; un crash del pool riesegue i task in volo uno alla volta per isolare la molecola (`WORKER_CRASH`). I file di cache sono `fsync`-ati prima della rinomina atomica, un `.npz` di lunghezza zero non è un hit, le run sono riprendibili.
- **Retry**: `-timeout`, `-maxiter`, `-scf-maxiter` allentano la convergenza per i fallimenti senza cambiare la stringa del metodo; `-retry-reason` ritenta solo certi codici. Un fallimento che si ripeterebbe lascia una *tombstone* `<key>.failed.json`: una run successiva con le stesse impostazioni e un budget non maggiore risponde da essa senza rieseguire xTB (anche l'app web).
- **Indice SQLite** (`ksmech-qm index stats|refresh`): `<root>/index.sqlite` di record, tombstone, token e sidecar di Wiberg, letto dalle intestazioni dei file e aggiornato incrementalmente; i file restano la fonte di verità.

- **Indici di Wiberg** (`ksmech-qm bond-index`): dei legami di Lewis di ogni record, in `data/cache/bond_index/`; la cache dei token ne ha bisogno.

**Throughput.** Su 2 241 molecole cronometrate: mediana 3,6 s, media 27 s, p90 55 s, massimo 905 s (il timeout); le molecole oltre 300 s prendono il 41% del tempo. Un ciclo di ottimizzazione su uno steroide di 103 atomi richiede circa 300 s con ALPB su un core.

**Scoperta – instabilità dell'host.** Durante il primo batch completo l'host (Ryzen 7 2700X, 16 thread) si è riavviato bruscamente quattro volte, senza tracce nel journal. Una causa contingente trovata il 20 ottobre: `ksmech-orbitals build` non fissava i thread BLAS e un worker girava al 528% di CPU; da allora ogni processo figlio riceve `OMP/OPENBLAS/MKL_NUM_THREADS=1`. La causa di fondo erano le impostazioni del BIOS e un bit-flip della RAM (corretto il 24 ottobre, memtest86+ superato il 25, un modulo DIMM rimosso); da allora le run pesanti sono consentite fino a 15 processi.

## 6.5 Accelerazioni del 9 novembre

Un profilo ha mostrato che l'ottimizzazione della geometria prende il 60–97% del tempo di una molecola, Python meno dello 0,2%, e la routine LAPACK `dsygvd_` il 63% di un single point; il tempo scala come $n^{3,1}$ e le molecole oltre 60 atomi (17% del totale) consumano il 75% della CPU.

**tblite su OpenBLAS.** tblite 0.7.0 include BLAS/LAPACK netlib 3.8.0. I worker *spawned* usano invece l'OpenBLAS seriale di sistema (0.3.29, LP64; le librerie ILP64 sono rifiutate) tramite `qm.blas` (`KSMECH_BLAS=auto|bundled|PATH`); un OpenBLAS globale è stato scartato perché libtorch esporta `sgemm_`/`dgemm_`. `-workers 1` e l'app web mantengono la copia netlib; un pod richiede `libopenblas0-serial`. Single point (ms, netlib → OpenBLAS): 21 atomi 30→20, 48 atomi 103→54, 70 atomi 315→129, 93 atomi 576→244, 120 atomi 739→268; da 1,5 a 2,7 volte per molecola. Risultati uguali a $10^{-9}$ a geometria fissa; piccole differenze di percorso in L-BFGS-B (93 atomi: 0,013 kcal/mol, 0,018 Å). La stringa del metodo non cambia; il fornitore BLAS è registrato nell'intestazione del record. Nella stessa modifica $V^{\mathrm{bur}}$ è calcolato su un'unica griglia condivisa di circa 45 000 punti (valori identici; da 0,25 a 0,07 s per 21 atomi, da 1,44 a 0,44 s per 120).

**Ottimizzatore ANC/RFO.** Candidati confrontati su 34 molecole (energie valutate / tempo): L-BFGS-B 8 225 / 1 360 s; L-BFGS-B con SCF riavviato 6 880 / 701 s; PreconLBFGS 5 411 / 582 s; geomeTRIC 1 797 / 1 917 s; il binario ANCopt di xtb 2 660 / 510 s. Scelta: `-optimizer tblite-anc` (`qm.anc`), Hessiana modello di Lindh con pavimento degli autovalori 0,01, regione di fiducia RFO 0,3 Bohr, aggiornamento BFGS con ricostruzione della base ogni 20 passi, `fmax` $3 \cdot 10^{-4}$, SCF riavviati, e una guardia di dissociazione ($1{,}3 \times r_{\mathrm{cov}}$ per 5 geometrie di fila). End-to-end: da 8 084 a 3 202 energie (2,5×), da 1 039 a 332 s (3,1×; da 0,84× a 6,1× per molecola); una molecola di 101 atomi ora converge in 30 s; l'energia ANC è minore o uguale in 32 casi su 33 (mediana −0,21 kcal/mol). Il suo `-maxiter` conta energie. Il default resta `tblite-lbfgs`: cambiarlo cambierebbe la chiave di cache di ogni record e imporrebbe il ricalcolo completo del QM.

# 7 Token orbitali e atomici (`ksmech.orbitals`)

`ksmech-orbitals build -source S -workers N` legge i record QM e scrive un file `.npz` (senza pickle) per molecola e conformero sotto `data/cache/tokens/`, con la stessa chiave della cache QM, più un `summary.json`. Il file contiene i token atomici (`atom_*`), i token orbitali (`orb_*`), i coefficienti localizzati `C_loc` ($n_{\mathrm{AO}} \times n_{\mathrm{orb}}$), $\boldsymbol{S}$, $\boldsymbol{F}$ (Hartree), la mappa AO e le statistiche di localizzazione. Una sequenza di id: prima gli atomi, poi gli orbitali occupati per energia crescente, poi i virtuali.

## 7.1 Localizzazione

### 7.1.1 Configurazione attuale

**Metodo**
IBO: Pipek–Mezey con popolazioni di Löwdin ed esponente 4 (`orbitals.localize.PMConfig`); Pipek–Mezey con Mulliken ($p = 2$) resta come confronto dietro un flag (`pm_mulliken`).

**Blocchi**
occupato e virtuale localizzati separatamente; dentro l'occupato, gli orbitali doppi e quelli a occupazione diversa (SOMO, livelli aperti) in sotto-blocchi separati: una rotazione che mescola occupazioni diverse cambia la densità $\sum_k n_k |\phi_k|^2$.

**Partenza**
fattorizzazione di Cholesky con pivot della matrice densità di ogni blocco nella base di Löwdin (pivot = massima diagonale residua): deterministica e già localizzata. Due tokenizzazioni del benzene sono identiche bit per bit.

**Convergenza**
sweep di rotazioni di Jacobi $2 \times 2$ fino a $\Delta\mathcal{P} < 10^{-8}$ per sweep o 200 sweep; un blocco non convergente è conservato e marcato (`pm_converged`).

**Vettorizzazione**
uno sweep è $n - 1$ round *round-robin* di $n/2$ coppie disgiunte, ogni round vettorizzato (esatto perché l'angolo di una coppia dipende solo da $Q_{ii}, Q_{jj}, Q_{ij}$). Da 47 s a 8 s su una molecola di 96 atomi; 0,07 s su una tipica di 31. Per l'esponente 4 i coefficienti di Jacobi sono in forma chiusa e le somme per atomo usano `np.add.reduceat` (22 ottobre).

### 7.1.2 Perché IBO e non Pipek–Mezey

**Scoperta – Pipek–Mezey si blocca sugli aromatici.** Con popolazioni di Mulliken e $p = 2$ la mediana è di 6 sweep per blocco, ma 20 molecole su circa 1 000 raggiungevano il limite di 200, quasi tutte piccoli aromatici (bromobenzene, 2-metilpiridina, stilbeni): il funzionale è quasi piatto lungo le rotazioni che mescolano gli orbitali $\pi$ dell'anello e a sweep 200 cambia ancora di $10^{-8}$–$4 \cdot 10^{-6}$. Uno script diagnostico ha verificato che ogni orbitale ancora in rotazione era $\pi/\pi^*$ delocalizzato. Con l'esponente 4 e Löwdin (IBO, spec v7.2, 22 ottobre) il benzene converge in poche decine di sweep a tre legami $\pi$ a due centri.

## 7.2 Occupazioni di riferimento e gusci aperti

Per una specie a guscio aperto i sotto-blocchi vengono dalle occupazioni che GFN2-xTB assegna. Percorrendo gli orbitali canonici in ordine di energia, i vicini con occupazioni uguali (a $10^{-6}$ e, `PLATEAU_TOL`: un'uguaglianza numerica, non una soglia chimica) formano un *plateau*, e l'*occupazione di riferimento* di ogni orbitale è l'occupazione di aufbau mediata sul suo plateau. Lo smearing di Fermi dà occupazioni uguali a livelli esattamente degeneri, quindi un livello aperto degenere divide i suoi elettroni in parti uguali ($\mathrm{Br}^\bullet$ $3 \times \frac{5}{3}$, NO $2 \times \frac{1}{2}$, la coppia $\pi^*$ del superossido

$2 \times \frac{3}{2}$), senza finestra di energia (P2b, 31 ottobre). Un centro radicalico dà quindi un token a occupazione 1 e un livello aperto degenere un token per orbitale all'occupazione del plateau.

**Molecole con smearing di Fermi.** Una molecola con un MO a più di 0,05 dalla sua occupazione di riferimento è marcata `fermi_smeared` (decisione dell'utente, 20 ottobre): le occupazioni frazionarie restano l'input continuo (non sono arrotondate) e i token con $0{,}2 < n < 1{,}8$ non sono né donatori né accettori. Le prime molecole marcate avevano i quattro gap HOMO–LUMO più piccoli di 2 001 (C=[O$^+$][Ti$^-$] 0,046 eV, un catione allilico 0,135 eV, un silicato pentacoordinato 0,175 eV; gap mediano 3,17 eV). Una regola letterale $0{,}05 < n < 1{,}95$ avrebbe marcato anche 6 radicali veri con gap di 0,39–6,1 eV: per questo il confronto è con l'aufbau dello stato di spin del record.

## 7.3 Descrittori del token orbitale

Sia $w_a^{(r)}$ la popolazione normalizzata dell'LMO $r$ sull'atomo $a$ e $\mathcal{A}_r$ l'insieme dei *centri*, gli atomi con $w_a^{(r)} \geq \tau_{\text{atom}} = 0{,}15$ (era 0,05 fino al 20 ottobre; con 0,15 il $\pi$ del benzene passa da 4 a 3 centri). Input del modello (`orbital_features`, 23 valori continui, più l'insieme spettrale e i pesi sui centri):

- occupazione $n_r$ (2, 0, 1 per una SOMO, il valore di plateau, frazionaria se con smearing);
- numero di centri e i quattro pesi maggiori (zero-padding);
- quote $s$, $p$, $d$ della popolazione sui quattro centri principali;
- per i due centri principali: $|\cos|$ tra il vettore $p$ e l'asse ($\sigma$ contro $\pi$ come numero), popolazione di sovrapposizione con segno, distanza;
- raggio di girazione dei centri attorno al centroide pesato;
- insieme spettrale $\{(\epsilon_k, p_{rk}) : p_{rk} \geq 0{,}01\}$, al più 8 voci, rinormalizzato, codificato da un set encoder;
- opzionali (`ModelConfig.orbital_extra`, spec v9.1): l'energia $E_r$ e la frazione di peso spettrale che l'insieme conserva prima della rinormalizzazione.

Lo spettro completo (usato da $G$), il centroide, gli atomi dei centri e il vettore dei coefficienti sono contabilità, non input. Un orbitale eredita gli embedding dei suoi atomi attraverso i pesi sui centri.

**Troncamento spettrale.** Un LMO si distribuisce su 20–40 MO canonici: l'insieme di 8 voci raggiunge il limite nell'88% dei token e conserva in media 0,77 del peso di un occupato e 0,83 di un virtuale (mediana 0,79, p5 0,52). Per questo $G$ è calcolato sugli spettri *completi*: su 49 605 coppie delle prime 20 molecole PMechDB, $G$ dagli spettri troncati differisce fino al 21,8% (un test lo verifica).

**Distribuzioni misurate** (2 001 molecole, schema 5): token per molecola mediana 93 (min 11, max 441); centri a $\tau = 0{,}15$: 1 nel 10,7%, 2 nell'86,6%, 3 nel 2,7%; il carattere $|\cos|$ è bimodale (0,9–1,0 nell'89,6%, 0,0–0,1 nel 9,8%), quindi la distinzione $\sigma/\pi$ non richiede soglie; popolazione di sovrapposizione mediana 0 (p5 −1,77, p95 +0,39); distanza dei centri mediana 1,10 Å; raggio di girazione mediano 0,55 Å.

## 7.4 Il token atomico

Input: numero atomico e carica formale (embedding categorici), carica di Mulliken, $f^+$ e $f^-$ a orbitali congelati, descrittore duale alle differenze finite, i quattro pesi di frontiera, $V^{\text{bur}}$, segno di chiralità. Il segno di chiralità è $-\text{sign}\det$ dei vettori verso i tre vicini di rango CIP più alto, impostato solo dove lo SMILES specifica lo stereocentro (ETKDG sceglierebbe altrimenti un enantiomero arbitrario); PMechDB ha stereochimica in 2 passi su 13 091. Un valore mancante

è 0 con un flag proprio (atom_missing). Nessuna coordinata assoluta è un input: la geometria entra solo attraverso le distanze.

Calcolati ma model_input=False: ibridazione, aromaticità, gruppi funzionali (vocabolario Ertl di 413 etichette viste almeno 5 volte su 14 383 molecole uniche PMechDB, copertura 90,4%), $f^{\pm}$ alle differenze finite, posizione. Le colonne opzionali per i gusci aperti sono descritte nella sezione 7.7.

## 7.5 Il classificatore diagnostico

Le etichette da chimico sono ancora calcolate dagli stessi descrittori da un classificatore a regole (ClassifyConfig: one_atom_frac 0,85, two_atom_frac 0,85, sigma_cos 0,7, max_s_share_pi 0,25, member_min_pop 0,05, segno della popolazione di sovrapposizione), con tipi aggiuntivi empty_s (l'1$s$ di $H^+$, l'accettore di ogni trasferimento di protone), empty_d (le funzioni $d$ di polarizzazione), delocalized_pi, unknown. Sono usate in test, diagnostica e interfaccia; un test verifica che cambiare ogni tipo non cambi orbital_features(). Statistiche sulle prime 991 molecole: occupati $\sigma$ 78%, doppietti 13,1%, $\pi$ 4,9%, delocalizzati 3,9% (tutti anelli aromatici con PM); virtuali $\sigma^*$ 83,4%, empty_d 7%.

**Scoperta – le etichette nascondono la fisica.** Le energie diagonali degli LMO $\sigma^*$(C–X) non seguono l'ordine Cl > Br > I (ALPB: C–Cl −2,07, C–Br −0,65, C–I −1,63 eV), mentre i LUMO canonici sì (−4,21, −5,92, −6,28 eV): $\boldsymbol{c}^{\top}\boldsymbol{F}\boldsymbol{c}$ di un LMO media sui virtuali canonici che mescola. È ciò che ha portato a calcolare il gap sui pesi spettrali canonici ($G$) invece che sulla diagonale dell'LMO. Altro esempio: la protonazione abbassa $\sigma^*$(C–O) di 9,67 eV in fase gas (metanolo → $CH_3OH_2^+$) ma solo di 2,92 eV in ALPB acqua, perché il solvente stabilizza il catione.

## 7.6 Ruoli di donatore e accettore

Un donatore è un token doppiamente occupato ($n \geq 1{,}8$). Un accettore è un token vuoto ($n \leq 0{,}2$) che non è una funzione di polarizzazione di tipo $d$ (centro principale dominato da $d$ su un elemento senza shell $d$ di valenza, cioè fuori da Z 21–29, 39–47, 57–79) e non è il partner antilegante del legame del donatore stesso (stessi due centri principali, sovrapposizione del donatore $> 0$ e dell'accettore $< 0$). Una terza regola, il *sito*, esclude una coppia in cui ogni centro dell'uno è lo stesso atomo dell'altro o è legato a esso con pesi $\geq 0{,}20$ (CandidateConfig.site_weight, decisione dell'utente del 21 ottobre): un legame che interagisce con se stesso.

Fino alla versione 9 queste esclusioni cancellavano le celle; dalla v18 (P0a) sono flag 0/1 su ogni cella (physics.pairs.pair_flags): con pair_cells='valid' il modello legge solo le celle valide, con 'all' legge tutte le celle con i flag come canali (sezione 8.4).

## 7.7 SOMO, lacune e popolazione SOMO

I token a guscio aperto di un record calcolato come guscio aperto (il gruppo di plateau P2b, mai un livello con smearing) sono donatori e ricevono, all'assemblaggio del campione, un gemello virtuale, la loro *lacuna* (hole), con gli stessi coefficienti, centri, descrittori, occupazione e insieme spettrale, che è un accettore: una donazione di un elettrone nella vacanza della SOMO. I file di token non cambiano mai: il gemello esiste solo nel campione (tokens.with_holes). Una colonna spin_role vale +1 sul token aperto nel ruolo di donatore, −1 sulla lacuna, 0 altrove. Dentro un frammento un token aperto e una lacuna non sono mai una coppia candidata (un'eccitazione interna al guscio aperto); tra frammenti sono una ricombinazione, letta dal canale di ricombinazione. Un token aperto il cui centro principale è uno shell $d$ metallico (Cu(II), Fe(III)) non ha ruolo di default (metal_d_open_shell).

L'input atomico del guscio aperto è la popolazione SOMO ristretta

$$\rho_A = \sum_t \min(n_t, 2 - n_t)\, w_A^{(t)}$$

sui token aperti, che somma al numero ristretto di elettroni spaiati (1 per $Br^{\bullet}$ come $3 \times \frac{1}{3}$, 2 per $O_2$), è nulla su un guscio chiuso e non dipende dal disegno di risonanza. Un modello la seleziona (`atom_spin_input='somo_population'`), oppure il numero di elettroni spaiati del disegno come categoria (`'categorical_u'`), oppure niente (`'none'`).

# 8 Feature di coppia geometry-free (`ksmech.physics`)

La fisica entra nel modello come feature di coppia tra un token occupato $r$ (donatore) e uno virtuale $s$ (accettore), calcolate senza una geometria d'avvicinamento (spec «Geometry-free pair features»). Inizializzano la traccia delle coppie, che ogni testa d'attenzione legge come bias appreso.

## 8.1 Il fattore spettrale di gap $G$

$$G(r,s) = \sum_k \sum_m \frac{(n_k - n_m)\, p_{rk}\, p_{sm}}{\epsilon_k - \epsilon_m}, \tag{8.1}$$

calcolato dai vettori spettrali *completi* sugli orbitali canonici di ogni blocco. Ogni MO dello spettro di un token ha l'occupazione del token (i blocchi sono plateau di occupazione uguale), quindi $n_k - n_m$ è la differenza delle occupazioni dei due token: 2 su ogni coppia a guscio chiuso (identico bit per bit a tutte le versioni precedenti alla 10), 1 per una SOMO con un accettore vuoto e per un donatore doppio con una lacuna, la differenza di plateau per un gruppo aperto degenere, 0 (indefinito) per un token aperto con una lacuna. Per una coppia a guscio chiuso $G < 0$ e $|G|$ più grande significa gap effettivo più piccolo. I denominatori sotto $10^{-3}$ eV sono bloccati a $10^{-3}$ mantenendo il segno (`MIN_GAP_EV`). Colonne: `g` e `log_abs_g` $= \log\max(|G|, 10^{-12})$.

Su 245 698 coppie inter-frammento di un primo campione, 90 (0,037%) avevano $G > 0$: coppie donatore/accettore forti (enolato + $Br_2$, $CBr_3^-$ + fosfonio) in cui un livello del donatore sta sopra uno dell'accettore. Il segno è conservato.

## 8.2 Il gap a due livelli $g_2$ (P7)

Una SOMO sta dentro il gap a guscio chiuso vicino a un bordo, e tra molecole un livello del donatore può incrociare uno dell'accettore: i denominatori canonici possono essere minuscoli. La colonna

$$g_2(r,s) = \sum_{k,m} \frac{(n_k - n_m)\, p_{rk}\, p_{sm}}{\operatorname{sign}(\Delta_{km})\, D_{km}}, \quad D_{km} = \tfrac{1}{2}\Big(|\Delta_{km}| + \sqrt{\Delta_{km}^2 + 4\beta_{rs}^2}\Big), \quad \Delta_{km} = \epsilon_k - \epsilon_m \tag{8.2}$$

sostituisce ogni denominatore con quello esatto a due livelli: $|\Delta|$ a gap grande, $|\beta|$ alla degenerazione. È finita attraverso un incrocio, limitata da $1/|\beta|$ per unità di peso, conserva il segno di $G$ su una coppia invertita ed è uguale a $G$ a $\beta = 0$. Dentro una molecola $\beta_{rs}$ è l'accoppiamento tra centri (sotto) a $K = 1{,}75$; tra molecole, che non hanno geometria, è la costante $\beta_{\text{inter}} = 0{,}40$ eV.

**Decisione – `BETA_INTER` = 0,40 eV.** La costante è la mediana di $|\beta|$ intramolecolare di due token i cui insiemi di centri sono legati ma non condividono atomi (due orbitali a un legame di distanza, come due molecole a un legame che si forma), misurata una volta su 500 pool di `closed_base` e congelata prima di P8, non ottimizzata. Mediane (p90) per distanza di grafo: centro condiviso 1,16 (1,87) eV; legati 0,405 (1,46); due legami 0,091 (0,40); tre o più $\approx 0$.

Tabulazione diagnostica su 2,27 milioni di celle di `closed_base`: $|g|$ mediana 0,105, p99 0,333, p99,9 0,661, massimo 742,7 (il blocco); $|g_2|$ p99,9 0,608, massimo 4,59. Sui 57 pool a guscio aperto: $|g|$ massimo 38,2, $|g_2|$ 2,10.

## 8.3 Accoppiamento tra centri e allineamento

Dentro una molecola la posa dei reagenti è una geometria che i due orbitali condividono. Con gli integrali di risonanza di Wolfsberg–Helmholz sugli AO della molecola stessa, $\beta_{\mu\nu} =$

$KS_{\mu\nu}\frac{1}{2}(F_{\mu\mu}+F_{\nu\nu})$, $K = 1{,}75$ (con le energie diagonali mediate per shell, nulle dentro un atomo), l'**accoppiamento tra centri** è $|\boldsymbol{l}_r^{\top}\boldsymbol{B}\,\boldsymbol{l}_s|$ ristretto ai blocchi AO tra i centri di $r$ e di $s$. È non nullo anche se il termine (3) completo si annulla dentro la molecola: misura la pre-organizzazione, e tra centri legati porta il requisito antiperiplanare dell'iperconiugazione negativa. Il segno è una fase arbitraria dell'LMO, quindi si usa il modulo.

**Scoperta – la scorciatoia a livello di orbitale è sbagliata.** La forma $KS_{rs}\frac{1}{2}(F_{rr}+F_{ss})$ non va usata: per un accettore antilegante $F_{ss} > 0$ (il $\sigma^*$(C–O) del metanolo è a +2,7 eV) e la media si annulla spuriamente. Per questo la contrazione è fatta a livello di AO, dove le energie diagonali sono negative per ogni shell di valenza. $K$ è fissato al valore da manuale: ridimensiona solo una feature standardizzata, un $K$ apprendibile non porterebbe nulla.

Altre feature intramolecolari: distanza tra i centroidi (quanto deve ripiegarsi la catena), $|\cos|$ tra i vettori $p$ sui centri principali (assi dei lobi; indefinito senza carattere $p$), e il coseno del diedro sito–partner–partner–sito quando i due orbitali stanno su legami adiacenti ($-1$ è antiperiplanare: il requisito delle eliminazioni e delle migrazioni).

## 8.4 Il vettore di feature

**Tabella 8.1:** Colonne di coppia (`physics/stage_a.py`, `physics/pairs.py`).

| Colonna | Definizione |
|---|---|
| `g`, `log_abs_g` | fattore di gap $G$ e il suo logaritmo, su ogni coppia |
| `inter` | 1 se i due token stanno su frammenti diversi |
| `bonded`, `bond_order`, `graph_distance` | tra gli insiemi di centri sulla struttura di Lewis del reagente; distanza limitata a 6; tra frammenti 0, 0, 6 |
| `wbi` | massimo indice di Wiberg dei legami che uniscono i due insiemi di centri |
| `centroid_distance`, `cross_coupling` | intramolecolari (sopra); accoppiamento salvato a $K = 1$ |
| `align_cos`, `dihedral_cos` | allineamento stereoelettronico |
| `d_acceptor`, `own_antibond`, `site_self` | flag 0/1 delle tre esclusioni storiche (sezione 7.6), dalla v18 |
| `g2`, `log_abs_g2` | gap a due livelli (P7) |
| `own_g`, `own_g2` | canale own-pair: $G$ e $g_2$ di un token legante con il proprio antilegante, solo su quelle celle |
| `somo_pair`, `somo_level_diff` | canale di ricombinazione: flag e differenza di livello $\epsilon_r - \epsilon_s$ sulle coppie token aperto × lacuna di due frammenti, dove $G$ e $g_2$ si annullano |

Il lettore prende: con `pair_cells='valid'` le 11 feature di base, indefinite fuori da `pair_valid`; con `'all'` le 14 salvate più `pair_valid` come colonna (15); una run open-shell aggiunge le 6 colonne di `OPEN_SHELL_BLOCK` (21). Ogni voce non definita (una feature intramolecolare tra frammenti, un allineamento senza carattere $p$) è zero e mascherata.

**Standardizzazione.** Valori arrotondati a float16 come salvati, convertiti ai parametri fisici (solo `cross_coupling` scala con $K$), media e deviazione standard sulle voci definite delle celle scelte del training (`pair_feature_stats_<split>[_allcells].json`, con intestazione di provenienza). Una run multi-sorgente o open-shell rifà il fit sui campioni di training della run stessa.

**Scoperta – le esclusioni cancellavano informazione.** Fino alla v17 le tre regole cancellavano celle: la regola «tipo $d$» toglieva il 10,4% dei token virtuali e, nel 15,2% dei passi di training PMechDB (1 520 su 9 997), più di metà del peso aggregato sull'ancora annotata (O–Si, C–S, O–P, Al–O); le regole own-antibond e sito toglievano l'1,6% delle celle, inclusa la coppia orbitale della freccia annotata in 237 passi di training e 34 di validazione. Come canali sono input che il modello pesa; la decisione P0b (GENERAL non peggiore di LEGACY) ha reso definitivo questo passaggio.

## 8.5 Dalla coppia di orbitali alla coppia di atomi

Il decodificatore lavora su coppie di atomi. La fisica della coppia $(i, j)$ è quella degli orbitali che i due atomi portano, nelle due direzioni di donazione. Con i pesi di canale $\omega_{ij}^{rs} = w_i^{(r)} w_j^{(s)} + w_j^{(r)} w_i^{(s)}$, le feature standardizzate sono aggregate nella media pesata sulle voci definite, e il **fattore di gap aggregato** è

$$G_{ij}^{\mathrm{atom}} = \sum_{(r,s)\in\mathcal{V}} \left(w_i^{(r)} w_j^{(s)} + w_j^{(r)} w_i^{(s)}\right) G(r,s), \tag{8.3}$$

dove $\mathcal{V}$ sono le celle valide (legacy) o tutte (general). Il vettore atomico $\boldsymbol{y}_{ij}$ contiene le medie aggregate, $\log(1 + \text{peso aggregato})$, $\operatorname{asinh} G_{ij}^{\mathrm{atom}}$ e l'indice di Wiberg (larghezza $F + 3$). Il canale own-pair è aggregato a parte, $B_{ij}^{\mathrm{own}} = \sum_{(r,s)\ \mathrm{own}} \omega_{ij}^{rs} G(r,s)$ sulle coppie legate, e misura l'omolisi di quel legame senza gonfiare $G^{\mathrm{atom}}$. La coppia $(r, s)$ con il termine più grande è la lettura orbitale del legame che si forma, mostrata insieme alla coppia canonica dominante $(k, m)$ di $G$.

## 8.6 Storia: lo stage B e la geometria di contatto (archiviato)

Dal 20 al 25 ottobre il progetto ha costruito e misurato una via alternativa: completare il termine (3) con un accoppiamento valutato a una geometria di contatto. In sequenza:

1. **tabella di contatto** di sovrapposizioni diatomiche per coppie di elementi e shell, campionate a contatti di van der Waals (∼353 000 campioni): il «gate» non addestrato ($KS_3$ deve mettere una coppia etichettata nel top-5 di metà dei passi) fallì all'8,2%, mentre $G$ da solo arrivava al 44,9%. Causa: la sovrapposizione di contatto è sbilanciata verso l'1$s$ di H e gli shell $s$ ($\beta^2$ 50–100 volte più grande per coppie con H);
2. **kernel orientato analitico** (base GFN2 riscritta in numpy, sovrapposizioni con Obara–Saika, lobo su 200 direzioni di Fibonacci, accettore posto a distanza $d$ lungo il lobo del donatore): fallisce a ogni $d$ (2,0–3,0 Å); il lobo di un orbitale di legame punta al suo partner e l'accettore finisce dentro il legame del donatore;
3. **lobi sul sito**, guardia anti-collisione, rotazioni, solo il «cap» locale dell'accettore: $KS_3 + KS_2$ a 2,3 Å arrivò al 44,9% top-5 su train+val, e fu adottato come floor non addestrato e come kernel della testa congelata;
4. il 25 ottobre, con le etichette sulle ancore e lo split per molecola, il test top-1 era circa 3% per la testa congelata, 14,3% per $KS_3 + KS_2$, 20,4% per $G$ da solo.

**Decisione – 25 ottobre, spec v9.** Lo stage B esce da `main`: niente geometria di contatto, niente bias d'attenzione di Klopman–Salem, niente testa congelata; tutte le teste apprese. Moduli, comandi (`contacts`, `contact-table`, `ks-rank`, `baseline`) e test sono archiviati nel ramo locale `archive/stage-b` (`8811e57`, spec v8.1). Restano $G$, l'accoppiamento WH a $K = 1{,}75$, le energie AO mediate per shell e la regola del sito.

# 9 Etichette e campioni (`ksmech.arrows`)

## 9.1 Dalle frecce alla matrice di reazione

Fino al 23 ottobre il bersaglio era la sequenza di frecce, ciascuna allineata a una coppia di orbitali (donatore, accettore) più una scelta del legame uscente. Il round trip delle frecce *predette* (tuple orbitali riconvertite in modifiche) arrivò al 99,67% (11 826/11 865) dopo molte regole (legame uscente come scelta categorica per freccia, regola dell'indice di Wiberg più basso, maschere sequenziali), ma i 39 casi rimasti erano irrisolvibili per principio: 12 Mitsunobu (nessun token occupato copre N–P, Wiberg 0,45–0,60), donatori iperconiugati a tre centri, legami C–Si, C–Sn, Li–O. Un orbitale localizzato non può dire se si è mosso «il doppietto» o «il legame».

**Decisione – 23 ottobre, spec v8.** Il bersaglio diventa la matrice di reazione $\boldsymbol{R} = \Delta\boldsymbol{B}$, applicata come $\boldsymbol{B} + \boldsymbol{R}$; le frecce sono una lettura. Etichette dalle frecce del dataset applicate come un'unica modifica, modulo risonanza; il codice delle frecce resta come diagnostica. Copertura immediata 98,4% contro 92,4% con le etichette per freccia; la modifica riproduce il prodotto nel 100% dei casi.

Misure su 12 830 passi: variazioni di ordine di legame sempre $\pm 1$, da 1 a 8 per passo (1: 1 876; 2: 7 332; 3: 1 736; 4: 1 443; 5–7: 236); elettroni non leganti $\pm 2$ su 0–2 atomi; nuovi legami 8 904 intermolecolari e 1 028 intramolecolari a distanza di grafo da 2 a 9: ogni coppia di atomi è candidata, senza soglia di distanza.

## 9.2 La classe parsimoniosa

«Modulo risonanza» preso alla lettera è inutilizzabile: in 2 727 passi esistevano più di 16 matrici equivalenti. L'etichetta è quindi la **classe parsimoniosa** di $\boldsymbol{R}_{\text{data}}$: ogni $\boldsymbol{R}$ tale che $\boldsymbol{B}_{\text{reac}} + \boldsymbol{R}$ sia una forma di risonanza del prodotto annotato (forme RDKit con H congelati, strutture dispari scartate, al più 256, le forme di un radicale tramite il proxy anionico, $\boldsymbol{R}_{\text{data}}$ sempre inclusa) il cui flusso è bilanciato, scelte prima per il minor numero di cariche formali nella struttura e poi per il minor numero di voci cambiate. Solo quando nessun candidato ha un flusso bilanciato la classe è presa senza quella legge (e il passo è un trasferimento elettronico dissociativo). Contare solo le voci preferirebbe forme a cariche separate (un idrazone scritto $N^-$–C=$OH^+$, un carbossilato $O^+$=C–$O^-$). Una classe contiene solo passi elementari (ogni ordine di legame cambia di 1, ogni $N$ di 2, o di 1 per una sorgente a un elettrone).

Su PMechDB la classe ha un membro nel 94,5% dei passi e due nel 5,0%; $\boldsymbol{R}_{\text{data}}$ è già parsimoniosa nel 94,8%. Il 99,6% dei passi di training ha un'etichetta; il resto coinvolge molecole il cui calcolo GFN2 non converge (soprattutto addotti di Ti e Sn). L'**ancora** di un membro è una qualsiasi delle sue voci $+1$ (un nuovo legame, o un legame $\pi$ su una coppia già legata), oppure, se non ne ha, una voce $-1$ (un'eterolisi): 11 838 ancore di formazione e 785 di rottura.

## 9.3 Chiusura per simmetria (schema 17)

**Scoperta – il plateau del 60% era in parte simmetria.** Analizzando le previsioni delle prime run: nel 21,5% dei campioni di validazione l'etichetta nomina uno di più atomi equivalenti per simmetria (un H di $H_3O^+$, di un $CH_3$ o di un $NH_3^+$, un atomo di una di due molecole identiche). Il modello impara la simmetria e dà a ciascun H di $H_3O^+$ probabilità 0,34: su quei campioni il top-1 è un lancio di moneta (18% contro 71% sugli altri), e la loss ha un pavimento di $\log k$. In bf16 gli atomi equivalenti pareggiano esattamente, e il 5,1% della validazione erano «hit» solo per pareggio: lo stesso `best.pt` valeva 63,4% in bf16 e 59,2% in float32.

Correzioni (26 ottobre): ogni automorfismo della struttura di Lewis del pool (elementi, cariche, ordini di legame, ogni H un atomo, niente stereochimica) mappa ogni variante in una equivalente,

e le immagini entrano nella classe con le loro ancore (`arrows.symmetry.symmetric_closure`). Il gruppo non è enumerato: gli H con un solo vicino pesante sono ripiegati sul vicino, le immagini distinte degli atomi dell'etichetta sono enumerate per backtracking dentro le classi di simmetria di RDKit, poi gli H sono redistribuiti (limiti: 1 000 piazzamenti, 200 000 controlli, 32 membri). E la metrica conta un pareggio con un candidato fuori dall'etichetta come errore. Effetto: il 25,2% delle classi guadagna membri; `flex_first` rivalutato senza riaddestramento passa da 55,9% a 63,9% di top-1 dell'ancora, identico in float32 e float16.

## 9.4 Augmentation di risonanza

Le cariche formali dei token atomici, la matrice $\boldsymbol{B}$ su cui si decodifica $\boldsymbol{R}$ e la feature `bond_order` appartengono a un disegno del pool; la funzione d'onda no. Senza augmentation il modello impara la scelta di disegno del dataset (decisione dell'utente del 25 ottobre). Ogni molecola del pool è disegnata nelle sue forme di risonanza con non più cariche formali del disegno originale (niente forme minori a cariche separate: RDKit restituisce $\geq 50$ strutture per 76 molecole su 875, quasi tutte zwitterioni); la classe di etichette è ricalcolata per ogni disegno, e una forma conserva la struttura di Kekulé disegnata di ogni anello che il suo cambiamento non richiede (`kekule_closest`: RDKit scambiava anelli che la carica non raggiunge, facendo crescere un'etichetta fino a 24 voci). Fino a 8 disegni per campione (`MAX_DRAWINGS`); ogni epoca ne estrae uno a caso (seme ed epoca). Nel 29% dei campioni esistono altri disegni (7 864 in tutto). Validazione e test leggono sempre il disegno del dataset.

**Scoperta – un'etichetta di disegno deve essere elementare.** La prima run con augmentation (`flex_resonance` su RTX 5080) si fermò all'epoca 2 con un assert CUDA nella loss: l'etichetta di un disegno cambiava un ordine di legame di 2 (un triplo che il prodotto vuole singolo). 5 dei 7 864 disegni avevano una classe così. Da allora una classe contiene solo passi elementari e i disegni non elementari sono scartati e contati.

Le coppie di risonanza (471 su PMechDB) richiederebbero un termine di consistenza proprio, rimandato finché una run non mostri che l'invarianza al disegno manca ancora.

## 9.5 ReactMech: etichette senza frecce

ReactMech non ha frecce. $\boldsymbol{R}_{\text{data}}$ è la differenza dei due lati mappati sugli indici del reagente; gli atomi non mappati (spettatori come $Et_3N$ o $CO_2$, in 3 294 passi) sono invariati.

- **Idrogeni mobili.** ReactMech scrive ogni H come conteggio sull'atomo pesante: un protone da O a Cl si legge `[OH+:5]` → `[O:5]` e `[Cl-:1]` → `[ClH:1]`, e la sola differenza degli atomi pesanti mostra due cambi di doppietto e nessun legame: il passo non ha ancora. Ogni H mobile è riscritto come atomo mappato con un numero nuovo, come fa PMechDB. Nei 9 183 passi di training del pilota: 4 729 non muovono H, 4 443 ne muovono uno, 11 due donatori verso un accettore, nessuno ambiguo.
- **Kekulé più vicino** (P0a). I due lati sono kekulizzati indipendentemente, quindi un anello aromatico può avere i doppi legami in posizioni diverse e la differenza chiederebbe al modello di spostare legami $\pi$ che il passo non cambia: nel 9,4% delle etichette ReactMech ogni membro della classe portava un ciclo alternato. Il prodotto è ridisegnato nella struttura di Kekulé più vicina al reagente: scatta su 868 campioni, 866 etichette si accorciano, le etichette con più di 8 voci di legame passano da 237 a 0.

**Scoperta – il rumore delle etichette nascondeva 6 punti.** Gli stessi checkpoint rivalutati dopo P0a: il modello ReactMech `stop0` passa da 85,0% a 91,1% full-R top-1 e quello congiunto da 86,0% a 91,9% senza alcun riaddestramento: 61 campioni con Kekulé più vicino diventano hit. La validazione ReactMech di P0 era sottostimata di circa 6 punti di rumore delle etichette.

## 9.6 Il campione e lo store

Un `MechanismSample` contiene: gli input di tutti i frammenti reagenti in un unico spazio di indici (token atomici e orbitali), la struttura di Lewis sui token atomici (legami, doppietti, $N$ e $u$ per atomo e per disegno, indici di Wiberg), le feature di coppia geometry-free di ogni cella con i flag, i disegni di risonanza con le loro etichette, la classe di etichette con le ancore, la chiave d'identità del prodotto, la lettura orbitale diagnostica (`aligned`, `HETEROLYSIS_ONLY`, `NO_ARROWS`…), la classe di dominio. Gate di costruzione: token presenti, atomi dell'etichetta tokenizzati, indice di Wiberg per ogni legame, `lewis.check_edit` (conservazione degli elettroni, ordini in $[0, 3]$, doppietti $\geq 0$, ancore che passano le maschere).

Lo store (`arrows.store`) è fatto di shard `.npz` compressi da 256 campioni più un `index.json` e una vista JSON per split e parte (`samples_<split>_<part>.json`); nessuna copia per split. Le feature di coppia sono salvate in float16 con piani di byte separati (senza perdita, compressione migliore del 20%), i flag binari impacchettati a bit. Errore massimo assoluto di 5e-3 su feature logaritmiche che coprono circa 30 unità. Un campione da 60 atomi pesanti occupa 0,32 MB; lo store completo meno di 1 GB.

**Tabella 9.1:** Evoluzione di `SAMPLE_VERSION`.

| Versione | Contenuto |
|---|---|
| 8–13 | provenienza; filtro delle etichette; feature stage-A e sidecar di contatto; maschere sequenziali; legame uscente $\lambda$ (era delle frecce) |
| 14 | chiave sigma del prodotto |
| 15 | bersaglio $\boldsymbol{R}$; struttura di Lewis nel campione; l'allineamento non filtra più i campioni |
| 16 | disegni di risonanza |
| 17 | classi chiuse per simmetria |
| 18 | (P0a) ogni cella di coppia con i flag, extra dei token, etichette Kekulé più vicino, `domain_class` |
| 19 | (P8) $N$ e $u$ per atomo e disegno, chiave d'identità v2, lacune, `spin_role`, `somo_population`, $g_2$, canali own-pair e di ricombinazione |

**Dimensioni.** Su PMechDB un pool ha circa 3,6 token per atomo: mediana 124 token (p90 287, p99 483, massimo 1 487) per una mediana di 34 atomi (massimo 411, idrogeni inclusi). La traccia densa del pool più grande ha 2,2 milioni di celle. ReactMech: mediana 251 token per pool, FlowER 336 (l'1% dei passi FlowER supera il limite di 1 024 token).

# 10 Il modello (`ksmech.model`)

`KSMech` ha tre parti: `prepare` (float32, senza gradiente né autocast: input dei token, $\hat{\boldsymbol{z}}$, maschere «defined», $\boldsymbol{y}_{ij}$), un encoder (denso, flex o sparse, stessa interfaccia) e il decodificatore `EditDecoder`.

## 10.1 Configurazione

**Tabella 10.1:** `ModelConfig` (`model/config.py`), valori di default.

| Campo | Default | Significato |
|---|---|---|
| `d` / `d_pair` | 256 / 64 | larghezza dei token / delle coppie |
| `layers` / `heads` | 8 / 8 | strati dell'encoder / teste ($d_{\text{head}} = 32$), tutte apprese |
| `rbf`, `rbf_max` | 32, 8,0 Å | gaussiane della distanza su $[0, 8]$ Å, larghezza 8/31 Å |
| `fragments` | 8 | righe dell'embedding di frammento (indice mod 8) |
| `decoder_hidden` | 128 | larghezza delle teste delle voci e della testa di stop |
| `physics_hidden` | 32 | larghezza di $\text{MLP}_Y$ (fisica della coppia di atomi) |
| `K` | 1,75 | costante di Wolfsberg–Helmholz (fissa) |
| `drop_pair_features` | () | feature di coppia rese indefinite (ablazioni) |
| `pair_cells` | `valid` | `valid` (legacy) o `all` (con i flag) |
| `orbital_extra` | () | `retained_fraction`, `energy_ev` |
| `frontier_blocks` | `window` | finestra 0,05 eV o `exact` ($10^{-4}$ eV) |
| `atom_fukui` | `frozen` | `frozen`, `fd`, `both` |
| `atom_spin_input` | `none` | `none`, `categorical_u` (max 7), `somo_population` |
| `spin_role` | False | colonna orbitale $\pm 1$ per SOMO e lacuna |
| `open_shell` | False | testa atomica a 5 classi e blocco open-shell delle coppie |
| `somo_channels` / `own_pair_channel` | True / True | canali di ricombinazione / own-pair |
| `checkpoint` | True | ricalcolo delle attivazioni per strato |
| `encoder` | `dense` | `dense`, `flex`, `sparse` |
| `topk`, `indexer_heads`, `indexer_dim` | 64, 4, 32 | encoder sparse |

**Tabella 10.2:** Preset (`-preset`); un'opzione esplicita vince sul preset. `frontier_blocks` e `atom_fukui` restano legacy in ogni preset.

| Preset | Campi impostati | $F$ | canali | $\lvert\boldsymbol{y}\rvert$ | atomo | orbitale |
|---|---|---|---|---|---|---|
| `legacy` (default) | `pair_cells=valid`, nessun extra | 11 | 65 | 14 | 20 | 23 |
| `general` | `pair_cells=all`, extra `retained_fraction`, `energy_ev` | 15 | 73 | 18 | 20 | 25 |
| `open_shell` | `general` + `open_shell`, `somo_population`, `spin_role` | 21 | 85 | 24 | 22 | 26 |

Numero di parametri (encoder denso): 7 372 587 in legacy (encoder 6 910 208, decoder 462 379), 7 373 739 in general, 7 375 725 in open-shell; flex circa 7,11 milioni, sparse circa 7,52. Nessun dropout in tutto il modello e nessuna inizializzazione personalizzata (default di PyTorch), tranne il token MECH ($\mathcal{N}(0,1)/\sqrt{d}$) e le tabelle di bias di flex.

## 10.2 Embedding dei token

**Token atomico.**

$$\boldsymbol{h}_a^{(0)} = \boldsymbol{E}_Z[Z_a] + \boldsymbol{E}_{\text{chg}}[\text{clip}(q_a, -4, 4)]\ [+\boldsymbol{E}_u[\text{clip}(u, 0, 7)]] + \text{MLP}_A(\boldsymbol{x}_a),$$

con $\boldsymbol{E}_Z$ di 119×256 righe, $\boldsymbol{x}_a$ gli input continui standardizzati più un flag di mancanza per colonna, $\text{MLP}_A$ lineare–GELU–lineare a 256.

**Token orbitale.**

$$\boldsymbol{h}_r^{(0)} = \text{MLP}_O(\boldsymbol{v}_r) + \sum_k \text{MLP}_\Sigma([\epsilon_k, p_{rk}]) + \sum_{a \in \mathcal{A}_r} w_a^{(r)} \boldsymbol{h}_a^{(0)},$$

dove il secondo termine è un set encoder (DeepSets) sulle fino a 8 voci spettrali reali (energia standardizzata, peso), e il terzo fa ereditare a un orbitale l'ambiente chimico dei suoi atomi. Ogni token riceve un embedding di frammento $\boldsymbol{E}_{\text{frag}}[\phi \bmod 8]$; un token di sintesi MECH appreso è anteposto: la sequenza è [MECH, atomi, orbitali], lunghezza $N + 1$.

**Standardizzazione.** Statistiche (`FeatureStats`) adattate sui campioni di training della run, per nome di colonna, salvate in ogni checkpoint; deviazione nulla sostituita da 1; le colonne `somo_population` e `spin_role` hanno scala fissa (media 0, deviazione 1).

## 10.3 La traccia delle coppie (encoder denso)

I canali sulla griglia $(N + 1) \times (N + 1)$ sono $2F + 43$:

1. RBF della distanza (32), tra posizioni atomiche e centroidi orbitali, solo dentro un frammento (le coppie intermolecolari non hanno distanza nella posa dei reagenti);
2. le feature $\hat{\boldsymbol{z}}$ ($F$) e la loro maschera di definizione ($F$) su ogni cella (occupato, virtuale), scritte su entrambe le celle $(r, s)$ e $(s, r)$, con un canale di orientamento (+1 dall'occupato, −1 indietro) così che l'attenzione di un virtuale veda anche i suoi donatori;
3. il flag di stesso frammento;
4. sulle coppie di atomi: legato, ordine di legame, $\min(d_{\text{grafo}}, 6)/6$;
5. il peso sul centro $w_a^{(r)}$ tra un orbitale e i suoi atomi;
6. un one-hot del tipo di coppia (atomo–atomo, atomo–orbitale, orbitale–atomo, orbitale–orbitale, MECH).

$\boldsymbol{p}^{(0)} = \text{MLP}_P(\text{canali})$, larghezza 64.

## 10.4 Lo strato dell'encoder denso

Pre-norm, 8 strati:

$$\boldsymbol{q}, \boldsymbol{k}, \boldsymbol{v} = \boldsymbol{W}_{qkv}\,\text{LN}(\boldsymbol{h}), \qquad \ell_{ij}^{(h)} = \frac{\boldsymbol{q}_i^{(h)\top} \boldsymbol{k}_j^{(h)}}{\sqrt{32}} + \boldsymbol{w}_h^\top \text{LN}(\boldsymbol{p}_{ij}) + m_{ij},$$

$$\boldsymbol{h} \leftarrow \boldsymbol{h} + \boldsymbol{W}_o\,\text{softmax}(\ell)\,\boldsymbol{v}, \qquad \boldsymbol{h} \leftarrow \boldsymbol{h} + \text{FFN}(\text{LN}(\boldsymbol{h})) \quad (256 \to 1024 \to 256,\ \text{GELU}),$$

$$\boldsymbol{p}_{ij} \leftarrow \boldsymbol{p}_{ij} + \boldsymbol{U}_{\text{out}}\,\text{GELU}(\boldsymbol{U}_p \text{LN}(\boldsymbol{p}_{ij}) + \boldsymbol{U}_i \text{LN}(\boldsymbol{h}_i) + \boldsymbol{U}_j \text{LN}(\boldsymbol{h}_j)).$$

Il bias $\boldsymbol{w}_h^\top \boldsymbol{p}_{ij}$ (lineare 64→8, in float32) è il modo in cui ogni testa vede la fisica di coppia; logit, softmax e loss restano sempre in float32. Ogni strato è avvolto in activation checkpointing: la memoria delle attivazioni è circa quella di uno strato. L'uscita è $\boldsymbol{h}$ e il blocco atomico simmetrizzato $(\boldsymbol{p}_{ij} + \boldsymbol{p}_{ji})/2$, perché $\boldsymbol{R}$ è simmetrica.

## 10.5 Encoder alternativi

**flex** (FlexAttention, solo GPU). Nessuno stato di coppia: in `score_mod` ogni strato e testa aggiunge un bias appreso lineare nelle feature di coppia: una funzione radiale della distanza (tabulata a 50 punti per Å fino a 10 Å e interpolata, perché FlexAttention non può indicizzare più di una volta un tensore con gradiente), tabelle per ordine di legame e distanza di grafo, un peso di Wiberg, un peso di frammento, e $\hat{\boldsymbol{z}}$ proiettato per testa sulle celle (occupato, virtuale). Le piccole tabelle sono copiate per (campione, testa, query): lette direttamente, gli *atomic add* del backward si serializzavano (2,28 s contro 0,30 s). Il decodificatore riceve vettori di coppia da un `AtomPairHead` statico.

**sparse** (alla DeepSeek Sparse Attention). Ogni query attende a MECH, a se stessa e alle sue 64 chiavi migliori secondo un indicizzatore leggero ($I_{ij} = \sum_h w_{ih}\mathrm{ReLU}(\boldsymbol{q}_{ih} \cdot \boldsymbol{k}_j)$, 4 teste da 32, su token staccati dal grafo), addestrato solo con una KL verso l'attenzione principale; durante il warm-up del learning rate l'attenzione è densa.

**Tabella 10.3:** Encoder sulla RX 6600 XT (split per molecola, budget 1,1 M celle, fp16, ricalcolo).

| Encoder | Parametri | Picco (batch più pieno) | s/batch | Epoca |
|---|---|---|---|---|
| denso | 7,37 M | 3,14 GiB | 0,87 | ∼6,2 min |
| flex | 7,11 M | 0,59 GiB | 0,54 | ∼3,8 min |
| sparse (top-64) | 7,52 M | 1,23 GiB | 0,70 | ∼5,0 min |

Alle dimensioni di PMechDB una query ha qualche centinaio di chiavi: il top-64 risparmia memoria, non tempo. L'encoder denso è risultato il più accurato (sezione 13.2) ed è quello di tutte le run di confronto.

## 10.6 Il decodificatore delle modifiche

**Ancora.** Dalla query di passo $\boldsymbol{s} = \mathrm{MLP}_S(\boldsymbol{h}_{\mathrm{MECH}})$ (nessun indice di passo: lo stato è il pool), per ogni coppia $i < j$ e segno $\sigma \in \{\mathrm{form}, \mathrm{break}\}$:

$$\mathrm{logit}(i, j, \sigma) = \boldsymbol{h}_i \cdot \boldsymbol{u}_\sigma + \boldsymbol{h}_j \cdot \boldsymbol{u}_\sigma + \boldsymbol{U}_\sigma(\boldsymbol{p}_{ij}) + \mathrm{MLP}_Y(\boldsymbol{y}_{ij})_\sigma,$$

con $\boldsymbol{u}_\sigma = \boldsymbol{W}_{\mathrm{sign}}\boldsymbol{s}$; l'ultimo termine è l'uso appreso della fisica della coppia di atomi. Maschere: `can_form` (atomi reali distinti con ordine $< 3$), `can_break` (ordine $> 0$). Log-softmax su tutti i $A \times A \times 2$ candidati.

**Voci date l'ancora** $(a, c)$**.** Con $\boldsymbol{h}_{\mathrm{anc}} = \boldsymbol{h}_a + \boldsymbol{h}_c$, i vettori di coppia che uniscono ogni atomo all'ancora e un embedding della distanza di grafo dall'atomo d'ancora più vicino (0–6, 7 = altra molecola):

- testa di coppia su $\{-1, 0, +1\}$ (MLP a 128); maschere: niente $-1$ senza legame, niente $+1$ su un triplo;
- testa atomica su $\{-2, 0, +2\}$, o $\{-2, -1, 0, +1, +2\}$ con `open_shell`; maschere: $-2$ richiede $N \geq 2$, $-1$ richiede $N \geq 1$. Nessuna regola sul pool entra nella testa o nella loss.

**Testa di stop.** $P(\texttt{END_MECH}) = \sigma(\mathrm{MLP}(\boldsymbol{h}_{\mathrm{MECH}}))$, con loss BCE pesata da $\lambda_{\mathrm{stop}}$ (0,5 di default, 0 su PMechDB e rmechdb).

**Scoperta – la testa di stop non è mai addestrata.** Nessuna sorgente fornisce ancora etichette di stop: `collate_samples` non produce mai `stop_label`, quindi il termine BCE è sempre assente, qualunque sia `-stop-weight`, e il decoder non legge la testa. Servirà una sorgente con pathway (differita: D3).

## 10.7 Loss

Ogni passo annotato dà una classe $\mathcal{R}$ di matrici equivalenti, ciascuna con le sue ancore ammissibili $\mathcal{E}(\boldsymbol{R})$. La loss marginalizza esattamente su entrambe:

$$\mathcal{L}_{\text{step}} = -\log \sum_{\boldsymbol{R}\in\mathcal{R}} \sum_{e\in\mathcal{E}(\boldsymbol{R})} P(e) \prod_{x\neq e} P(R_x \mid e),$$

dove $x$ percorre ogni coppia $i < j$ (zeri inclusi) e ogni atomo reale: il modello impara che la maggior parte della molecola non cambia. Calcolata con un log-sum-exp segmentato per campione; la loss del batch è la media sui campioni, più l'eventuale KL dell'indicizzatore sparse. Riportate accanto: la NLL della sola ancora (deduplicata per ancora: un'ancora condivisa da più varianti era contata più volte e la sua probabilità poteva superare 1, una NLL di $-0{,}045$ in un log di `flex_first`) e la $\boldsymbol{R}$ «greedy» (ancora più probabile più argmax di ogni voce).

# 11 Training (`ksmech.train`)

## 11.1 Configurazione

**Tabella 11.1:** `TrainConfig` (`train/loop.py`) e costanti collegate.

| Campo | Default | Significato |
|---|---|---|
| `source` | `pmechdb` | `a+b` per più sorgenti |
| `split` | `molecule` | anche `pmechdb=molecule,rmechdb=rc_molecule` |
| `epochs` | 30 | |
| `cell_budget` | 1 100 000 | celle della traccia di coppia (con padding) per batch |
| `max_batch` | 64 | |
| `max_tokens` | 1 024 | campioni più grandi saltati (`TOO_LARGE`, contati) |
| `lr`, `weight_decay` | $3 \cdot 10^{-4}$, 0,01 | AdamW; decadimento solo sui parametri con $\geq 2$ dimensioni; $\beta = (0{,}9; 0{,}999)$, $\epsilon = 10^{-8}$ (default di PyTorch) |
| `warmup` | 2 000 | passi di warm-up lineare, poi coseno fino a fine run |
| `grad_clip` | 1,0 | norma del gradiente |
| `amp` | `auto` | float16 su ROCm, bfloat16 su CUDA, niente su CPU |
| `seed` | 0 | `torch.manual_seed`; sampler `default_rng((seed, epoch))` |
| `workers` | 2 | DataLoader con fork (i 4,4 GB di campioni sono condivisi) |
| `stop_weight` | $-1$ | $< 0$: default della sorgente (0,5; 0 su PMechDB e rmechdb) |
| `resonance_draws` | True | un disegno casuale per campione e epoca |
| `eval_test` | False | valuta anche il test ogni epoca (solo report) |
| `drop_domain_class` | () | classi di dominio tolte dal training (ablazione dei metalli) |
| `val_cohort` | – | restringe la validazione a una coorte (`pmechdb=closed_base`) |

Schedula: $\mathrm{lr}(t) = \mathrm{lr} \cdot (t+1)/2000$ per $t < 2000$, poi $\mathrm{lr} \cdot \frac{1}{2}(1 + \cos(\pi \min(1, (t-2000)/(T-2000))))$ con $T$ = epoche $\times$ batch per epoca. Il GradScaler in float16 parte da $2^{10}$: al default $2^{16}$ le prime loss (circa 700 nat, una somma su tutte le voci) andavano in overflow nei primi dieci backward.

## 11.2 Batching a budget di celle

Il costo di un campione è $(n_{\text{token}} + 1)^2$. Ogni epoca il `CellBudgetSampler` permuta i campioni, li taglia in gruppi da 1 024 ordinati per dimensione e li impacchetta finché $B \cdot (N_{\max} + 1)^2 \leq$ budget o $B = 64$, poi mescola i batch. L'ordine dipende solo da (seme, epoca). Con il budget di 1,1 M celle un'epoca su PMechDB ha circa 425 batch; sulla RX 6600 XT 1,0 M celle danno un picco di 3,2 GiB e circa 7,7 minuti per epoca, 2,4 M toccano il limite di 8 GiB.

## 11.3 Caricamento e filtri

La vista `samples_<split>_<part>.json` è verificata (provenienza, obsolescenza, albero pulito). Filtri in ordine: `-limit`; solo in training `-drop-domain-class` (`DROPPED_DOMAIN_CLASS`) e, per rmechdb in una run con pmechdb, il `LEAK_GUARD`; `TOO_LARGE`; i disegni con classe non elementare. Una run multi-sorgente unisce le parti di training, valida per sorgente (`val_<source>_*`) e sul pool, e adatta le statistiche di coppia sul pool.

## 11.4 Gestione dei valori non finiti

**Scoperta – NaN produceva un top-1 del 100%.** La run ReactMech sparse con $\lambda_{\text{stop}} = 0{,}5$ è diventata non finita al passo ∼9 480: il GradScaler in fp16 ha dimezzato la scala a ogni overflow fino a zero; a scala zero i gradienti non scalati sono $0 \cdot \infty = \text{NaN}$, e il controllo degli infiniti li manca perché guarda i gradienti scalati, che sono 0. 257 tensori di pesi su 261 sono diventati NaN e la run è andata avanti per 11 epoche; poiché NaN confrontato con qualunque cosa dà falso, ogni rango era 0, il top-1 di validazione 1,000, e `best.pt` è stato sovrascritto con i pesi NaN (il checkpoint buono dell'epoca 18, 0,900, è andato perso). I gradienti della run solo ReactMech erano circa 10 volte quelli della run congiunta (mediana 116 contro 12).

Da allora: un batch con loss non finita non fa backward né passo dell'ottimizzatore ed è contato (`nonfinite_batches`); la run si ferma (exit 3, `NONFINITE`) dopo 50 batch di fila, se la scala fp16 scende sotto 1 o se un parametro non è finito (controllo ogni 20 passi e a fine epoca), prima di scrivere i checkpoint dell'epoca; un punteggio NaN conta contro l'etichetta; un'epoca con loss non finita non è mai `best.pt`.

## 11.5 Metriche, checkpoint e directory della run

Ogni epoca: loss, NLL dell'ancora, top-1/3/5 dell'ancora con marginalizzazione sull'insieme (i pareggi contano contro l'etichetta), $R$ esatto (la $\boldsymbol{R}$ greedy è una variante della classe), $p_{\text{stop}}$, e sui pool con un elettrone spaiato `open_top1`, `open_r_exact`, `open_n`. `best.pt` è il checkpoint con il miglior (top-1 di validazione, −loss); `last.pt` è scritto a ogni epoca. Il checkpoint contiene provenienza, `ModelConfig`, statistiche dei token e delle coppie, stato del modello, dell'ottimizzatore, della schedula e dello scaler. `load_model` rifiuta un checkpoint addestrato da un albero sporco; `-resume` rifiuta input o configurazioni diversi da `last.pt`. La directory `data/models/<source>/<run>/` contiene `run.json` (provenienza delle viste e delle statistiche, parti usate con i salti contati, criterio di selezione), `train.log` nel formato `step n / total` letto dallo strumento `ksmech-eta`, `metrics.jsonl`, i checkpoint e i report di decodifica.

## 11.6 Hardware e tempi delle run

**Tabella 11.2:** Ambienti di training usati.

| Macchina | Run | Note |
|---|---|---|
| RX 6600 XT locale (ROCm 7.2) | smoke run, P0b timing | 1 epoca PMechDB: 429 s (general), 435 s (legacy), ∼23 campioni/s; 30 epoche ≈ 3,6 h |
| RunPod RTX 3090 Ti (cu130) | `flex_first` | 27 minuti per 30 epoche, ∼220 campioni/s, 3,3 GB |
| RunPod RTX 5080 | le cinque run del 26 ottobre | 25–45 minuti per run |
| RunPod RTX 5090 (cu130) | P0b (12 run) | PMechDB 75 minuti, ReactMech 95; tre run per GPU |
| RunPod 2×RTX 3090 (cu126) | P4 passo 7, P9 passo 8 (13 run) | Release A 1 h 42; `pm_control` 1 h 45; `joint` 1 h 57; due run per GPU |
| vast.ai RTX 5090, Seoul (cu130) | P9 passo 9 (29 run) | una run `joint` 57 minuti (due alla volta); 13 ore totali |

**Lezioni operative dai pod.** Verificare CUDA prima di tutto (due pod RTX 3080 Ti avevano `cuInit` 999 a livello di host; un driver 550 richiedeva di rimuovere la voce di compatibilità cuda-13 e torch cu126); su un host CUDA impostare `UV_NO_SYNC=1`, altrimenti `uv run` reinstalla il gruppo ROCm; controllare la quota di CPU del cgroup; il proxy SSH di RunPod accetta solo sessioni

PTY, serve SSH su TCP esposto per trasferire dati; i dati viaggiano dal dataset Hugging Face privato con un token di sola lettura. Sulla decodifica: il beam è sensibile al numero di thread di torch in caso di quasi-pareggi (un campione trovava 4 completazioni invece di 5 con 1–3 thread): tutte le decodifiche usano `OMP_NUM_THREADS=4`, e un test di riproduzione lo verifica.

# 12 Decodifica e valutazione (`ksmech.eval`)

## 12.1 Il beam decoder

`model.decode.decode_pool` (spostato dall'app web nel motore il 27 ottobre):

1. codifica il pool una volta (CPU, senza gradiente) e calcola $\log P$(ancora);
2. prende le $B = 5$ ancore più probabili;
3. per ogni ancora calcola le tabelle delle voci e il problema di completamento: $\log P(\boldsymbol{R}) = \log P(e) + \sum_x \log P(R_x \mid e)$, cioè lo sfondo tutto-zero più, per ogni voce cambiata, il guadagno $\log P(v) - \log P(0)$; tiene i 48 guadagni migliori (`max_candidates`); un'ancora di rottura vieta le voci +1;
4. **branch and bound** best-first (includi/escludi ogni candidato in ordine di guadagno; il limite somma i migliori guadagni positivi ammessi dalla capacità residua): le completazioni escono in ordine esatto di $\log P$, con al più 8 cambi di legame (ancora inclusa) e 2 cambi atomici (caps `legacy`), 20 000 nodi e 64 applicazioni per ancora;
5. scarta, contando ogni motivo (`DECODE_DROPPED:<code>`): troppi cambi, ancora non ammissibile, carica non conservata ($\sum R_{ii} + 2\sum_{i<j} R_{ij} \neq 0$), lo schermo a guscio chiuso (`ODD_OFF_BROKEN_BOND`, e `CLOSED_MINUS1` solo con il filtro);
6. applica $\boldsymbol{B} + \boldsymbol{R}$ sulla struttura di Lewis del campione (`lewis_apply`): ordini in $[0, 3]$, doppietti $\geq 0$, nessun atomo toccato da H a Ne oltre l'ottetto (H e He: 2), nessun non metallo toccato che finisca con $|q| \geq 2$ senza averla prima (`FORMAL_CHARGE_RANGE`: le prime run producevano $[Cl^{2-}]$, $[O^{2-}]$, $[C^{2+}]$; in PMechDB solo 11 prodotti su 13 090 hanno un atomo con $|q| \geq 2$, tutti metalli o un perclorato), RDKit deve scrivere il prodotto, e la chiave mappata deve cambiare (altrimenti `RESONANCE_ONLY_STEP`);
7. per ogni ancora si ferma a 5 prodotti distinti; unisce tra ancore per chiave d'identità del prodotto tenendo il miglior $\log P$ (`DECODE_MERGED`).

Il punteggio è la probabilità del cammino attraverso la sua ancora, non la marginale su tutte le ancore ammissibili (che richiederebbe una chiamata per ancora di ogni completazione).

**Hit.** Un *hit* a $k$ quando una delle prime $k$ completazioni unite è esattamente (legami, doppietti) una variante della classe di etichette, che è chiusa per simmetria. Il report contiene top-1/3/5, `not_found`, `odd_top1`, il tempo medio, i conteggi dei salti, la frazione coperta, il dettaglio per forma di $\boldsymbol{R}$ e per campione (`decode_hits_*.json`, per i confronti appaiati). Tempo tipico: 0,6–1,2 s per campione su un thread di CPU.

**Caps.** `-caps legacy` (8 legami, 2 atomi, 48 candidati) o `labels` (i massimi delle etichette di training della sorgente: 6/2 su PMechDB, 8/3 su ReactMech, 12/5 su rmechdb). Su PMechDB le due scelte danno ranghi identici; su ReactMech i caps `labels` costano 2–5 hit per seed: i caps restano `legacy`, tranne su rmechdb (12-5-48).

**Scoperta – perché il top-1 full-R supera quello dell'ancora.** Per `flex_first` il top-1 full-R in validazione (63,0%) è più alto del top-1 dell'ancora registrato in training (60,9%): una completazione è confrontata con l'intera classe (ogni variante, ogni immagine simmetrica) e il beam completa cinque ancore, quindi un hit può venire da un'ancora diversa dalla prima. Le scarti dominanti sono la conservazione della carica (1,03 milioni) e l'ottetto (312 000).

## 12.2 Riferimenti

**Floor fisico (`ksmech-eval floor`)**

ogni coppia candidata è proposta una volta, «form» se il suo ordine può salire, «break»

altrimenti, e ordinata per $|G^{\text{atom}}|$ sulle celle scelte (`-cells valid|all`); nessun parametro appreso. Variante `own_break` (P7): ancore di formazione per $|G^{\text{atom}}|$, ancore di rottura su ogni coppia legata per $|B^{\text{own}}|$, così un'omolisi è raggiungibile; `-kernel g2` usa $g_2$.

**Ranker LightGBM (`ksmech-physics gbdt`)**
lambdarank sulle coppie di orbitali della prima freccia (sezione 13.1); non esiste su ReactMech, che non ha frecce.

**Casi da manuale (`ksmech-physics textbook`)**
cinque casi di selettività ($MeO^-$ + MeCl, $MeO^-$ + EtBr $S_N2$ contro E2, $CN^-$ + acroleina 1,4 contro 1,2, $MeNH_2$ + AcCl, $H_3O^+$ + acetone); i casi visti in training sono marcati «non un test» e un secondo modello è addestrato con quei pool esclusi. `-radical`: una suite radicalica ($Me_3Sn^\bullet$ + MeX, $Br^\bullet$ + propene anti-Markovnikov, polarità di $tBu^\bullet$ e $CF_3^\bullet$, astrazione $\alpha$ dal THF, bromurazione benzilica del toluene, $CH_3^\bullet$ + $CH_3^\bullet$).

## 12.3 Confronti appaiati, coorti, fette

`ksmech-eval compare` implementa la regola U20 (sezione 5): tolleranza $\max(2\sigma_{\text{pooled}}, 1)$ punti e McNemar esatto unilaterale sui hit a maggioranza dei seed e su ogni coppia di seed; rifiuta bracci con campioni, parti, coorti, beam, caps o versione del decoder diversi (`-expect-diff` verifica in `run.json` che i bracci differiscano solo nel campo atteso). `ksmech-eval pin` fissa una coorte con i digest per colonna; `cohort` definisce coorti per classe di dominio; ogni report riporta la frazione coperta e le fette `fermi_smeared` e `bond_stretched`.

## 12.4 Importanza e traccia degli embedding

`ksmech-eval importance` permuta ogni colonna di input dei token (e blocchi di colonne correlate: Fukui, frontiera, pesi sui centri, quote $s/p/d$, coppia di centri, energia) tra i token reali dello stesso tipo dentro ogni batch, 3 ripetizioni, e misura le variazioni di NLL, top-1/3 ed $R$ esatto; `spin_role` e le feature di coppia non sono permutati. `ksmech-eval embedding` produce una pagina HTML autonoma con un campione attraverso il modello: i token sulla struttura di Lewis, le parti di $\boldsymbol{h}^{(0)}$, attenzione e aggiornamenti di ogni strato (ricostruiti dagli input registrati e verificati contro il forward del modello), e i logit delle ancore divisi in termini di sito, di coppia e di fisica.

# 13 Risultati sperimentali

Questo capitolo raccoglie i risultati di tutti i training e delle valutazioni eseguite tra il 26 ottobre e il 9 novembre 2026. Salvo dove indicato, i numeri sono stati ricalcolati per questo documento direttamente dai file `decode_report_*.json`, `metrics.jsonl`, `anchor_floor_report_*.json` e `gbdt_lgbm_report_*.json` sotto `data/models/` e `data/processed/`.

**Come leggere i numeri.** Le metriche principali sono due e non vanno confuse:

- **top-$k$ dell'ancora** (`val_top1` in `metrics.jsonl`): la frazione di campioni per cui un'ancora ammissibile dell'etichetta è tra le $k$ più probabili; è la metrica con cui si sceglie `best.pt` durante il training;
- **top-$k$ della matrice completa** (*full-R*, nei report di decodifica): la frazione di campioni per cui una delle prime $k$ completazioni della beam search (larghezza 5) è una variante della classe di etichette, verificata applicando $\boldsymbol{B} + \boldsymbol{R}$ sulla struttura di Lewis del campione. È la metrica di riferimento per tutti i confronti («gate») tra configurazioni.

Il riferimento a guscio chiuso è la coorte `closed_base` (passi PMechDB dello split per molecola dopo le modifiche di generalità di P0a, fissata una volta il 29 ottobre): 1 238 campioni di validazione e 1 182 di test, copertura 99,4% (7 campioni senza token, `MISSING_TOKENS`). Il test è stato letto una sola volta per la configurazione scelta.

## 13.1 Riferimenti non addestrati e ranker GBDT

### 13.1.1 Il floor fisico sulle ancore

Il floor ordina le coppie di atomi candidate ad ancora per $|G^{\mathrm{atom}}_{ij}|$, senza alcun parametro appreso (`ksmech-eval floor`). È l'analogo, sulle ancore, dell'ordinamento delle coppie di orbitali per $G$. Tabella 13.1.

**Tabella 13.1:** Floor fisico: top-$k$ dell'ancora (%) ordinando per $|G^{\mathrm{atom}}|$. «valid»: pooling sulle coppie valide (configurazione legacy); «all»: su tutte le celle. «Irr.»: campioni la cui ancora non è raggiungibile dal floor (rotture senza formazione), resi raggiungibili dalla variante `own_break` (P7).

| Sorgente / split | Celle | parte | top-1 | top-3 | top-5 | rango med./irr. |
|---|---|---|---|---|---|---|
| PMechDB, `closed_base` | valid | val | 15,6 | 26,0 | 31,1 | 25 / 39 |
| | valid | test | 17,8 | 27,3 | 31,6 | 26 / 77 |
| | all | val | 14,4 | 23,7 | 28,8 | 19 / 39 |
| | all | test | 13,3 | 22,9 | 27,0 | 28,5 / 77 |
| | valid + `own_break` ($g$) | val | 15,6 | 26,0 | 31,1 | 25 / 0 |
| | valid + `own_break` ($g_2$) | val | 15,4 | 25,6 | 30,8 | 26 / 0 |
| ReactMech (pilota), molecule | valid | val | 5,2 | 11,5 | 14,3 | 214 / 4 |
| | valid | test | 7,1 | 11,8 | 14,8 | 250 / 8 |
| | all | val | 3,1 | 7,1 | 8,3 | 163 / 4 |
| RMechDB, `rc_molecule` | valid | val | 12,6 | 17,1 | 27,9 | 19 / 15 |
| | valid | test | 9,7 | 22,1 | 28,3 | 14 / 17 |
| | all + `own_break` ($g$) | val | 11,7 | 19,8 | 29,7 | 13 / 0 |
| | all + `own_break` ($g_2$) | val | 13,5 | 20,7 | 30,6 | 15 / 0 |

Il floor fisico porta l'ancora al primo posto in circa un caso su sei su PMechDB, contro un rango casuale dell'ordine di centinaia di candidati: la sola fisica di Klopman–Salem contiene un'informazione reale ma molto lontana da quella di un modello addestrato (circa 65% full-R).

Su ReactMech il floor è quasi cieco (5%): i passi di ReactMech sono dominati da trasferimenti di protone e da passi in cui il legame che si forma non è quello con il gap più favorevole. Su RMechDB la variante $g_2$ (gap a due livelli) migliora leggermente il floor sui passi radicalici (13,5 contro 11,7), come atteso dove i denominatori canonici di una SOMO sono piccoli.

### 13.1.2 Ranker della prima freccia

Prima del modello neurale, il riferimento veloce è un ranker LightGBM *lambdarank* sulle feature di coppia geometry-free, che ordina le coppie di *orbitali* candidate per la prima freccia di un passo (tutti i positivi, 20 negativi difficili scelti per $G$, 30 negativi casuali per passo; `learning_rate` 0,05, `num_leaves` 31, `min_data_in_leaf` 50, `feature_fraction` e `bagging_fraction` 0,8, `truncation_level` 10, fino a 2 000 alberi con pazienza 100 sul top-5 di validazione). Questo insieme di candidati è diverso da quello delle ancore, quindi i numeri non sono confrontabili con l'accuratezza delle ancore.

**Tabella 13.2:** Ranking della prima freccia (coppie di orbitali), PMechDB, top-$k$ in %.

| | | validazione | | | test | | |
|---|---|---|---|---|---|---|---|
| Split | Ordinatore | top-1 | top-3 | top-5 | top-1 | top-3 | top-5 |
| molecule | kernel di gap $G$ non addestrato | 23,0 | 29,8 | 35,9 | 20,5 | 27,9 | 34,6 |
| molecule | LightGBM (celle valide) | 43,1 | 60,8 | 69,2 | 47,8 | 66,6 | 74,1 |
| molecule | LightGBM, casi da manuale esclusi | 43,2 | 61,1 | 69,9 | 49,2 | 66,4 | 72,6 |
| molecule | LightGBM (tutte le celle) | 41,9 | 60,5 | 70,0 | 47,6 | 65,6 | 74,4 |
| scaffold | kernel di gap $G$ non addestrato | 6,5 | 10,1 | 12,6 | 10,9 | 18,3 | 20,7 |
| scaffold | LightGBM (celle valide) | 40,2 | 58,0 | 68,0 | 45,5 | 66,1 | 76,0 |
| *Storico, feature di contatto (stage B, archiviato):* | | | | | | | |
| molecule | testa congelata (contatto) | 3,6 | 7,8 | 9,5 | 2,5 | 6,1 | 8,0 |
| molecule | $KS_3$ | 14,0 | 18,0 | 22,1 | 12,2 | 16,3 | 19,7 |
| molecule | $KS_3 + KS_2$ | 15,1 | 22,5 | 27,7 | 13,9 | 20,6 | 24,2 |
| molecule | LightGBM con contatto | 43,6 | 57,9 | 66,0 | 46,3 | 60,7 | 68,8 |

Il ranker con le feature di contatto non fa meglio di quello geometry-free (test top-1 46,3 contro 47,8; top-5 68,8 contro 74,1) pur costando un modello d'avvicinamento intero: è la seconda evidenza, dopo quella dei termini non addestrati, che ha portato alla rimozione dello stage B. Il GBDT geometry-free su PMechDB richiede circa 100 s di caricamento e 30 s di fit a 14 thread (picco di memoria 3,6 GB); sullo split scaffold il fit sale a 156 s e la memoria a 8,3 GB.

## 13.2 I primi training (26–28 ottobre)

Le prime run sono state eseguite sullo split per molecola di PMechDB prima delle modifiche di generalità di P0a: `flex_first` su un pod RunPod RTX 3090 Ti (schema 15, 27 minuti per 30 epoche), le altre quattro su una RTX 5080 (schema 16, codice `682b7f2`); 30 epoche, seed 0, $d = 256$, 8 strati, lr $3 \cdot 10^{-4}$, bf16, budget 1,1 M celle. La tabella 13.3 riporta la miglior epoca per top-1 dell'ancora in validazione (1 239 campioni) e, dove esiste, la decodifica full-R successiva (rivalutata su `closed_base` dopo P0a). Riferimenti sullo stesso split: floor 15,6%, LightGBM 44,1% (prima freccia).

**Tabella 13.3:** Prime run su PMechDB (split molecule, 30 epoche, seed 0). Ancora: miglior epoca. Full-R: decodifica di `best.pt`. s/ep: secondi medi per epoca.

| Run | Encoder | Ris. | top-1 anc. | top-3 | top-5 | $R$ esatto | full-R val / test | s/ep |
|---|---|---|---|---|---|---|---|---|
| `flex_first` | flex | no | 60,9 | 79,9 | 87,3 | 55,6 | 62,9 / 62,7 | 49 |
| `flex_resonance` | flex | sì | 63,4 | 81,0 | 87,0 | 56,7 | – | 51 |
| `dense_plain` | dense | no | 65,6 | 84,3 | 91,1 | 61,7 | – | 90 |
| `dense_resonance` | dense | sì | 62,3 | 85,5 | 91,6 | 57,1 | – | 90 |
| `sparse_resonance` | sparse | sì | 62,1 | 81,0 | 88,1 | 55,6 | – | 80 |

Ogni run raggiunge il massimo del top-1 tra le epoche 20 e 22 e il minimo della loss di validazione tra le 11 e le 15; l'ultima epoca è 2–6 punti sotto la migliore. L'encoder denso con la traccia delle coppie completa (`dense_plain`) è risultato il migliore e il più stabile; l'augmentation di risonanza non ha dato un guadagno misurabile sull'ancora a seed singolo (62,3 contro 65,6 per il denso, 63,4 contro 60,9 per flex): è stata quindi lasciata disattivata nelle run di confronto successive (`-no-resonance-draws`). FlexAttention (`flex`) è circa due volte più veloce del denso ma perde circa 5 punti.

## 13.3 Il pilota ReactMech e la valutazione incrociata

ReactMech fornisce passi senza frecce, con l'etichetta ottenuta come differenza dei due lati mappati (capitolo 9). Il pilota contiene 10 000 passi (548 passi di risonanza esclusi, 9 450 passi di training), split per molecola (val 911, test 902 campioni).

**Tabella 13.4:** Run su ReactMech e sulle due sorgenti insieme (seed 0, full-R top-$k$ in %).

| Run | Valutato su | parte | top-1 | top-3 | top-5 |
|---|---|---|---|---|---|
| `reactmech/sparse_resonance` | ReactMech | val | run non finita (NaN), vedi sotto | | |
| `reactmech/sparse_resonance_stop0` | ReactMech | val | 91,1 | 93,0 | 93,2 |
| | ReactMech | test | 72,1 | 74,7 | 75,5 |
| | ritorni P5 (scaffold) | val/test | 0,0 / 0,0 | 9,1 / 10,0 | 9,1 / 10,0 |
| `pmechdb+reactmech/sparse_resonance` | ReactMech | val | 91,9 | 94,7 | 95,6 |
| | ReactMech | test | 70,4 | 73,9 | 74,9 |
| | PMechDB `closed_base` | val | 61,6 | 79,3 | 81,6 |
| | PMechDB `closed_base` | test | 64,1 | 82,4 | 84,2 |
| | ritorni P5 (scaffold) | val/test | 18,2 / 16,7 | 18,2 / 20,0 | 18,2 / 20,0 |

**Scoperta – una run non finita produceva un falso 100%.** La prima run ReactMech con la testa di stop (`sparse_resonance`) ha prodotto valori non finiti: la sua `metrics.jsonl` riporta top-1 dell'ancora pari a 1,000 con NLL `nan`, perché un logit NaN vinceva il confronto. Dal 28 ottobre una run non finita si ferma e un NaN non produce mai un «hit» né un `best.pt`; la run ripetuta con $\lambda_{\text{stop}} = 0$ (`sparse_resonance_stop0`) è quella valida.

**Scoperta – il test ReactMech per molecola è sbilanciato.** Lo split per molecola di ReactMech mette nel test quasi da sole le famiglie Swern (solfonio) e cromo: il top-1 crolla da circa 91% in validazione a circa 72% in test. I numeri di test ReactMech vanno quindi riportati divisi per famiglia.

## 13.4 P0b: GENERAL contro LEGACY

La fase P0a ha reso «generale» il percorso del modello: tutte le celle di coppia lette con le esclusioni come flag (`pair_cells='all'`), la frazione spettrale conservata e l'energia $E_r$ come input (`orbital_extra`), caps del decoder dalle etichette. P0b ha deciso se la configurazione GENERAL costa accuratezza rispetto a LEGACY (gli input della v17), con tre seed per braccio, encoder denso, 30 epoche, senza augmentation di risonanza, su un pod RunPod RTX 5090.

**Tabella 13.5:** P0b, full-R top-$k$ (%), media su tre seed (valori per seed tra parentesi).

| Braccio | Valutazione | top-1 | top-3 | top-5 |
|---|---|---|---|---|
| GENERAL, PMechDB | `closed_base` val | 65,62 (67,7; 64,1; 65,1) | 83,58 | 86,65 |
| LEGACY, PMechDB | `closed_base` val | 64,49 (65,2; 65,9; 62,4) | 82,82 | 85,25 |
| GENERAL, PMechDB | `closed_base` **test** | 66,27 (67,6; 65,2; 66,0) | 83,95 | 86,46 |
| GENERAL, ReactMech | ReactMech val | 94,00 (94,7; 92,9; 94,4) | 95,61 | 95,76 |
| LEGACY, ReactMech | ReactMech val | 93,49 (93,2; 93,6; 93,6) | 94,77 | 95,02 |
| GENERAL, PMechDB → ReactMech | ReactMech val | 52,29 | 70,65 | 74,06 |
| LEGACY, PMechDB → ReactMech | ReactMech val | 50,64 | 70,40 | 73,73 |
| GENERAL, ReactMech → PMechDB | `closed_base` val | 27,65 | 42,70 | 45,69 |
| LEGACY, ReactMech → PMechDB | `closed_base` val | 29,05 | 43,65 | 46,10 |

**Decisione – 30 ottobre 2026.** GENERAL passa contro LEGACY su `closed_base` val (65,6 contro 64,5, tolleranza 3,75 punti, McNemar $p = 0{,}70$) e su ogni altro braccio (ReactMech 94,0 contro 93,5, bracci incrociati entro tolleranza). L'unico McNemar significativo (ReactMech→PMechDB top-3, $p = 0{,}021$) è dovuto a un seed GENERAL debole (s1). I caps del decoder restano legacy (8 legami, 2 atomi); i default di `ModelConfig` restano legacy e le fasi successive addestrano con `-preset general`. Il test `closed_base` di GENERAL, letto una volta, è 66,3.

**Scoperta – le due sorgenti si trasferiscono male.** Un modello addestrato su PMechDB ottiene circa 52% su ReactMech, uno addestrato su ReactMech circa 28% su PMechDB, contro 94% e 66% in distribuzione (McNemar: $p \approx 4 \cdot 10^{-103}$ e $7 \cdot 10^{-95}$). ReactMech (generato da template su 67 classi) è molto più facile ma molto più stretto di PMechDB (curato a mano); il secondo conserva l'80% della sua accuratezza sul primo, il contrario no. Il 94% di ReactMech dice che lo split per molecola tiene fuori molecole *dentro template noti*: la misura fuori distribuzione di ReactMech sono gli split `loco`/`loco_all`.

**Tabella 13.6:** P0b GENERAL, full-R top-1 (%) per distanza dal training. «Condiviso»: una molecola con più di 3 atomi pesanti in comune con il training; «elemento nuovo»: un elemento che nessun passo di training contiene. Tra parentesi il numero di campioni.

| Modello → validazione | nuovo | condiviso | elemento nuovo |
|---|---|---|---|
| PMechDB → PMechDB | 66,2 (1 213) | – | 37,3 (25; As, Ca) |
| PMechDB → ReactMech | 53,0 (849) | 54,1 (45) | 5,6 (30; Cr, Ni, Pd) |
| ReactMech → ReactMech | 95,3 (895) | – | 20,8 (16; K) |
| ReactMech → PMechDB | 27,6 (1 159) | 38,1 (14) | 26,2 (70; As, Ca, Cu, K, Sn, Ti, Zn) |

Un elemento mai visto è quasi fatale quando partecipa alla modifica (PMechDB→ReactMech: 0% su 25 campioni) e molto meno quando è spettatore (33% su 5).

## 13.5 I ritorni del pilota ReactMech (P5)

P0a e P1 hanno riportato nel dominio 225 campioni ReactMech (191 riparazioni dative di Pd/Ni, 34 CDI). I checkpoint ReactMech precedenti (addestrati sulla v17) sono stati rivalutati per fetta

di validazione (full-R top-1/3/5, tabella 13.7).

**Tabella 13.7:** Ritorni P5: checkpoint `stop0` (solo ReactMech) e `joint` (PMechDB+ReactMech).

| Fetta | $n$ | `stop0` | `joint` |
|---|---|---|---|
| intera validazione | 911 | 91,1 / 93,0 / 93,2 | 91,9 / 94,7 / 95,6 |
| (a) invariati da P0 | 814 | 94,5 / 96,6 / 96,6 | 94,7 / 97,3 / 97,3 |
| (b) Kekulé più vicino | 68 | 89,7 / 89,7 / 89,7 | 89,7 / 94,1 / 94,1 |
| (c) ricarica dativa | 29 | 0,0 / 0,0 / 6,9 | 17,2 / 24,1 / 51,7 |
| (d) ritorni (viste scaffold) | 225 | 0,0 / 4,9 / 4,9 | 16,0 / 16,9 / 17,3 |

I modelli v17 non hanno mai visto i disegni di Pd ricaricati e falliscono quasi sempre su di essi: i ritorni servono come dati di training, non come test di generalizzazione.

## 13.6 Stabilità del riferimento GBDT

Il ranker della prima freccia è stato riaddestrato a ogni ricostruzione dello store. Il test top-1 (celle valide / tutte) è oscillato tra 45,9 e 50,4: v17 48,8; P0a 48,1 / 50,4; P1 47,1 / 47,8; P3 47,1 / 45,9; P4 48,8 / 49,1; v19 (P8) 47,8 / 47,6. Il modello salvato di una fase, valutato sui dati della fase successiva, riproduce esattamente i propri numeri: le oscillazioni vengono dall'early stopping che si sposta con lo 0,1% di query in più e dal numero di thread, non dai dati. Ne segue che le differenze di 1–2 punti sul GBDT non sono significative.

## 13.7 Release A: P4 passo 7 e ablazione dei metalli

La Release A (fasi P1–P5) ha aggiunto a PMechDB i passi con specie a guscio aperto ammesse dalle leggi di dominio, i passi con carbeni di singoletto e quelli con metalli Cu(II) e Fe(III) (coorte `open_shell_metal`). Il passo 7 di P4 la confronta con P0b e decide con un'ablazione se i passi con metalli restano in training (`-drop-domain-class OSM_CU2 OSM_FE3`). Pod con 2×RTX 3090 (torch cu126), 8 novembre.

**Tabella 13.8:** Release A (3 seed, full-R top-1 %, `closed_base` val).

| Braccio | top-1 | top-3 | top-5 | seed |
|---|---|---|---|---|
| P0b GENERAL (riferimento) | 65,62 | 83,58 | 86,65 | 67,7; 64,1; 65,1 |
| Release A GENERAL | 64,49 | 82,44 | 85,78 | 64,0; 64,7; 64,8 |
| Release A senza metalli | 64,38 | 82,12 | 85,11 | 64,8; 62,9; 65,4 |

Release A contro P0b: −1,13 punti, tolleranza 2,72, passa (McNemar 75/74 discordanti, $p = 0{,}50$). Ablazione dei metalli: con 64,49, senza 64,38: i passi con Cu e Fe restano (solo Cu è misurabile nello split per molecola, dove tutti i 17 passi Fe sono `SPLIT_CONFLICT`). Sui 15 passi di training della coorte `open_shell_metal`, i modelli che li hanno visti li riproducono al 100%, quelli addestrati senza al 60% (top-1), 60–93% (top-5).

## 13.8 P9 passo 8: la porta U20 e l'addestramento congiunto

Il passo 8 di P9 addestra il preset `open_shell` (GENERAL + testa atomica a cinque classi $\{-2, -1, 0, +1, +2\}$ + input P7) in due collegamenti:
1. `pm_control`: solo PMechDB v19, preset `open_shell`, contro Release A (stessa GPU);

2. `joint`: PMechDB + RMechDB (`rc_molecule`, RC per molecola, RS in training), contro `pm_control`; `LEAK_GUARD` toglie 34 campioni RMechDB che condividono specie con la validazione PMechDB.

Selezione del checkpoint sulla coorte `closed_base` (`-val-cohort pmechdb=closed_base`), decodifica v4.

**Tabella 13.9:** P9 passo 8 (2×RTX 3090, cu126). Full-R top-$k$ in %, media di tre seed. †Riferimenti a un seed addestrati nel passo 9 su RTX 5090.

| Run | Valutazione | top-1 | top-3 | top-5 | seed |
|---|---|---|---|---|---|
| `pm_control` | `closed_base` val | 65,24 | 84,54 | 87,24 | 66,0; 66,6; 63,1 |
| `joint` | `closed_base` val | 63,14 | 83,58 | 86,35 | 64,3; 62,5; 62,6 |
| `pm_v19` (1 seed) | `closed_base` val | 62,44 | 81,42 | 85,95 | 62,4 |
| `joint` | RMechDB RC val ($n = 111$) | 69,67 | 80,18 | 80,48 | 70,3; 68,5; 70,3 |
| `rad_only` (solo RMechDB)† | RMechDB RC val | 63,1 | 79,3 | 81,1 | s0 |
| `rad_given` (split FlowER)† | `flower_given` val ($n = 101$) | 71,3 | 84,2 | 84,2 | s0 |

- Collegamento 1: `pm_control` 65,24 contro Release A 64,49, +0,75 (tolleranza 2,75), passa; McNemar $p = 0{,}60$.
- Collegamento 2: `joint` 63,14 contro `pm_control` 65,24, −2,10 (tolleranza 3,03): passa formalmente, ma McNemar $p = 0{,}085$ e 5 delle 9 coppie di seed hanno $p < 0{,}05$: un probabile costo reale di circa 2 punti sul dominio a guscio chiuso dell'addestramento congiunto.
- `joint` contro P0b: −2,48 (non soggetto a porta).

**Passi radicalici per forma di *R*.** La decodifica su RC val divide i 111 campioni per forma della matrice di reazione (tabella 13.10). Il modello congiunto risolve bene ricombinazioni (92%), astrazioni $S_H2$ (80–85%) e omolisi (73–80%), meno le addizioni (64–86%) e quasi mai le forme «altre» (11–21%). Il floor fisico sullo stesso insieme è 12,6%.

**Tabella 13.10:** `joint`, RMechDB RC val, full-R top-1 per forma di ***R*** (caps 12 legami, 5 atomi, 48 candidati).

| Forma | $n$ | s0 | s1 | s2 |
|---|---|---|---|---|
| omolisi | 15 | 0,73 | 0,73 | 0,80 |
| ricombinazione | 13 | 0,92 | 0,92 | 0,92 |
| addizione | 14 | 0,86 | 0,79 | 0,64 |
| astrazione ($S_H2$) | 46 | 0,80 | 0,80 | 0,85 |
| $\beta$-scissione | 4 | 0,75 | 0,50 | 0,50 |
| altre | 19 | 0,16 | 0,16 | 0,21 |

L'addestramento sul solo RMechDB (`rad_only`, 63,1) è inferiore al congiunto (69,7) sullo stesso RC val: i passi polari di PMechDB aiutano i passi radicalici, mentre (collegamento 2) i passi radicalici costano circa 2 punti a quelli polari.

## 13.9 P9 passo 9: le ablazioni

Il passo 9 ripete `joint` togliendo un elemento alla volta (9 bracci × 3 seed) e aggiunge due bracci di sola decodifica. Le run sono state eseguite il 9 novembre su un pod vast.ai a Seoul con una RTX 5090 (torch cu130), mentre il riferimento `joint` era stato addestrato il giorno prima su una RTX 3090 con cu126.

**Tabella 13.11:** P9 passo 9, full-R top-1 (%), media di tre seed, e differenza da `joint`. `closed_base`: 1 238 campioni; RC val: 111. Tutti i bracci su RTX 5090, `joint` su RTX 3090.

| Braccio | Cosa toglie | `closed_base` | Δ | RC val | Δ |
|---|---|---|---|---|---|
| `joint` (rif.) | – | 63,14 | – | 69,67 | – |
| `g_only` | $g_2$, $\log\lvert g_2\rvert$ | 65,08 | +1,94 | 68,47 | −1,20 |
| `g2_only` | $G$, $\log\lvert G\rvert$ | 64,97 | +1,83 | 66,07 | −3,60 |
| `own_off` | canale own-pair, $B^{\text{own}}$ | 66,83 | +3,69 | 66,07 | −3,60 |
| `somo_off` | canali SOMO/ricombinazione | 63,54 | +0,40 | 66,97 | −2,70 |
| `spin_cat` | $\rho_A \to u$ categorico | 64,14 | +1,00 | 66,07 | −3,60 |
| `spin_none` | nessun input di spin | 65,27 | +2,13 | 66,07 | −3,60 |
| `frontier_exact` | finestra $10^{-4}$ eV | 64,51 | +1,37 | 66,67 | −3,00 |
| `fukui_fd` | Fukui alle differenze finite | 65,35 | +2,21 | 66,37 | −3,30 |
| `no_rs` | righe RS di FlowER | 64,43 | +1,29 | 66,37 | −3,30 |

**Scoperta – confondimento hardware.** Ogni braccio sta *sopra* `joint` su `closed_base` (+0,4 … +3,7) e *sotto* su RC val (−1,2 … −3,6), qualunque cosa tolga. Un effetto uniforme di questo tipo non è attribuibile ai singoli componenti: è il segno che il riferimento è stato addestrato su hardware e software diversi (RTX 3090 + cu126 contro RTX 5090 + cu130; 225 s contro 108 s per epoca). I nove bracci addestrati **non sono decisi**: serve rieseguire `joint`×3 sulla stessa 5090 (`ONLY=joint_r`, circa 2 ore). Lezione: il riferimento di un'ablazione deve essere addestrato sulla stessa GPU e sullo stesso torch dei suoi bracci.

**Bracci di sola decodifica.** Sui checkpoint di `joint`: (viii) il filtro opzionale `-closed-minus1-filter` (rifiuta ogni voce atomica −1 su un pool senza elettroni spaiati) e (x) `max_candidates` 96 invece di 48. Su `closed_base` nessun cambiamento di top-1/3/5 per alcun seed; su RC val un solo campione cambia per il seed 0 (70,3 → 69,4). Decisione: il filtro resta spento e `max_candidates` resta 48.

## 13.10 Curve di apprendimento

La figura 13.1 mostra l'evoluzione del top-1 dell'ancora e della NLL dell'ancora in validazione per due run tipiche.

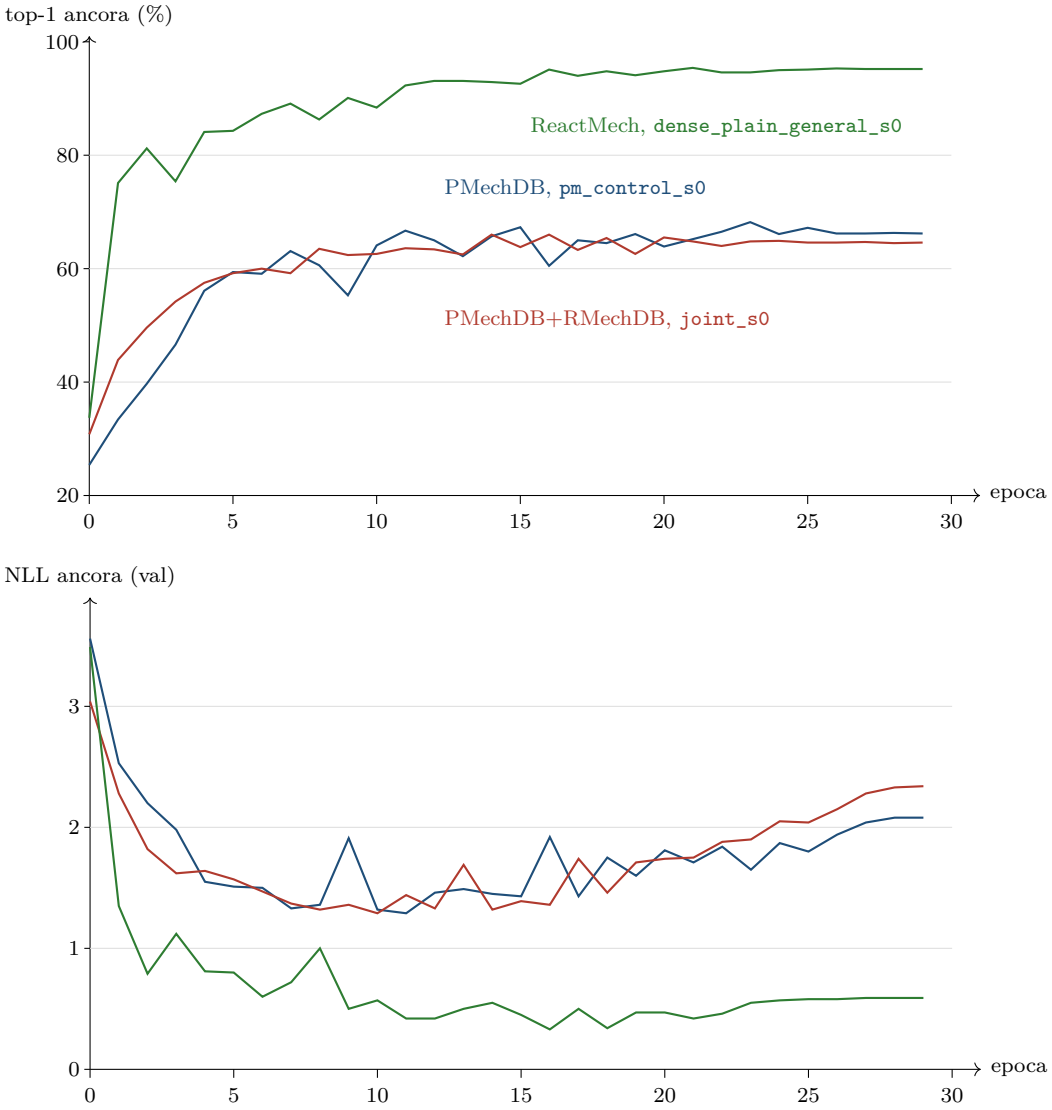



**Figura 13.1:** Validazione per epoca. Sopra: top-1 dell'ancora. Sotto: NLL dell'ancora. Blu: `pm_control_s0`; rosso: `joint_s0` (validazione congiunta PMechDB `closed_base` + RC); verde: ReactMech `dense_plain_general_s0`.

**Scoperta – sovra-confidenza dopo l'epoca 10–14.** La loss di training scende fino a circa 0,05 (PMechDB) mentre la NLL dell'ancora in validazione tocca il minimo tra le epoche 8 e 14 (circa 1,3) e poi risale fino a 2,1–2,3 alla trentesima epoca, mentre il top-1 resta su un plateau attorno al 65%. Il modello continua a ordinare bene ma diventa sovra-confidente sugli errori. La selezione di `best.pt` sul top-1 dell'ancora (non sulla NLL) protegge l'accuratezza, ma probabilità calibrate (necessarie per la ricerca su reti di reazione) richiederanno early stopping sulla NLL, regolarizzazione o calibrazione a posteriori.

## 13.11 Importanza per permutazione degli input

`ksmech-eval importance` (9 novembre) permuta ciascuna colonna di input dei token (e blocchi di colonne correlate) tra i token dello stesso tipo della parte valutata, e misura l'aumento della NLL dell'ancora e la caduta di top-1/3 rispetto al baseline; le feature di coppia non sono toccate. Run `pm_v19_s0`, PMechDB molecule val, 1 241 campioni, baseline top-1 dell'ancora 65,8%, NLL 1,24, 3 ripetizioni.

**Tabella 13.12:** Importanza per permutazione (`pm_v19_s0`): caduta di top-1 dell'ancora in punti percentuali e aumento della NLL dell'ancora, ordinati per effetto.

| Gruppo | Tipo | Δ top-1 (pt) | Δ NLL |
|---|---|---|---|
| elemento ($Z$) | atomo, categorico | −43,6 | +4,57 |
| carica formale | atomo, categorico | −30,2 | +1,98 |
| blocco Fukui ($f^+$, $f^-$, duale) | atomo | −18,3 | +0,79 |
| carica di Mulliken | atomo | −13,9 | +0,71 |
| $V^{\mathrm{bur}}$ | atomo | −5,3 | +0,37 |
| $f^-$ (orbitali congelati) | atomo | −5,3 | +0,21 |
| $f^+$ (orbitali congelati) | atomo | −5,1 | +0,16 |
| blocco pesi di frontiera (HOMO−1…LUMO+1) | atomo | −4,3 | +0,19 |
| descrittore duale | atomo | −3,9 | +0,12 |
| peso LUMO | atomo | −2,6 | +0,10 |
| peso LUMO+1 | atomo | −1,2 | +0,06 |
| peso HOMO−1 | atomo | −0,9 | +0,04 |
| energia $E_r$ e frazione conservata | orbitale | 0,0 | +0,002 |
| insieme spettrale | orbitale | −0,1 | −0,03 |
| ogni altro descrittore orbitale | orbitale | $\lvert\Delta\rvert < 0{,}6$ | $\lvert\Delta\rvert < 0{,}03$ |
| segno di chiralità | atomo | 0,0 | 0,0 |

**Scoperta – gli input orbitali hanno poca importanza marginale.** Elemento e carica formale dominano, seguiti dai descrittori di reattività per atomo (Fukui, Mulliken, $V^{\mathrm{bur}}$, pesi di frontiera), che insieme valgono decine di punti. I descrittori propri dei token orbitali (pesi sui centri, quote $s/p/d$, coseni, popolazione di sovrapposizione, insieme spettrale) valgono ciascuno meno di mezzo punto. Due cautele: l'importanza per permutazione sottostima le feature ridondanti (un token orbitale eredita gli embedding dei suoi atomi attraverso i pesi sui centri, quindi gran parte della sua informazione è già presente), e la fisica orbitale entra soprattutto attraverso le feature di coppia ($G$, $g_2$), che questo esperimento non permuta. La misura dice quindi che, nella forma attuale, i token orbitali agiscono soprattutto come supporto delle feature di coppia, non come portatori autonomi di informazione.

# 14 Scoperte e lezioni

Questo capitolo raccoglie, in ordine tematico, le scoperte principali fatte durante lo sviluppo; i capitoli precedenti ne danno il contesto e i numeri.

## 14.1 Sulla fisica

1. **La geometria di contatto peggiora il ranking.** Ogni tentativo di valutare l'accoppiamento di Klopman–Salem a una geometria di contatto (tabella di van der Waals, MLP, kernel orientato analitico, lobi sul sito) ha classificato la prima freccia peggio del solo fattore di gap $G$ (test top-1: testa congelata ∼3%, $KS_3 + KS_2$ 14,3%, $G$ 20,4%). La tabella di contatto favorisce sistematicamente gli H e gli shell $s$; il lobo di un orbitale di legame punta al partner. L'avvicinamento non è una proprietà dei reagenti: è stato tolto dal modello (spec v9).
2. $G$ **va calcolato sugli spettri canonici completi.** Le energie diagonali degli LMO non rispettano neppure l'ordine $\sigma^*$(C–Cl) > $\sigma^*$(C–Br) > $\sigma^*$(C–I); i pesi spettrali sì. Troncare lo spettro a 8 voci cambia $G$ fino al 21,8%.
3. **La fisica da sola porta l'ancora al primo posto una volta su sei** (floor 15,6% su PMechDB), contro circa 65% del modello addestrato; il ranker GBDT sulle sole feature geometry-free arriva al 48% sulla prima freccia. La fisica è un segnale reale ma non sufficiente.
4. $g_2$ **conta sui radicali**: sul floor dei passi radicalici il gap a due livelli migliora il top-1 (13,5% contro 11,7%) e nella suite radicalica sposta i ranghi attesi, ma nelle ablazioni addestrate né $G$ né $g_2$ da soli sono peggiori dell'altro.
5. **Gli input orbitali hanno poca importanza marginale.** L'importanza per permutazione mostra che il modello si appoggia a elemento, carica formale e descrittori di reattività per atomo; ogni descrittore orbitale vale meno di mezzo punto. Ipotesi: ridondanza con le feature di coppia e con gli embedding ereditati dagli atomi; da verificare con un'ablazione (capitolo 16).

## 14.2 Sui dati e sulle etichette

6. **Le frecce non sono un bersaglio ben posto.** Un orbitale localizzato non distingue il doppietto dal legame ionico (Mitsunobu, O–Li); dopo molte regole il round trip si fermava al 99,67%. Prevedere $\boldsymbol{R}$ ha risolto il problema alla radice (copertura dal 92% al 99,6%).
7. **Il plateau del 60% era in parte simmetria**: nel 21,5% della validazione l'etichetta sceglie uno di più atomi equivalenti, e i pareggi in bf16 contavano come hit. La chiusura per simmetria ha portato `flex_first` da 55,9% a 63,9% senza riaddestrarlo.
8. **RDKit non è un oracolo di risonanza**: la chiave enumerata concorda tra le forme di un prodotto solo nel 59% dei casi; RDKit raggiunge il prodotto disegnato in 147 delle 471 coppie di risonanza; restituisce forme con H rigenerati e radicali spuri. La chiave sigma (senza enumerazione) è esatta.
9. **Il rumore delle etichette ReactMech valeva 6 punti**: ridisegnare il prodotto nella struttura di Kekulé più vicina al reagente ha portato la validazione ReactMech da 85% a 91% senza riaddestrare.
10. **Le regole di cancellazione toglievano proprio i segnali utili**: la regola «tipo $d$» rimuoveva più di metà del peso sull'ancora annotata nel 15,2% dei passi; renderle canali ha dato una configurazione (GENERAL) non peggiore.
11. **Gli esclusi non erano radicali**: nessuno dei 290 passi esclusi da PMechDB e dal pilota ReactMech era un passo radicalico; quasi tutti erano errori di disegno (metili senza H, catalizzatori senza leganti, fosfine di Pd(0) disegnate come fosfonio su un metallo neutro).
12. **Lo split per molecola deve guardare anche i prodotti**: con i soli reagenti il 32% dei collegamenti tra passi attraversava le parti; i reagenti comuni legano il 78% dei passi in un

gruppo, quindi validazione e test sono fuori distribuzione per costruzione.

13. **Il test ReactMech per molecola è sbilanciato** su famiglie mai viste (Swern, cromo): top-1 da ∼91% a ∼72%.
14. **Il punto di sella lineare dei carbeni**: ETKDG disegna $CCl_2$ lineare; da lì il tripletto sembra più basso; da una partenza spostata il singoletto vince di 1 eV. Da qui le leggi delle due partenze e di `BOND_FORMED`.

## 14.3 Sul training e sulla valutazione

15. **GENERAL non costa accuratezza**: leggere tutte le celle con i flag e gli extra orbitali dà 65,6% contro 64,5% (entro il rumore): le esclusioni a mano non servono al modello.
16. **Le sorgenti si trasferiscono in modo asimmetrico**: PMechDB→ReactMech 52%, ReactMech→PMechDB 28%. ReactMech è facile ma stretto.
17. **L'addestramento congiunto con i radicali costa circa 2 punti sui passi polari** (`joint` 63,1% contro `pm_control` 65,2%, McNemar $p = 0{,}085$, 5 coppie di seed su 9 significative), mentre i passi polari aiutano i radicalici (69,7% congiunto contro 63,1% solo RMechDB).
18. **Sovra-confidenza**: la NLL di validazione tocca il minimo tra le epoche 8 e 14 e poi risale, mentre il top-1 resta su un plateau; il modello non ha dropout. La selezione per top-1 protegge l'accuratezza ma non la calibrazione.
19. **Il confondimento hardware**: nove ablazioni addestrate su RTX 5090/cu130 confrontate con un riferimento su RTX 3090/cu126 si spostano tutte nella stessa direzione; non sono interpretabili finché il riferimento non è rieseguito sulla stessa macchina.
20. **Il determinismo dipende dai thread**: gli alberi di LightGBM cambiano con il numero di thread (deterministico solo a numero di thread fisso); il beam decoder cambia un campione su un quasi-pareggio tra 3 e 4 thread di torch.
21. **NaN può sembrare perfezione**: una run fp16 diventata NaN segnava top-1 1,000 e aveva sovrascritto `best.pt`; da qui le regole di arresto sui valori non finiti.
22. **Due metriche diverse**: il top-1 full-R della decodifica supera il top-1 dell'ancora del training perché valuta l'intera classe e cinque ancore.

## 14.4 Sull'ingegneria

23. **Provenienza come vincolo, non come documentazione**: ogni artefatto porta i suoi input, i loader rifiutano gli obsoleti, training e valutazione rifiutano gli alberi sporchi; una modifica di un file tracciato durante una build ne invalida gli output.
24. **Instabilità hardware**: i riavvii sotto carico erano BIOS e RAM, non il software; un worker senza limite ai thread BLAS bastava però a saturare la macchina.
25. **La CPU e il QM**: l'ottimizzazione domina il tempo; OpenBLAS nei worker (1,5–2,7×) e l'ottimizzatore ANC/RFO (3,1×) sono pronti, il secondo non ancora di default perché richiederebbe il ricalcolo completo della cache.

# 15 Applicazione web ChemRho, client e server MCP

L'app web (`ksmech.serve`) è un componente aggiuntivo: il motore non la importa mai (lo verifica `tests/test_engine_boundary.py`) e le sue dipendenze sono un gruppo separato (FastAPI 0.142.2, uvicorn 0.54.0, SQLModel, pydantic 2.13.5, markdown-it-py). Il marchio dell'interfaccia è **ChemRho**; il codice resta `ksmech`.

## 15.1 Struttura a strati

113 moduli e 26 101 righe in 17 sotto-pacchetti, in un monolite modulare a un processo. Le fondamenta sono una catena, `core` → `compute` → `chem` → `science` → `predict` → `runtime`, dove ogni strato importa solo quelli precedenti; `depiction` poggia su `core`; i pacchetti funzionali (`pool`, `workbench`, `embedding`, `browse`, `curate`, `ml`, `accounts`, `ocr`, `content`) importano le fondamenta ma mai l'uno l'altro; `app` collega tutto. Un test analizza gli import con l'AST e verifica che il grafo consentito sia aciclico. Una route è sottile: controlla gli id, chiama una funzione di servizio e ne restituisce il valore.

**Tabella 15.1:** Sotto-pacchetti dell'app web (righe).

| Pacchetto | Righe | Contenuto |
|---|---|---|
| `core` | 4 187 | impostazioni, errori, DB di curation, chiavi, versioni e fingerprint, capability e profili, autenticazione, audit, schemi (1 693 righe, rispecchiati in TypeScript) |
| `chem` | 3 353 | normalizzazione, featurizzazione (tre modi), pool, geometrie dei metalli (860), base STO-$n$G, cubi, Hückel esteso sintetico |
| `predict` | 3 353 | predittori, rete di reazione (849), espansione, job (730) |
| `depiction` | 2 244 | depiction 2D lato server con RDKit, template prospettici |
| `workbench` | 1 852 | molecola, insiemi orbitali, densità, proprietà, simmetria |
| `browse` | 1 460 | indice e ricerca di PMechDB e dei passi curati |
| `app` | 1 345 | cablaggio FastAPI, CLI |
| `science` | 1 305 | $G^{\mathrm{atom}}$, lettura delle frecce, analisi, applicazione dei passi |
| `accounts` | 1 251 | utenti, sessioni, password, OIDC, quote |
| `curate` | 1 223 | curation, ricontrollo, esportazione |
| `ocr` | 1 145 | motore OCR (MolScribe), installazione |
| altri | 3 378 | `compute` (worker), `runtime`, `pool`, `ml`, `embedding`, `content` (wiki) |

## 15.2 Calcolo, chimica e predittori

**Worker di calcolo.** Uno scheduler a processi (`ProcessScheduler`) con 1–2 worker *spawned*, coda a priorità (interattivo < job < background) con deduplicazione per chiave, deadline con SIGKILL (`WORKER_TIMEOUT`), interruttore dopo più di 5 timeout o crash in 600 s (`WORKER_UNSTABLE`, 503), riciclo del worker dopo 500 task o 2 048 MB. Una richiesta interattiva attende al più 15 s e poi restituisce 202 *Pending*.

**Modi della chimica.** `real`: cache condivise in sola lettura, poi `data/serve`; una mancanza esegue GFN2-xTB nel worker. `cached`: solo cache (`FEATURIZATION_DISABLED`). `synthetic`: Hückel esteso iterato sulla carica nella base di valenza GFN2 (veicolo di test, «mai un'affermazione

chimica»). I metalli di transizione usano geometrie iniziali con template di coordinazione (metodo `+metal-geom-v1`): su 33 complessi, da 19 corretti a 27 corretti.

**Predittori.** `floor` (ancore per $|G^{\text{atom}}|$, completate dai canali dominanti; punteggio in 1/eV, non una probabilità), `gbdt` (il ranker LightGBM sulle coppie di orbitali), `network` (un checkpoint locale di `ksmech-train` con il decoder del motore, su CPU; punteggi $\log P(\boldsymbol{R})$; la testa di stop non è letta), `user` (pseudo-predittore per le correzioni). L'espansione applica ogni candidato come $\boldsymbol{B}+\boldsymbol{R}$ e raggruppa i prodotti per chiave di pool; la rete di reazione (`ksmech-serve/network-v2`) ha nodi per id di pool, archi per (stato, $\boldsymbol{R}$), cammini ordinati per peso, motivi terminali (`NO_CANDIDATES`, `TURNOVER`, `MAX_STEPS`, `NODE_BUDGET`); parametri di default: beam 5, 8 passi, 200 nodi.

**Predict e Correct sulle matrici di reazione.** Il contratto (`docs/serve_v9_contract.md`) scambia $\boldsymbol{R}$ come `{bonds: [i,j,delta], lone: [i,delta]}`, l'ancora, la lettura in frecce (decomposizione in mosse di coppie come *perfect matching* che massimizza il $|G|$ diretto) e la lettura orbitale (coppia di token, $G$, coppia canonica dominante con le energie). Correct è un editor di $\boldsymbol{R}$ (strumenti legame, doppietto, freccia, gomma).

## 15.3 Funzioni dell'interfaccia

Frontend React 19 + TypeScript 6 + Vite 8, RDKit.js e 3Dmol (184 file, 21 488 righe; pacchetto di visualizzazione `@ksmech/viz` di 90 file e 12 723 righe, solo props, formato di scena JSON versionato). Quattro sezioni:

**ChemPtah (`#/workbench`)**
: sketcher 2D con scorciatoie in stile ChemDraw, lunghezze e angoli fissi, template, CIP; esportazione SVG/PNG/PDF/LaTeX (chemfig) con stile ACS; importazione da immagine via OCR; vista 3D con superfici orbitali (valutazione separabile della base GTO: da 2,8–3,9 s a 0,02 s per orbitale), diagramma dei livelli, proprietà, gruppo puntuale.

**ClotoMech (`#/predict`, `#/correct`, `#/curate`)**
: previsione di reti con viste Scheme e Pathways e viste del passo (Textbook, Animated, Orbitals); correzione; curation dei meccanismi (stati predicted, validated, corrected, rejected; esportazione come record PMechDB grezzi, mai dati di training).

**ChemThot (`#/browse`)**
: ricerca esatta (sotto-multiinsieme di chiavi di specie) e per sottostruttura su PMechDB e sui passi curati.

**Wiki (`#/wiki`)**
: pagine Markdown con front matter TOML renderizzate dal server (`index`, `reaction-matrix`, `orbital-tokens`, `klopman-salem`, `sn2-example`) e `llms.txt`.

Altre funzioni: traccia degli embedding di un pool attraverso un checkpoint (job «trace»); account con password locali (scrypt) o OIDC (ORCID, Google), sessioni in cookie HttpOnly, ruoli reader $<$ user $<$ curator $<$ admin, quote (user 100 al giorno e 10 al minuto); profili `lab` e `public` (il profilo pubblico disabilita browse, curate, export, embedding e ml: i dati di PMechDB non devono mai essere raggiungibili online, e un test percorre ogni route dell'app pubblica); log di audit di ogni richiesta che modifica; OCR con MolScribe 1.1.1 in un ambiente isolato (`tools/ocr`, torch CPU 2.5.1, timeout 60 s). Test end-to-end: Playwright con il Chrome di sistema, 10 file di specifiche, 30 test.

## 15.4 Client HTTP e server MCP

`ksmech` è una CLI sopra `KsmechClient` (header `X-Ksmech-Via: cli`; comandi `health`, `meta`, `predict`, `jobs`, `check`, `pool`, `molecule`, `depict2d`, `ocr`, `ml`, `embedding`, `curate`, `browse`; exit 0/1/2). `ksmech-mcp` (FastMCP, `mcp==1.30.0`) espone 20 strumenti sopra lo stesso client, registrati

in base alle capability di `/api/meta`; tutti in sola lettura tranne `predict_mechanism`. L'API HTTP è l'unico contratto.

# 16 Lavoro futuro e questioni aperte

## 16.1 Questioni aperte immediate

- **Rieseguire `joint` sulla RTX 5090** (`ONLY=joint_r`, circa 2 ore) per decidere i nove bracci addestrati del passo 9 di P9.
- Il braccio con la maschera dei metalli $d$ (iv) e i flag delle soglie (xi: $\tau_{\text{atom}}$, fusione spettrale), che richiedono ricostruzioni dello store o dei token.
- Il costo di circa 2 punti dell'addestramento congiunto; `pm_v19` a un solo seed.
- Le parti di test di `closed_base` (per le configurazioni open-shell) e di RC non ancora lette.
- Casi di dominio lasciati all'utente: l'addotto $Et_3B$–$O_2$ (`SPIN_NOT_CONSERVED`), la legge per i singoletti che collassano (non carbeni), le esclusioni di ozono e persolfato, il costo dell'etichetta di `reactmech-7172766faa0ff062`, l'estensione dello snapshot ReactMech per molecola (224 ritorni di P5 sono `SPLIT_CONFLICT`), uno split ReactMech per classi.
- Calibrazione: early stopping o regolarizzazione sulla NLL (sovra-confidenza dopo l'epoca 10–14); il modello non ha dropout.
- Infrastruttura: caricare i checkpoint dei passi 8 e 9 sul dataset privato, pubblicare `main`, decidere se passare a `tblite-anc` come ottimizzatore di default (ricalcolo completo del QM), regola dei plateau anche per i gusci chiusi, un arbitro DFT per gli stati di spin.
- Differiti dal piano open-shell: D1 (il CSV ufficiale di RMechDB), D2 (spGFN2 per $u > 0$, ablazione dopo P9), D3 (catene radicaliche USPTO di FlowER e una testa `END_MECH` addestrata); le fasi dell'app web S1–S3.
- Pre-addestramento dell'encoder su QMugs: dati in preparazione, obiettivo non ancora definito dalla specifica.

## 16.2 I token orbitali servono al modello? (`docs/todo.md`)

L'importanza per permutazione (sezione 13.11) suggerisce che le colonne dei token orbitali e l'insieme spettrale non portino informazione marginale. Piano: un interruttore di `ModelConfig` per togliere le colonne orbitali e uno per l'insieme spettrale; tre bracci sullo split per molecola, `closed_base` val, 3 seed, preset general: (a) riferimento, (b) senza colonne orbitali, (c) senza colonne orbitali né insieme spettrale, eventualmente (d) senza embedding ereditati; confronto appaiato con la regola U20.

## 16.3 Reti di reazione (`docs/reaction_network_plan.md`)

**Motivazione.** Il metossido con l'acetone dà sia l'emichetale sia l'enolato; in acqua la deprotonazione è in salita di circa 4 unità di $pK_a$ (acetone 19,3 contro metanolo 15,5), in DMSO in discesa (26,5 contro 29,0). 474 pool di PMechDB hanno più di un prodotto (973 passi, 7,7%): sui loro 36 campioni di `closed_base` val il top-1 colpisce l'etichetta propria nel 44–50% dei casi e *un* esito registrato nel 72–83%. Oggi la rete vive nell'app web, cresce con un beam cieco rispetto al bersaglio, non ha stop (la testa `END_MECH` non è addestrata) né passi inversi, e il QM non contiene energie libere.

**Decisioni.** La massima verosimiglianza resta la loss per esiti distinti (una loss «any» solo per insiemi definiti da leggi); lo stop è termodinamico, agisce sugli stati e non taglia mai un passo in salita; sono consentiti pacchetti come xtb, CREST, un atom mapper.

**Elementi del piano (in ordine).**

**A** valutazione multi-riferimento: `rank_any` per chiave del pool prodotto, `compare -metric own|any`. È in corso su un ramo di lavoro (`claude/network`): `DECODE_VERSION` 5, con la misura di 901 passi di training PMechDB in 436 pool multi-esito, e il punto d'ingresso `model.decode.matrix_logp` per la $\log P$ di una $\boldsymbol{R}$ data.

**B** energia libera: `xtb -hess`, quasi-RRHO, solvente come parametro, CREST per le specie raggiunte; $G(\text{pool}) = \sum G + RT \ln(c/c^0)$.

**C** il nucleo della rete nel motore (`ksmech.network`).

**D** ricerca verso un prodotto: arrivo per chiave d'identità o per scheletro a meno di protoni; $\Delta\boldsymbol{B} = \boldsymbol{B}_{\text{target}} - \boldsymbol{B}_{\text{corrente}}$ da una mappatura atomica.

**E** senza prodotto: archi inversi $-\boldsymbol{R}$, bilancio dettagliato con peso $s_{ij} \exp(-\Delta G_{ij}/2RT)$, assorbimento di una catena di Markov.

**F** stop termodinamico a temperatura fisica: `SINK_AT_T` quando ogni passo sale più di $\delta = RT \ln(1/\varepsilon)$ + margine (con $\varepsilon = 1\%$: 2,7 kcal/mol a 25 °C, 3,5 a 110 °C).

**G** dati di pathway: il benchmark umano (solo valutazione), le catene ReactMech `ID_CRM` con etichette di stop, ORD/USPTO, mech-USPTO-31K.

**H** modello inverso (da prodotto a reagente).

**I** ricerca su grafo: A* con costo $-\log p$ ed euristica $\Delta\boldsymbol{B}$ residuo, ricerca bidirezionale (con $b = 20$, $d = 6$: circa 16 000 espansioni invece di 64 milioni), $k$ cammini migliori, teoria dei cammini di transizione, espansione per massa di probabilità.

**J** grafo delle specie AND-OR (iper-cammini), analisi del flusso atomico.

**K** retrosintesi meccanicistica da un prodotto soltanto (Retro*, proof-number search o MCTS con il modello inverso).

**Note di ricerca.** Una rassegna dell'utente (`reports/Deep learning ricerca combinatoria.md`, 8 novembre) su deep learning e ottimizzazione combinatoria conclude che, per KSMECH, il floor non addestrato del fattore di gap è il controllo che il modello appreso deve battere *dentro la stessa ricerca* a parità di calcolo, che una ricerca best-first alla Levin è un'alternativa garantita al beam, e che conviene un training consapevole della ricerca (LaSO, DAgger).

# A Codici di motivo

Ogni passo o molecola esclusi sono contati sotto un codice. Elenco dei principali.

**Tabella A.1:** Codici di motivo per livello.

| Codice | Significato |
|---|---|
| *Ingest e validazione* | |
| `SANITIZE_FAILED` | valenze non valide per RDKit (complessi ate ipervalenti) |
| `UNBALANCED`, `MAP_NOT_BIJECTIVE` | atomi pesanti non bilanciati; mappe non biiettive |
| `BOND_NOT_FOUND`, `ARROWS_MISMATCH` | legame sorgente assente; le frecce non riproducono i prodotti |
| `RADICAL_ARROW` | freccia a un elettrone in una sorgente polare (0 casi) |
| `DUPLICATE_STEP` | riga FlowER con la stessa reazione canonica (553) |
| *Dominio* | |
| `RESONANCE_STEP` | interconversione di risonanza (diventa coppia di risonanza) |
| `NULL_STEP`, `NONCONJUGATED_SHIFT` | lati identici; stessa chiave senza flusso bilanciato |
| `METAL_PLACEHOLDER` | metallo disegnato senza leganti |
| `CHARGE_UNREPAIRED` | errore di carica che nessuna legge ripara |
| `RADICAL_DRAWN` | atomo radicalico in un passo polare fuori dalle leggi dei gusci aperti |
| `MULTIPLICITY_UNTABULATED`, `_AMBIGUOUS` | nessun conteggio di spin dalle tabelle |
| `SPIN_NOT_CONSERVED` | $u$ non conservato dal passo |
| `NOT_ELEMENTARY` | voce atomica dispari in un passo polare, o classe non elementare |
| `SOMO_NOT_INTEGRAL` | SOMO non intera (smearing) in un frammento a guscio aperto |
| `MULTI_RADICAL_SPECIES` | specie non metallica con $u \geq 2$ che non è un caso di Hund |
| `SPIN_CENSUS_REFUSED` | specie che il censimento di spin tiene fuori dal QM |
| `SET_STEP`, `DISSOCIATIVE_ET_STEP` | trasferimento di singolo elettrone; trasferimento dissociativo |
| `SPLIT_CONFLICT` | passo nuovo che unisce gruppi congelati in parti diverse |
| `LEAK_GUARD` | passo rmechdb con una specie della validazione/test PMechDB (run congiunte) |
| *Classi di dominio (ammesse, contate)* | |
| `REPAIRED_DATIVE`, `_V1`, `_A1`, `_BL` | riparazioni conservate |
| `METAL_SUSPECT` | metallo 3d lasciato $d^1$–$d^9$ in un frammento pari (flag) |
| `PAIRED_DRAWING`, `HIGH_SPIN_DRAWING` | ridisegni di spin |
| `RADICAL_SPECTATOR`, `RADICAL_REACTIVE` | passi polari con un radicale ammesso |
| `CARBENE_SINGLET`, `OSM_CU2`, `OSM_FE3` | carbeni di singoletto; Cu(II) e Fe(III) a guscio aperto |
| *QM* | |

| Codice | Significato |
|---|---|
| OPT_NOT_CONVERGED, SCF_NOT_CONVERGED | limiti di iterazione (500/250, retry 2 000/1 000) |
| TIMEOUT, EMBED_FAILED | oltre 900 s (retry 3 600); embedding impossibile |
| CONNECTIVITY_CHANGED, RESTRAINED_GEOM | legame rotto oltre 1,3×; geometria con vincoli (avviso) |
| SPIN_PARITY, BOND_FORMED | numero di elettroni incompatibile; l'ottimo forma un legame |
| WORKER_CRASH, FUKUI_SCF_FAIL | crash del worker; SCF a $N \pm 1$ fallito (avviso) |
| *Campioni, training, decodifica* | |
| MISSING_TOKENS, NO_BOND_INDEX | token o indici di Wiberg mancanti |
| STALE_ARTIFACT | artefatto obsoleto o da albero sporco |
| TOO_LARGE | più di 1 024 token |
| DROPPED_DOMAIN_CLASS | classe tolta per ablazione |
| NONFINITE | run fermata per valori non finiti (exit 3) |
| DECODE_DROPPED:* | CHARGE_NOT_CONSERVED, OCTET_EXCEEDED, FORMAL_CHARGE_RANGE, ODD_OFF_BROKEN_BOND, CLOSED_MINUS1, RESONANCE_ONLY_STEP… |
| DECODE_MERGED, DECODE_ANCHOR_UNCOMPLETED | completazione già trovata; ancora senza completazione |

# B Versioni

**Tabella B.1:** Costanti di versione nel codice attuale.

| Costante | Valore | Storia |
|---|---|---|
| specifica | v10 | v3 → v7.2 (IBO) → v8 (***R***) → v8.1 (chiave sigma) → v9 (niente stage B) → v9.1/9.2 → v10 |
| `SAMPLE_VERSION` | 19 | tabella 9.1 |
| token `SCHEMA_VERSION` | 10 | 7→8 (degeneri), 9 (IBO), 10 (extra, P0a) |
| record QM `SCHEMA_VERSION` | 1 | invariato |
| `FLOOR_VERSION` | 4 | 2 (P0), 3 (celle), 4 (`own_break`) |
| `DECODE_VERSION` | 4 | 2 (P0), 3 (caps), 4 (schermo); 5 in corso (`rank_any`) |
| `FROZEN_SPLIT_SCHEMA` | 1 | |
| indice QM `INDEX_VERSION` | 2 | 2 dal P4 (spin) |
| `SPIN_RULE_VERSION` | `spin-rule-v2` | |
| tabelle di spin | `spin_table_v2`, `spin_theory_v2`, `metal_v1` | |
| `OPEN_SHELL_RULE` | `occ-plateau-v1` | P2b |
| `LGBM_VERSION`, `TRAIN_VERSION`, `IMPORTANCE_VERSION` | 1, 1, 1 | |

# C Comandi tipici

```
uv sync                                     # ambiente (torch ROCm)
uv sync --no-group rocm --group cuda        # host NVIDIA; poi export UV_NO_SYNC=1
uv run pytest -n 14 -m "not realdata"       # test di unita'
uv run pytest -m realdata                   # test su dati reali

uv run ksmech-data ingest --source pmechdb
uv run ksmech-qm run --source pmechdb --workers 8
uv run ksmech-qm bond-index
uv run ksmech-orbitals build --source pmechdb --workers 4
uv run ksmech-arrows samples --source pmechdb --workers 8
uv run ksmech-physics gbdt --source pmechdb --split molecule
uv run ksmech-eval floor --source pmechdb --split molecule

HSA_OVERRIDE_GFX_VERSION=10.3.0 uv run ksmech-train fit --run NOME \
    --preset general --no-resonance-draws --seed 0
uv run ksmech-train fit --run joint_s0 --preset open_shell \
    --source pmechdb+rmechdb --split pmechdb=molecule,rmechdb=rc_molecule \
    --val-cohort pmechdb=closed_base --no-resonance-draws

uv run ksmech-eval decode --run NOME --part val --beam 5 --cohort closed_base
uv run ksmech-eval compare --a A_s*/decode_hits_*.json --b B_s*/decode_hits_*.json
uv run ksmech-eval importance --run NOME --part val
uv run ksmech-serve --port 8000
```

# D Glossario

**Ancora**
la voce di $\boldsymbol{R}$ attorno a cui è fattorizzata la previsione: un legame che si forma (+1) o, se nessuno, uno che si rompe (−1).

**Classe parsimoniosa**
l'insieme delle matrici di reazione equivalenti (modulo risonanza e simmetria) con meno cariche formali e meno voci: l'etichetta di un passo.

`closed_base`
la coorte di riferimento a guscio chiuso (PMechDB, split per molecola).

**Coppia di risonanza**
un passo che interconverte due disegni della stessa specie, escluso dal bersaglio e conservato come esempio di equivalenza.

**Floor fisico**
l'ordinamento delle ancore per $|G^{\mathrm{atom}}|$ senza parametri appresi.

**full-R top-$k$**
frazione di campioni per cui una delle prime $k$ completazioni del beam è una variante della classe di etichette.

**GENERAL / LEGACY**
preset del modello: tutte le celle con flag ed extra orbitali, oppure gli input della v17.

**Lacuna (hole)**
il gemello virtuale di un token a guscio aperto, che fa da accettore.

**Pool**
il multiinsieme di tutte le specie presenti, scritto come una matrice $\boldsymbol{B}$.

**Release A / B**
gli insiemi di fasi P1–P5 e P6–P8 (+P9) del piano open-shell.

**U20**
la regola di tolleranza dei confronti appaiati.